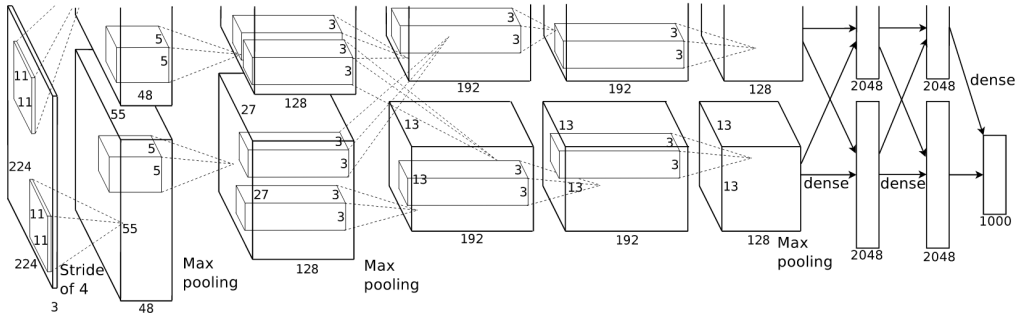
Representation Learning
Attention and Transformers

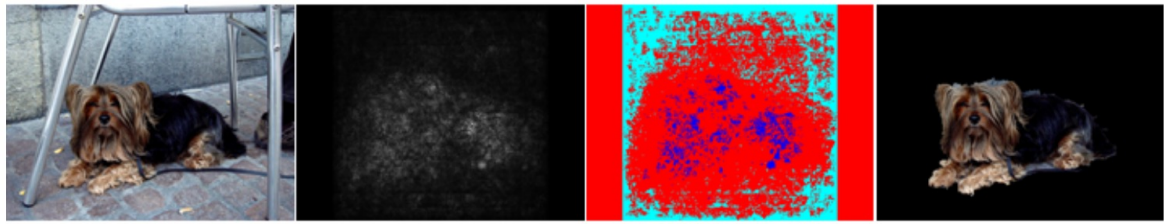
Loïc Le Folgoc
LTCI, Télécom Paris, IP Paris
loic.lefolgoc@telecom-paris.fr

Some history in computer vision

- AlexNet [Krizhevsky, 2012]:
  - The rise of CNNs in computer vision
  - Greatly outperformed traditional approaches in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC)



- Saliency maps [Simonyan, 2013]:
  - Visualizing where CNNs look to predict a class
  - = post-hoc **attention maps**



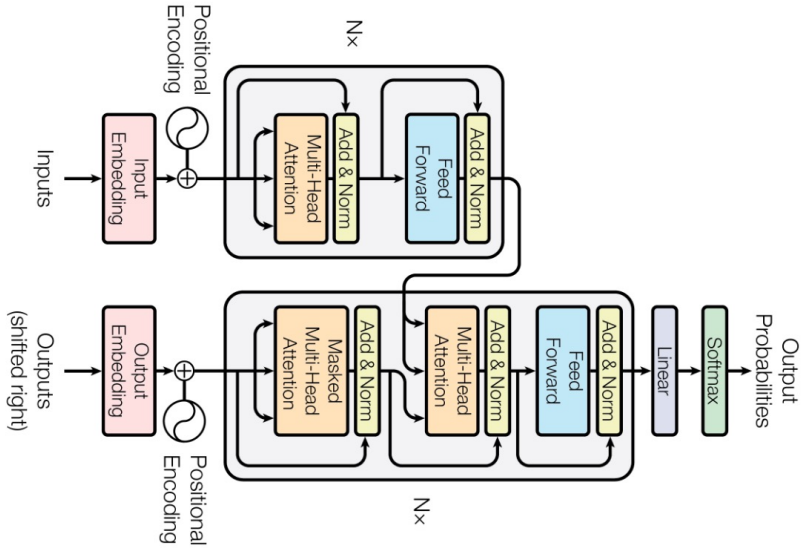
- Attention mechanisms with CNNs/RNNs
  - For image captioning [Xu, 2015]
  - For visual Question Answering [Yang, 2016]
  - …



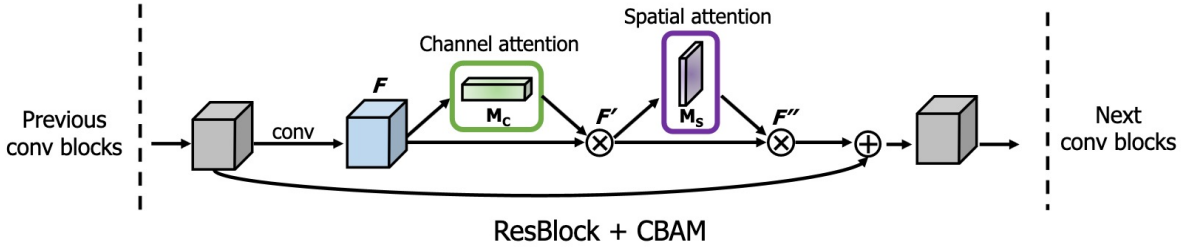
[Krizhevsky, 2012] **Imagenet classification with deep convolutional neural networks**, Krizhevsky, A. et al., *NIPS*, *2012*
[Simonyan, 2013] **Deep inside convolutional networks: Visualising image classification models and saliency maps**, Simonyan, K. et al., *arXiv 2013*
[Xu, 2015] **Show, attend and tell: Neural image caption generation with visual attention**, Xu, K. et al., *ICML*, *2015*
[Yang, 2016] **Stacked attention networks for image question answering**, Yang, Z. et al., *CVPR*, *2016*

Some history in computer vision

- **Transformers** [Vaswani, 2017]:
  - First architecture fully based on **self-attention**
  - Revolutionizes **Natural Language Processing**, outperforms RNNs and gives rise to main **Large Language Models** in the following years



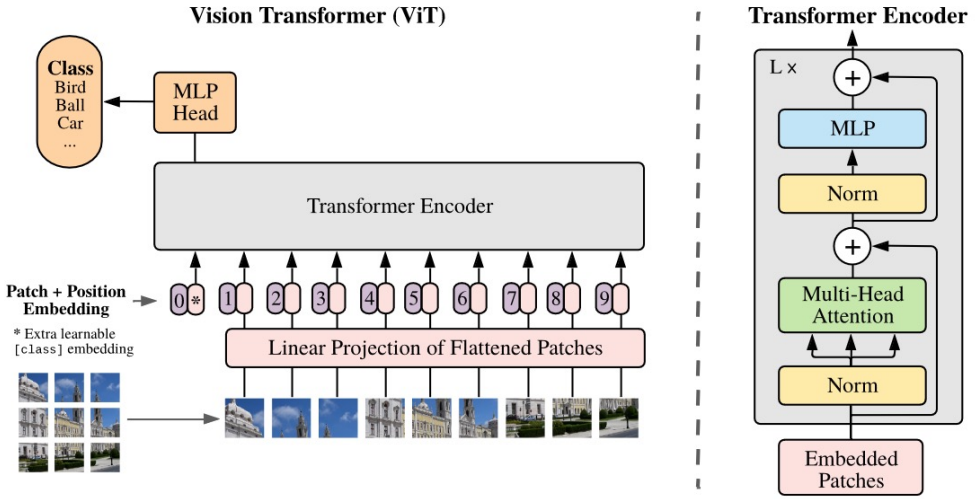
- 2018: trainable attention in CNNs
  - Trainable **attention blocks** (SE blocks*, CBAM**, etc.) integrated in CNNs to help them **attend to relevant information**



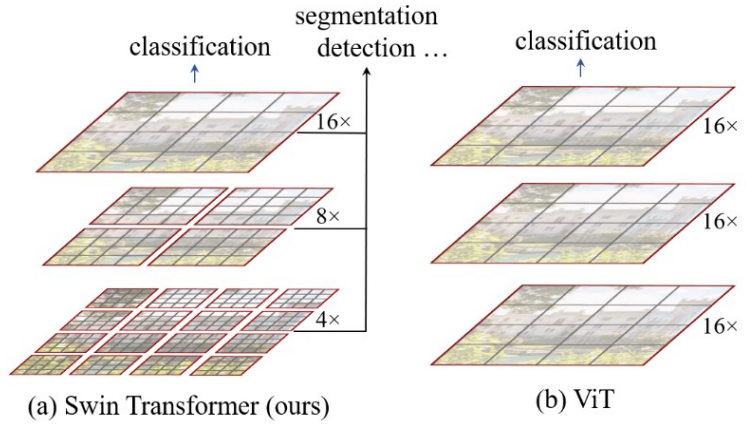
[Vaswani, 2017] **Attention is all you need**, Vaswani, A. et al., *NeuRIPS, 2017*
* **Squeeze-and-excitation networks**, Hu, J. et al., *CVPR, 2018*
** **Cbam: Convolutional block attention module**, Woo, S. et al., *ECCV 2018*

Some history in computer vision

- **Vision Transformers** [Dosovitskiy, 2021]:
  - First **computer vision model** purely adapted from the concept of transformer
  - State-of-the-art on ImageNet when pretrained from large datasets
  - Start of **new paradigm** ≠ CNNs



- 2021-today:
  - New vision transformer architectures*
  - Transformer backbones for all vision tasks
  - New ways to train transformers
  - Fast/efficient attention, flexible attention…
  - Foundation models (transformers)



[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*
* **Swin transformer: Hierarchical vision transformer using shifted windows**, Liu, Z. et al., *CVPR*, *2021*

ImageNet SOTA (End 2024)

Model	Top 1 Accuracy	Number of params	GFLOPs	Top 5 Accuracy	Extra Training Data	Paper	Code	Result	Year	Tags
BASIC-L (Lion, fine-tuned)	91.1%	2440M			×	Symbolic Discovery of Optimization Algorithms			2023	Conv+Transformer, ALIGN, JFT-3B
CoCa (finetuned)	91.0%	2100M			×	CoCa: Contrastive Captioners are Image-Text Foundation Models			2022	ALIGN, Transformer, JFT-3B
Model soups (BASIC-L)	90.98%	2440M			×	Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time			2022	ALIGN, JFT-3B, Conv+Transformer
Model soups (ViT-G/14)	90.94%	1843M			×	Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time			2022	JFT-3B, Transformer
ViT-e	90.9%	3900M			×	PaLI: A Jointly-Scaled Multilingual Language-Image Model			2022	Transformer, JFT-3B
CoAtNet-7	90.88%	2440M	2586		✓	CoAtNet: Marrying Convolution and Attention for All Data Sizes			2021	Conv+Transformer, JFT-3B
ViT-G/14 (Lion)	90.71%	1843M			×	Symbolic Discovery of Optimization Algorithms			2023	Transformer, JFT-3B

Pure transformers or mixed architectures top the benchmark

Transformers & Self-Attention

Transformers

- Transformers are architectures that **process** and modify **sequences of tokens**

Sentence: *"hi, we are doing deep learning"*

Word tokens: *'hi' ',' 'we' 'are' 'do' '##ing' 'deep' 'learn' '##ing'*

- The process of transforming an input into tokens is called **tokenization**

Tokenization of images

- Images are divided into **non-overlapping patches**; each patch is a token



Image adapted from: wikipedia

Tokenization of images

- Images are divided into **non-overlapping patches**; each patch is a token

- The standard for vision transformers is to take 224×224 images as input
  - Images are then divided in $N = 14 \times 14$ patches of size $P \times P$, with $P = 16$

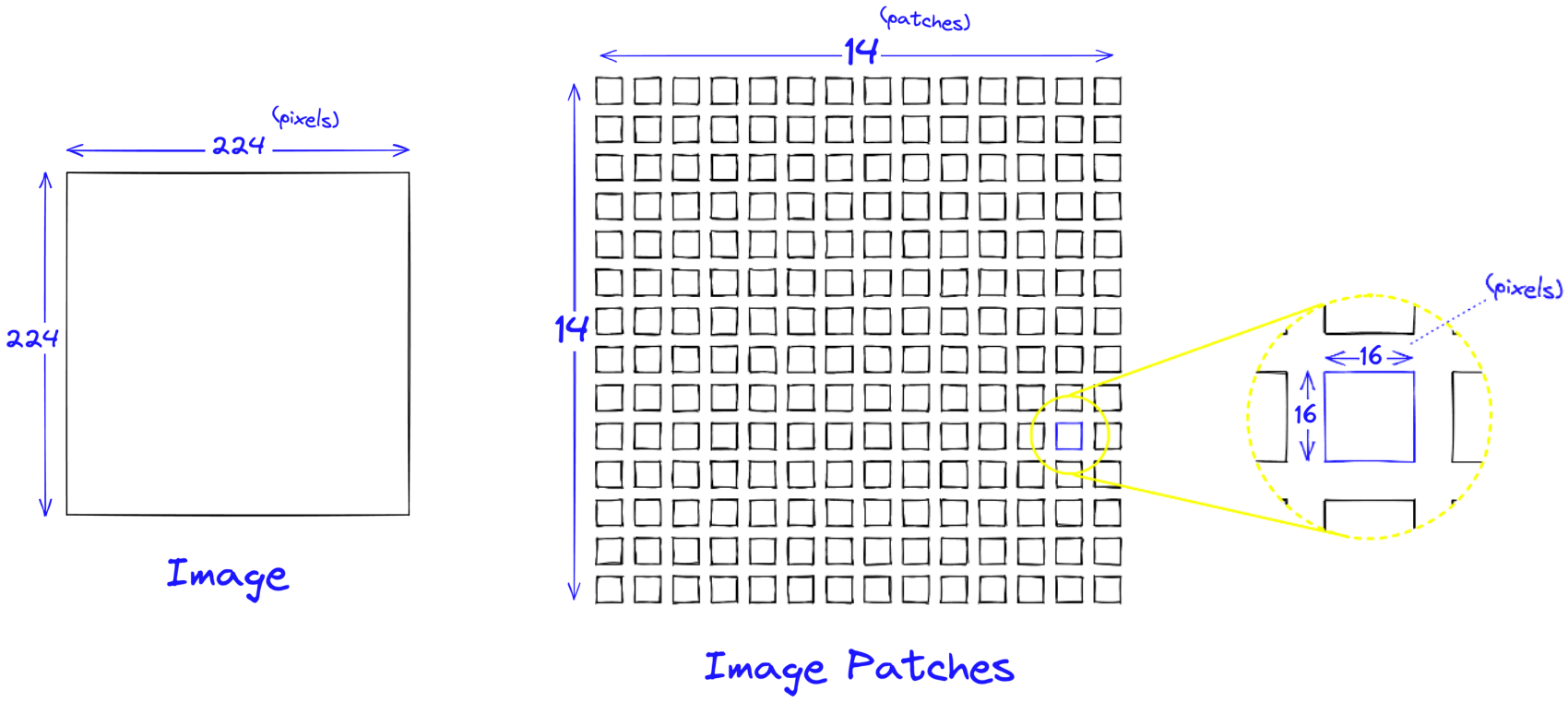


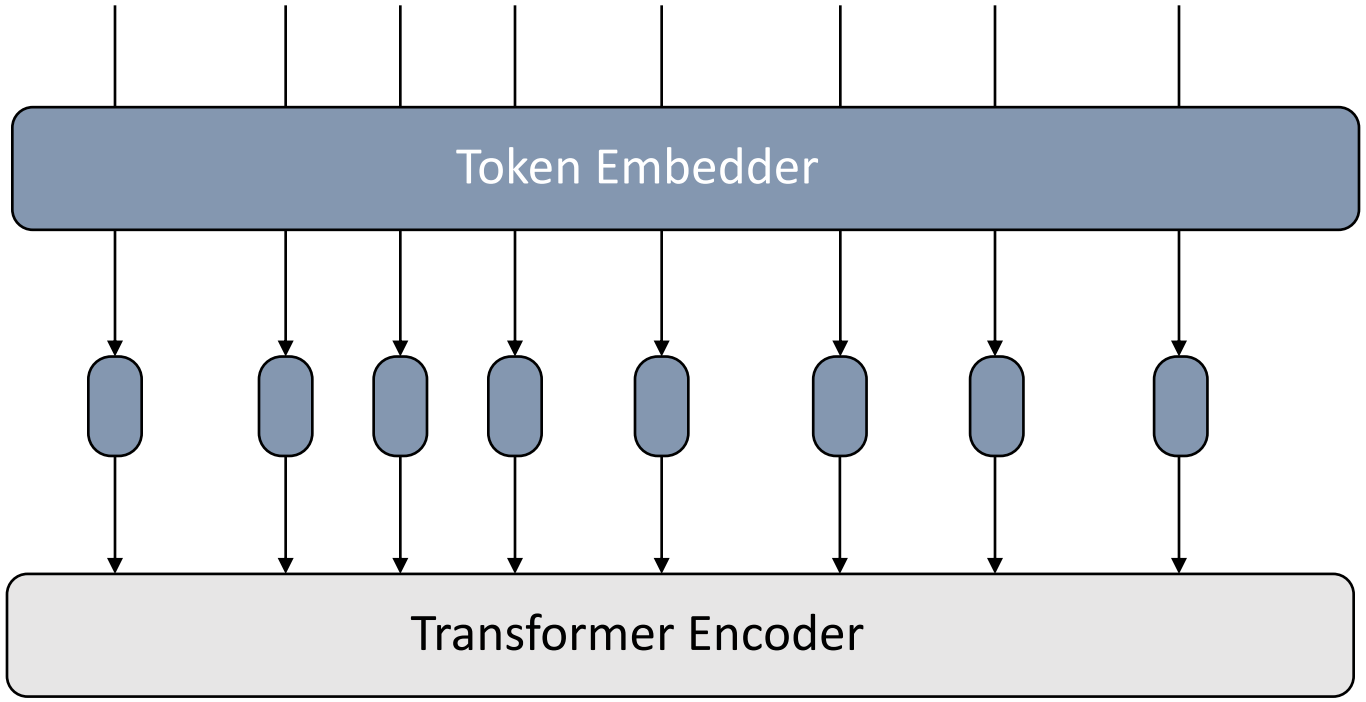
Image adapted from: pinecone.io

Transformers

- Tokens are **embedded in** $\mathbb{R}^D$ before being fed to the transformer encoder

Sentence: "hi, we are doing deep learning"

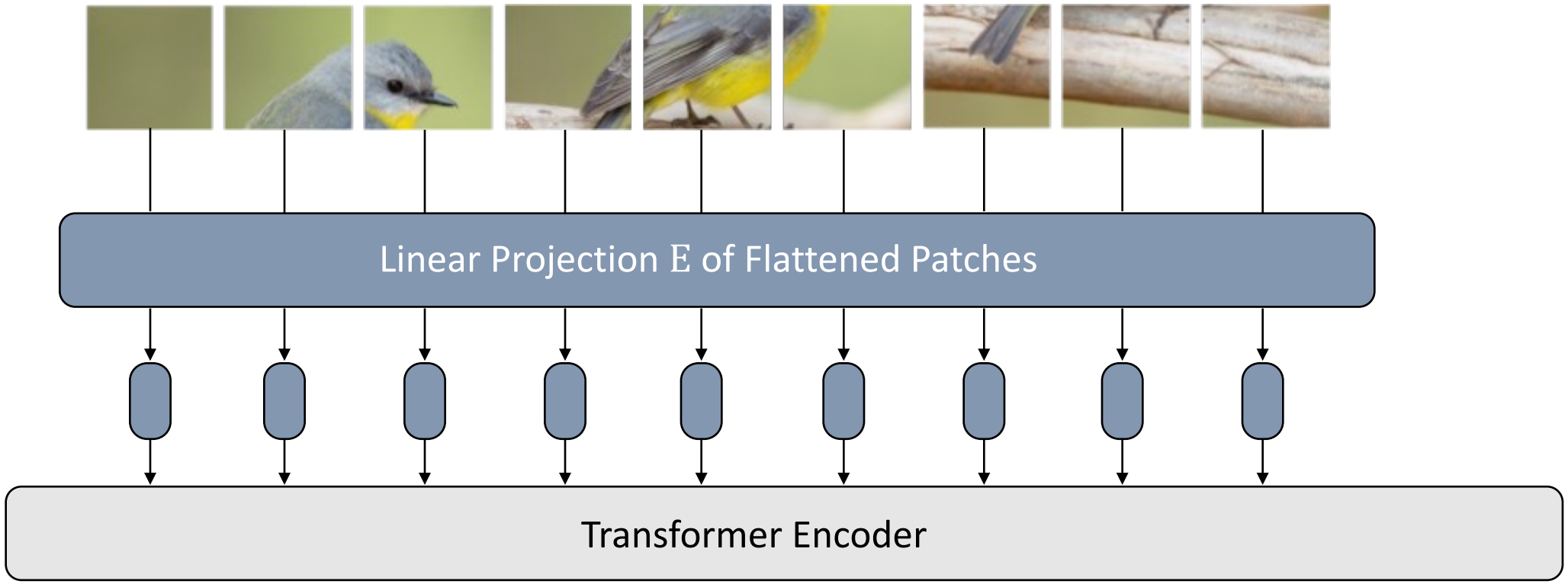
Word tokens: 'hi' ',' 'we' 'are' 'do' '##ing' 'deep' 'learn' '##ing'



Embeddings:

Vision Transformers

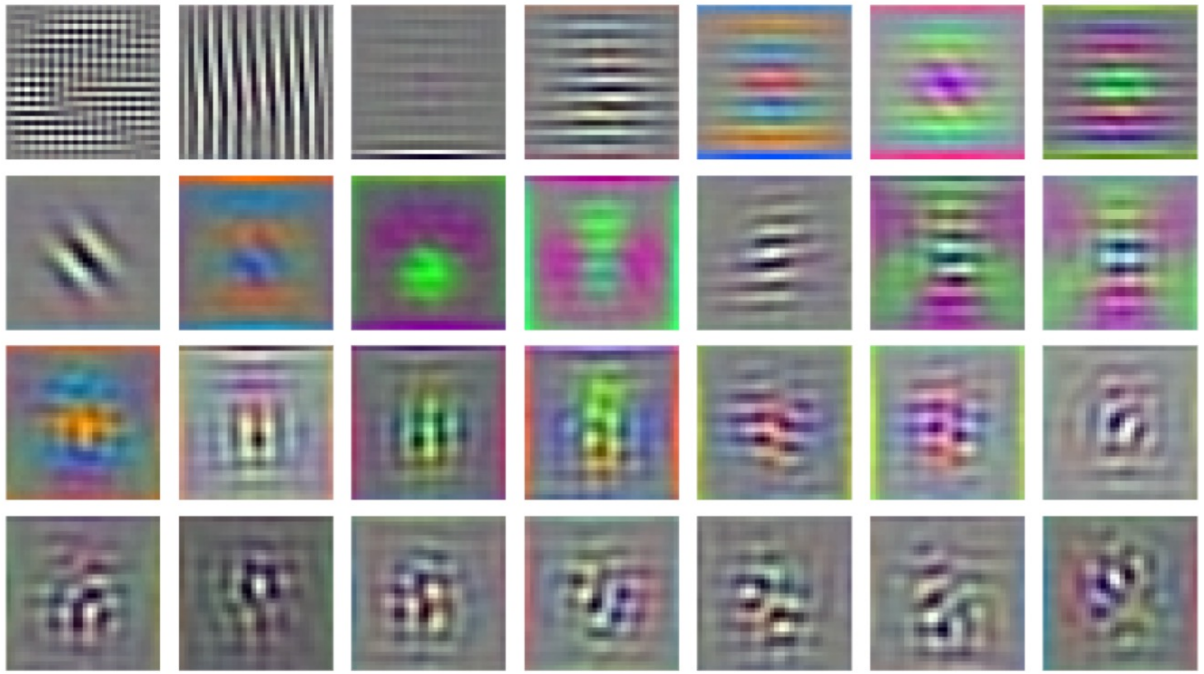
- Image patches are (linearly) **embedded in** $\mathbb{R}^D$ before being fed to the transformer encoder: $\mathrm{x}_p^i \mapsto \mathrm{E}^T \mathrm{x}_p^i$



Linear embedding vizualization

- In the base model the dimension D is the same as the input patch dimension, $D \coloneqq 16\times16\times3$

- We can visualize each column of E as a filter/patch whose dot products with input patches are computed
  - Below, the 28 first filters are shown



[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

The Transformer Encoder

- Before the transformer, these 3 instances of *"bank"* are embedded to the same vector in $\mathbb{R}^D$
- The transformer is able (via self-attention) to use the **context to disambiguate** the 3 embeddings

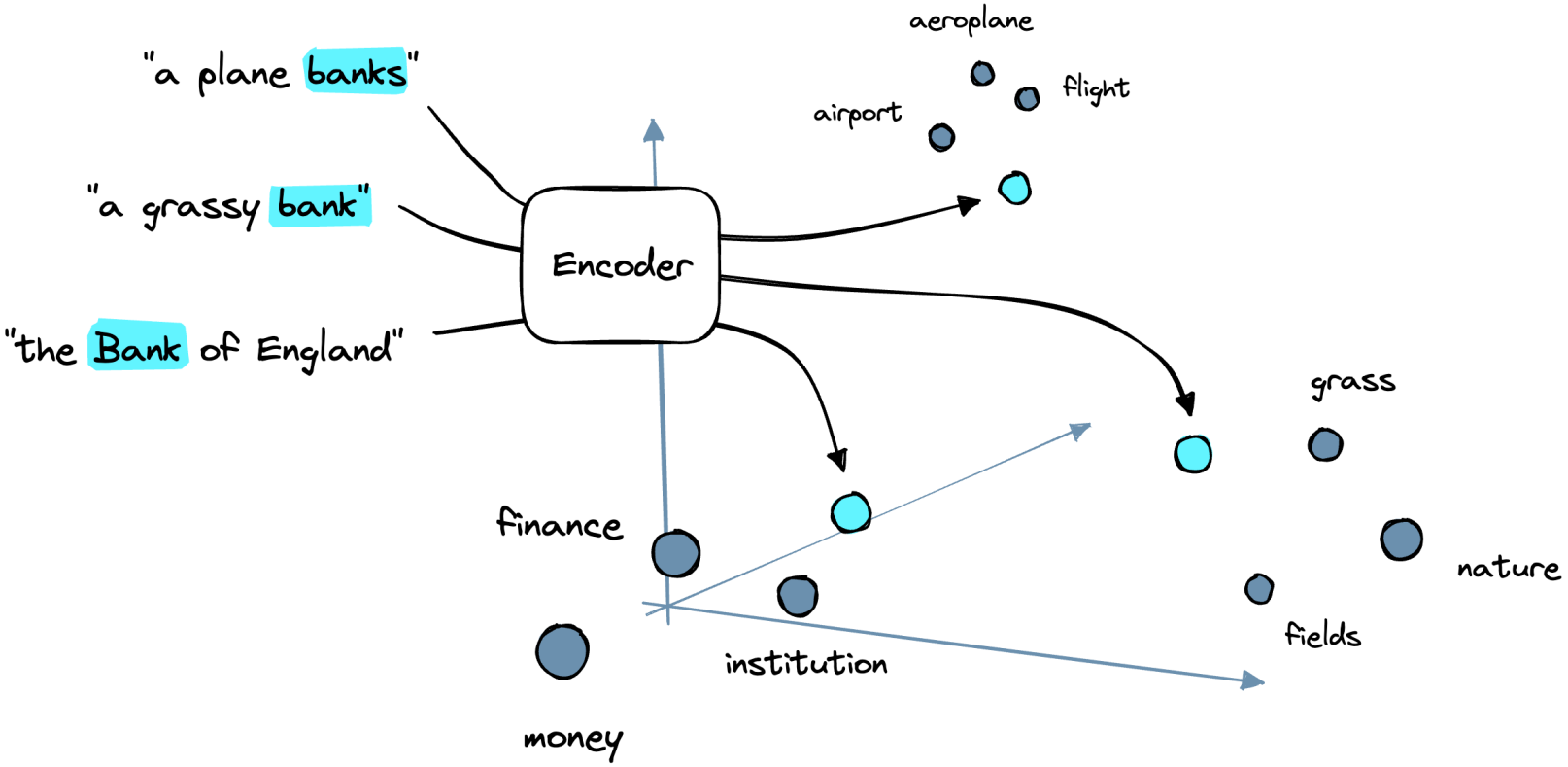


Image source: pinecone.io

The Transformer Encoder

- **Self-attention** allows to **incorporate context into** the token **embeddings**

"The animal didn't cross the street because it was too tired"

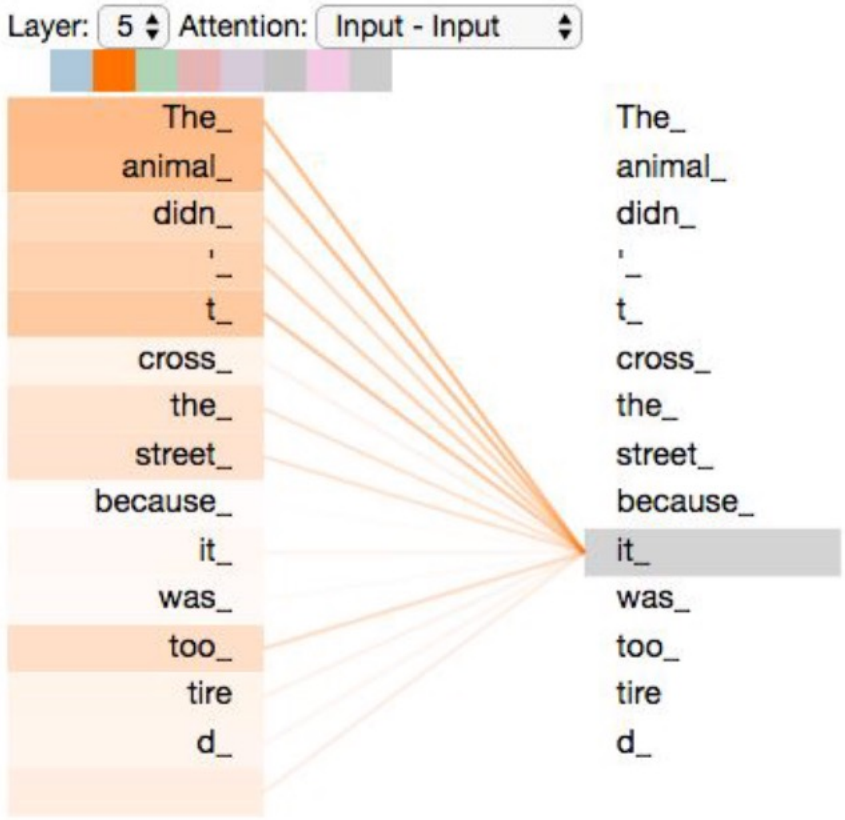
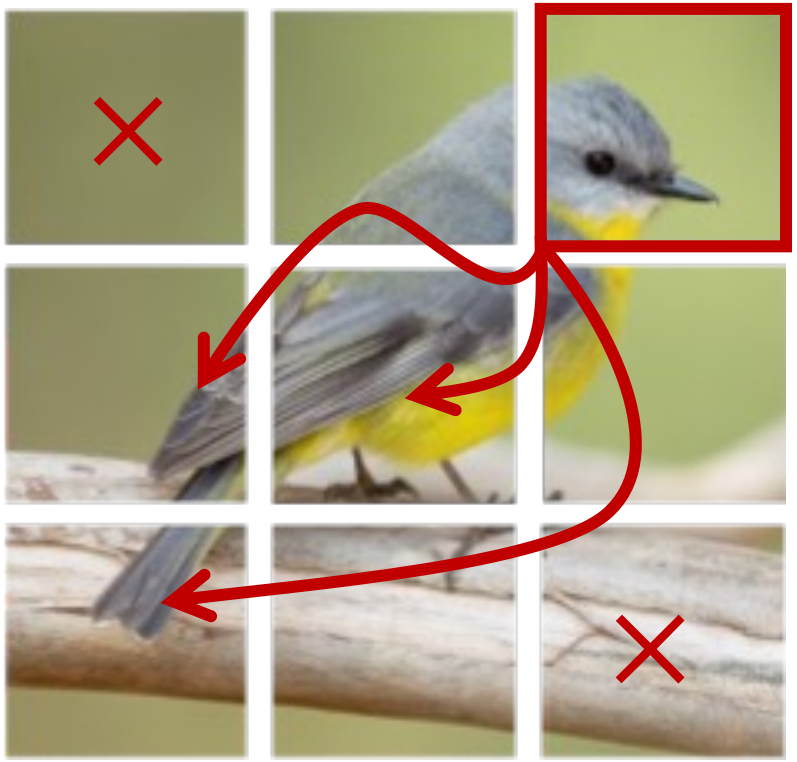


Image source: Anurag Arnab

The (Vision) Transformer Encoder

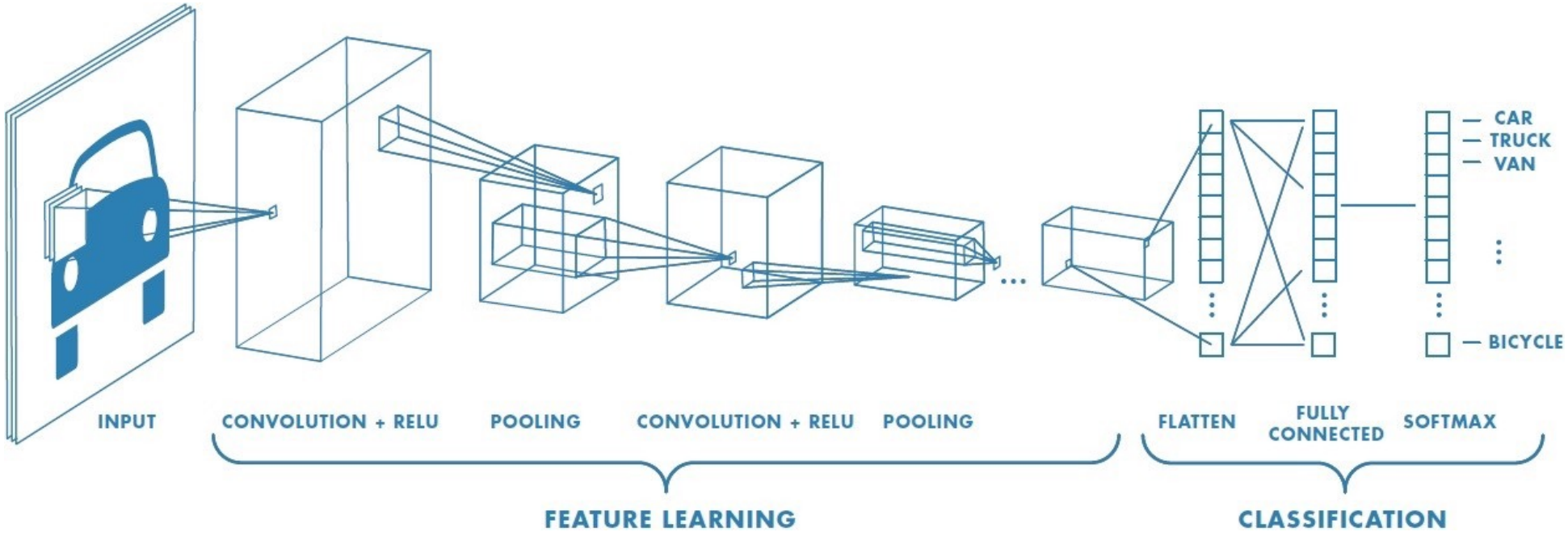
- **Self-attention** allows to **incorporate context into** the token **embeddings**



The (Vision) Transformer Encoder

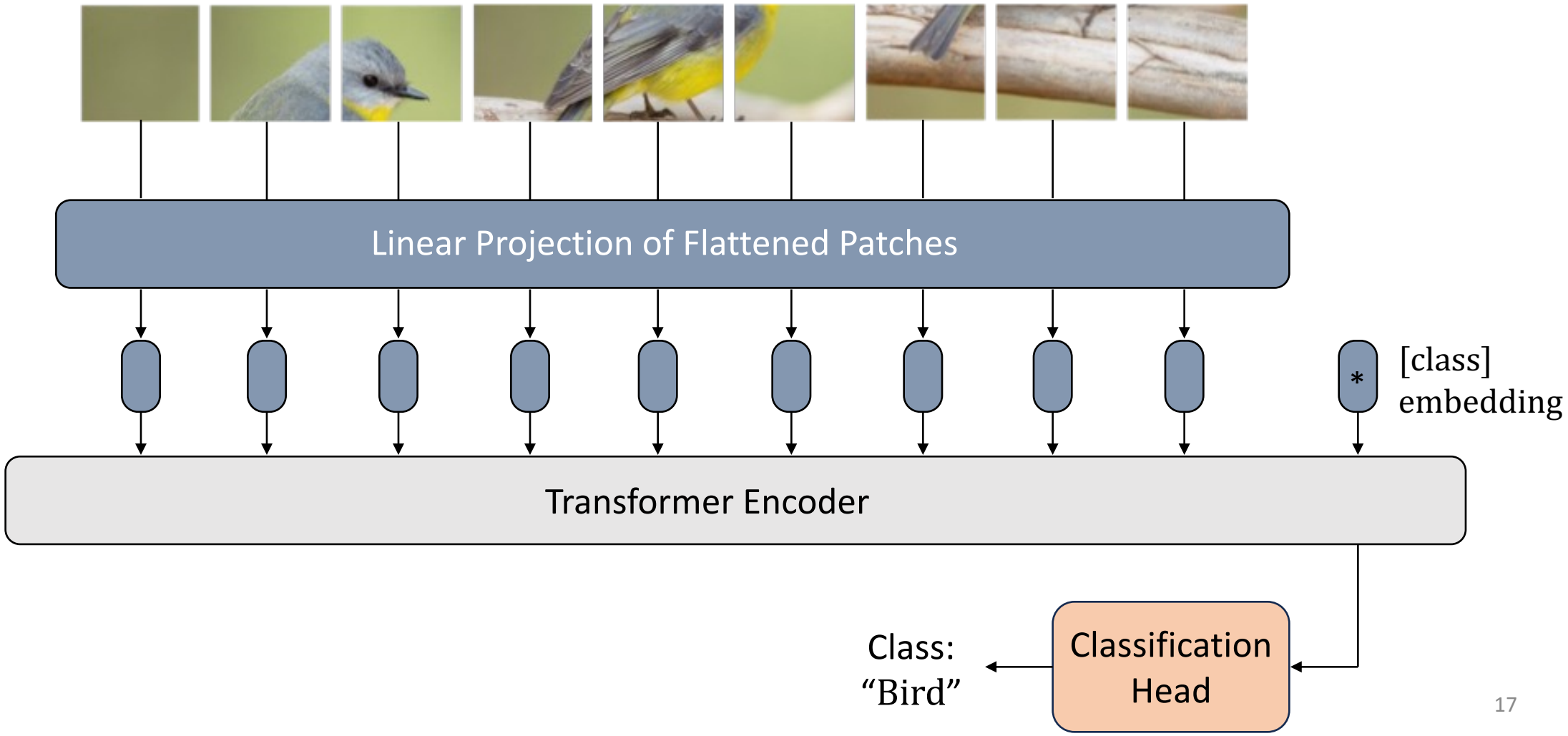
- **Self-attention** allows to incorporate context into the token embeddings
  - Self-attention involves **all tokens**: **global** inter-dependencies are **captured very quickly**

- This departs from CNNs, which build context progressively as the receptive field increases over many layers



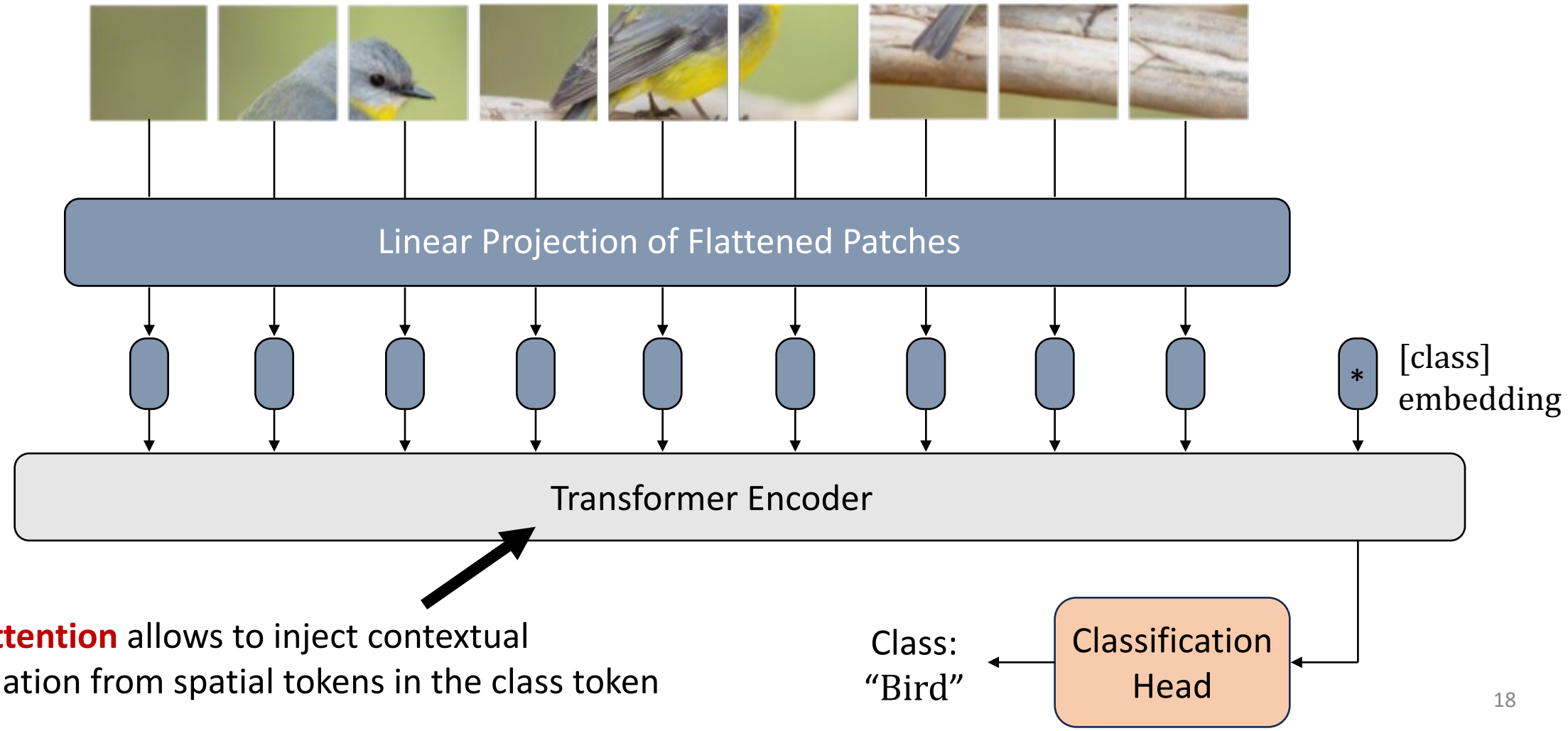
Vision Transformers for Classification

- We add a learnable **[class] embedding** in $\mathbb{R}^D$; a classification head is added on top of the final (transformed) class embedding to perform classification



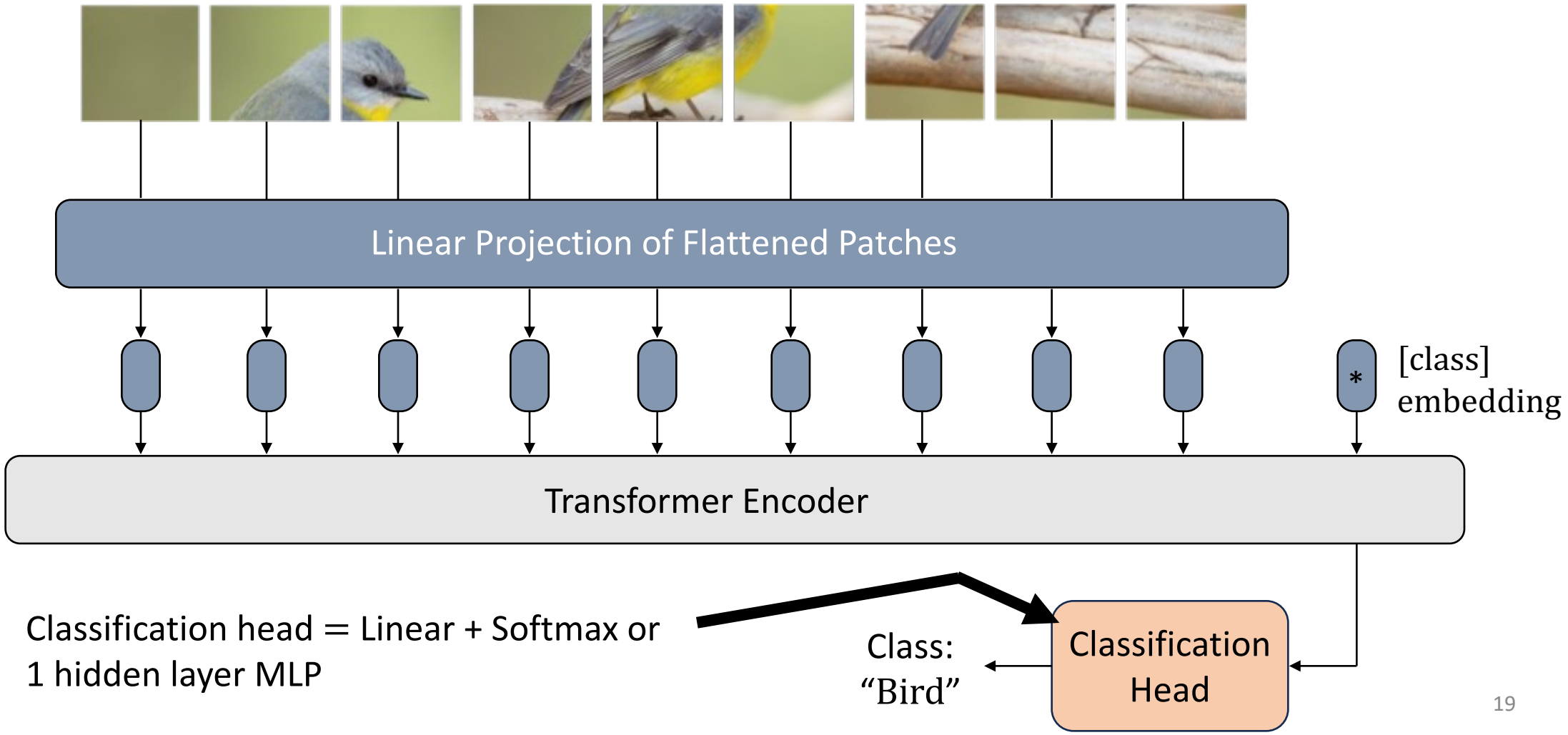
Vision Transformers for Classification

- We add a learnable **[class] embedding** in $\mathbb{R}^D$; a classification head is added on top of the final (transformed) class embedding to perform classification



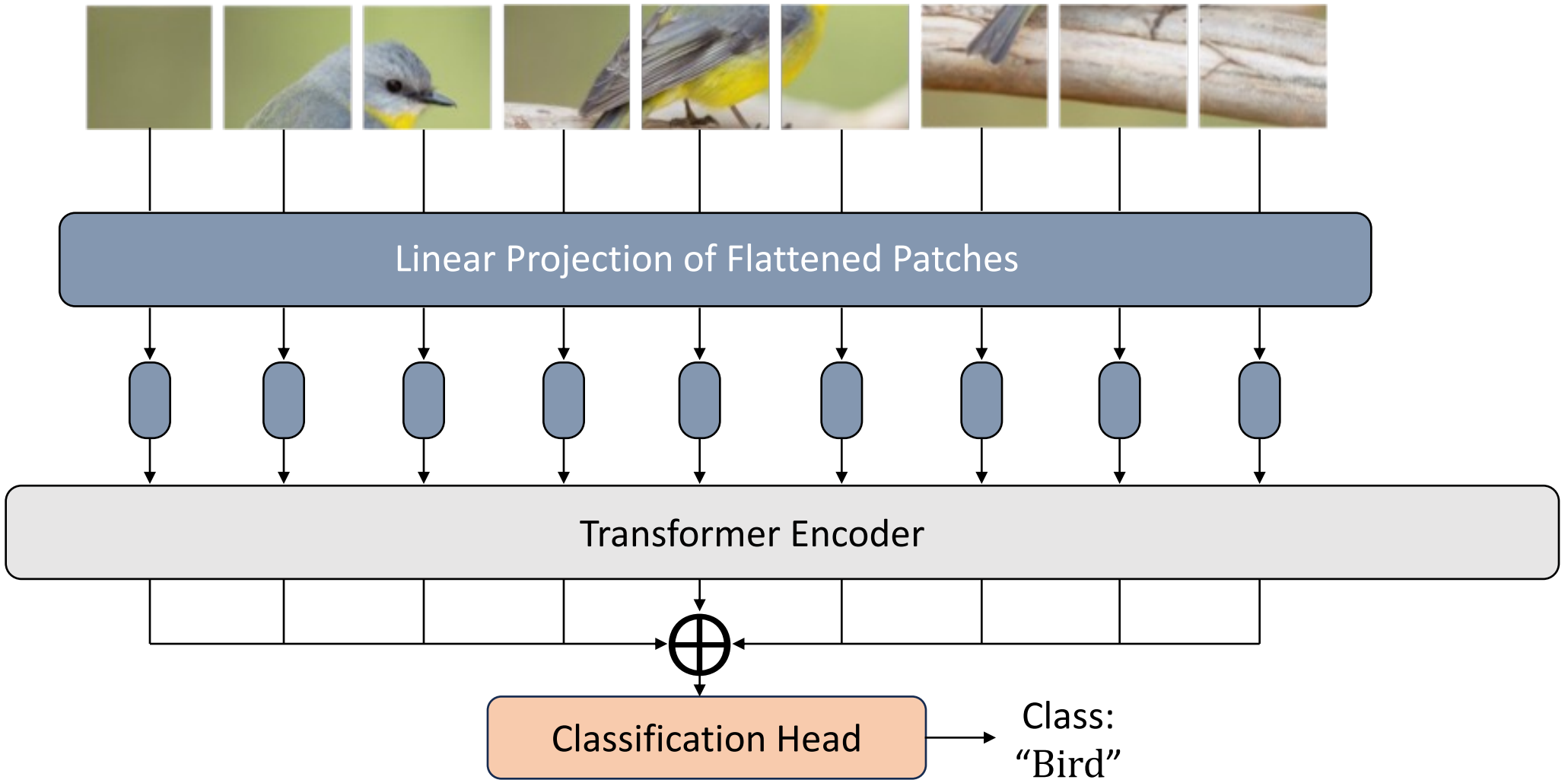
Vision Transformers for Classification

- We add a learnable **[class] embedding** in $\mathbb{R}^D$; a classification head is added on top of the final (transformed) class embedding to perform classification



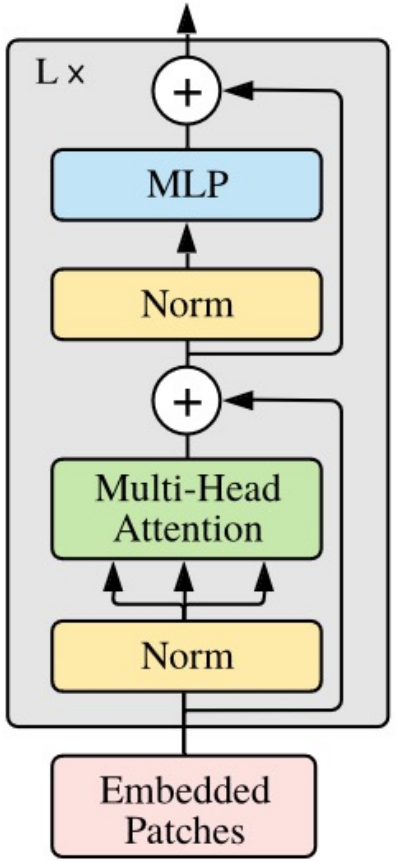
Vision Transformers for Classification

- A common alternative is to do global average pooling of the transformed embedded patches;
  - The classification head is added on top of the pooled embedding



The Vision Transformer Encoder

- The encoder is a succession of L identical blocks
- Each block modifies the token embeddings ($\mathbb{R}^{N\times D} \rightarrow \mathbb{R}^{N\times D}$), relying on two parts:



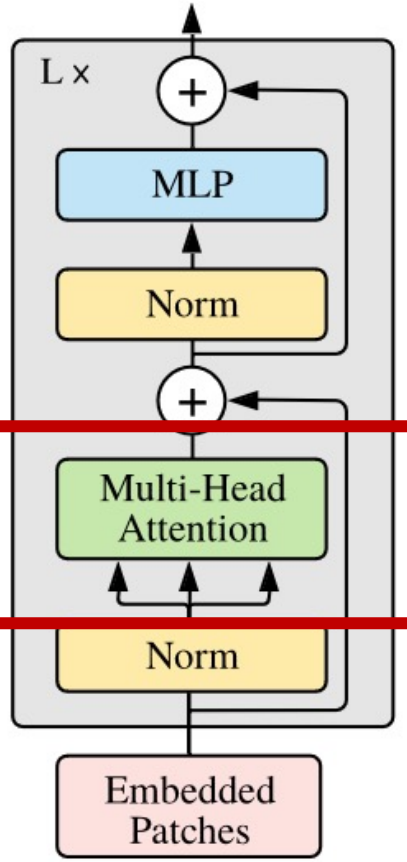
An **MLP** to **mix channel information** (i.e., from the D dimensions) within each token, without mixing information across tokens

Self-Attention to **mix spatial information** i.e., mix information across all tokens

Image adapted from: **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

The Vision Transformer Encoder

- The encoder is a succession of L identical blocks
- Each block modifies the token embeddings ($\mathbb{R}^{N\times D} \rightarrow \mathbb{R}^{N\times D}$), relying on two parts:



An **MLP** to **mix channel information** (i.e., from the D dimensions) within each token, without mixing information across tokens

Self-Attention to **mix spatial information** i.e., mix information across all tokens

Image adapted from: **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

Self-Attention

- Self-attention is a form of **dot-product attention** based on a **“query, key, value” formulation**

- It is called *self*-attention because the queries, keys, and values are generated from the **same input** (the token embeddings)
  - When the queries come from a different source than the keys and values, it is called cross-attention instead

Query, key, value formulation

- Analogy with databases where data can be stored as a (key K, value V) tuple
  - ArXiv: title + abstract (K), article (V); Youtube: title + description (K), video (V)

- In self-attention, each token embedding X_i generates a key K_i and a value V_i
- Each token embedding X_i also generates a query Q_i
  - We compute its similarity $Q_i \cdot K_j$ to all keys K_j
  - The output Y_i for the token is created by weighting all values V_j according to the similarities

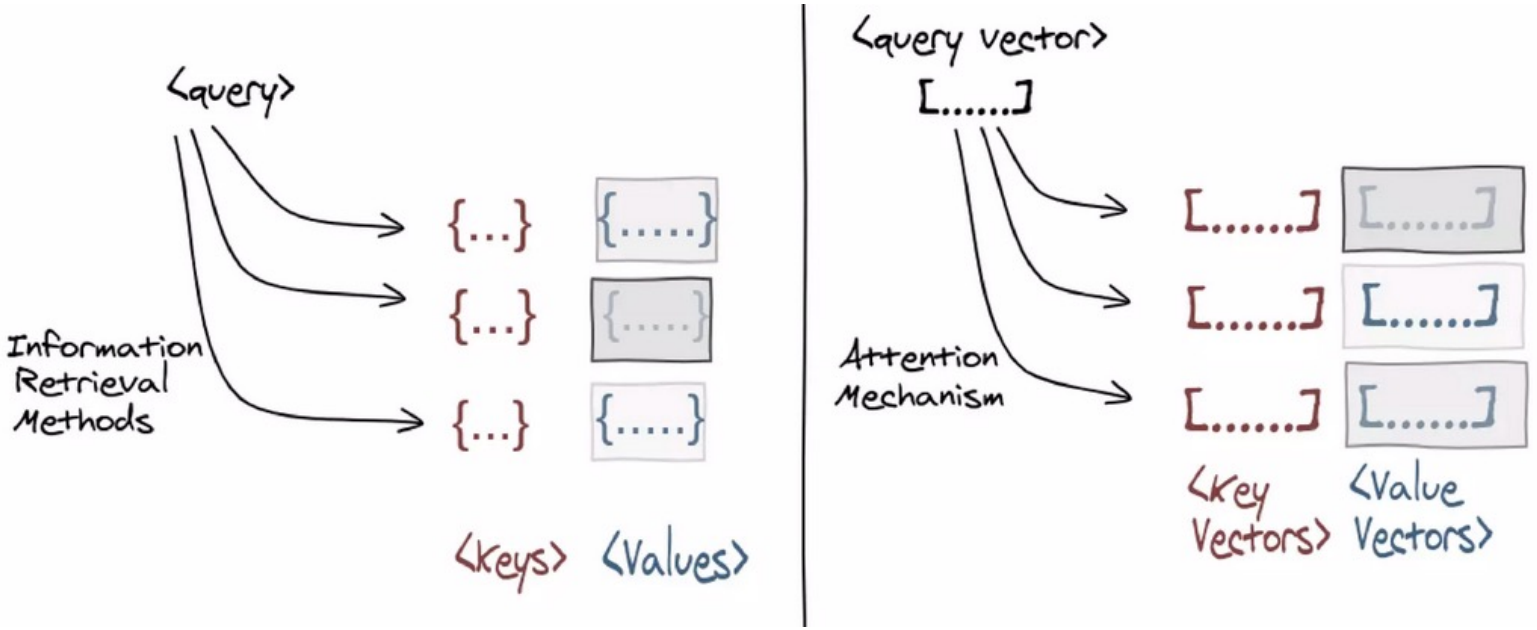


Image source: Hajji Hicham's slides "Visual Transformer Overview"

(QKV) Attention Layer

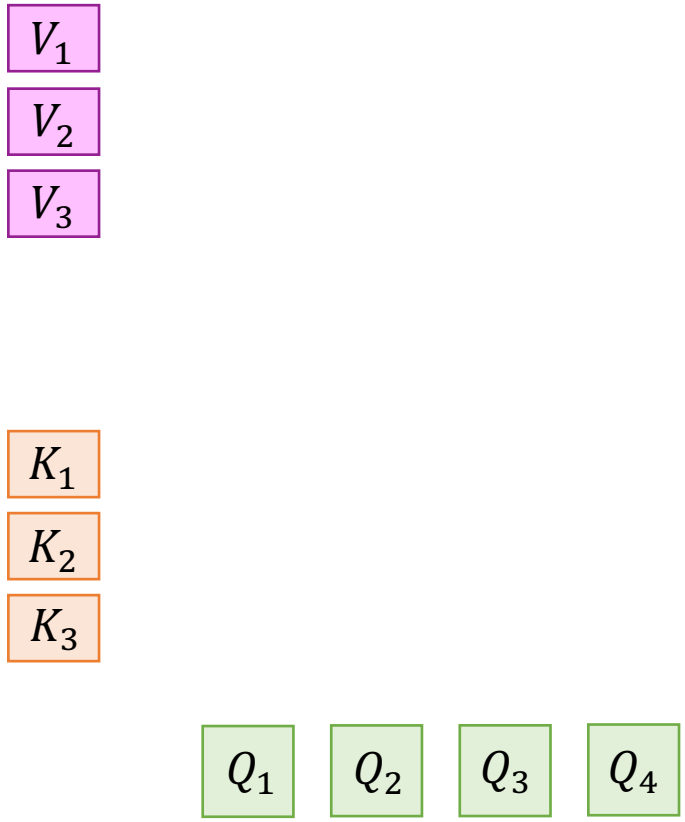
Inputs:

Key vectors: K (Shape: $N_K \times D_h$)

Value vectors: V (Shape: $N_K \times D_h$)

Query vectors: Q (Shape: $N_Q \times D_h$)

Computations:



Following slides inspired from Justin Johnson's lecture slides

(QKV) Attention Layer

Inputs:

Key vectors: K (Shape: $N_K \times D_h$)

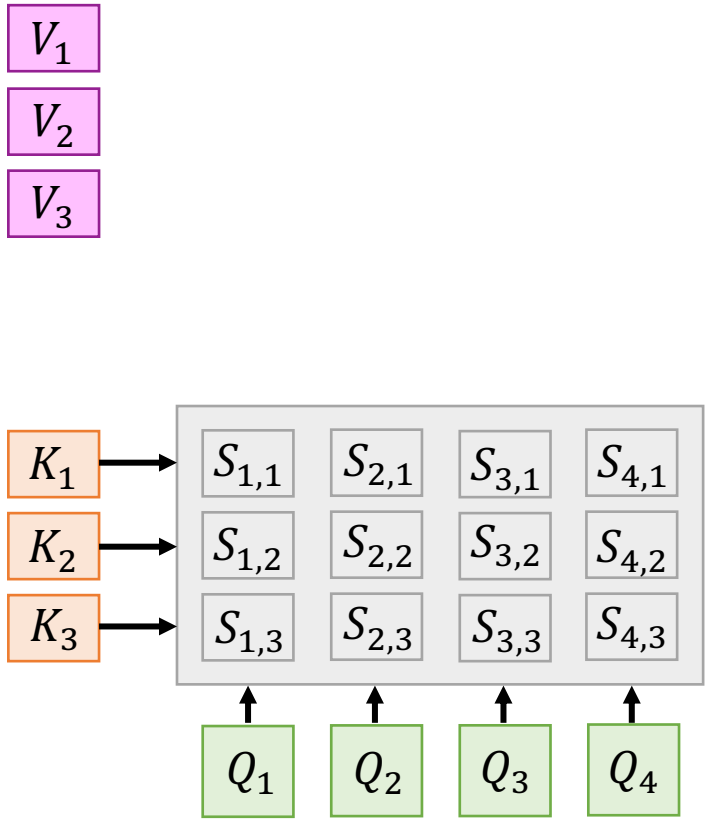
Value vectors: V (Shape: $N_K \times D_h$)

Query vectors: Q (Shape: $N_Q \times D_h$)

Computations:

Similarities: $S = \frac{QK^T}{\sqrt{D_h}}$ (Shape: $N_Q \times N_K$)

i.e., $S_{i,j} = \frac{Q_i \cdot K_j}{\sqrt{D_h}}$



(QKV) Attention Layer

Inputs:

Key vectors: K (Shape: $N_K \times D_h$)

Value vectors: V (Shape: $N_K \times D_h$)

Query vectors: Q (Shape: $N_Q \times D_h$)

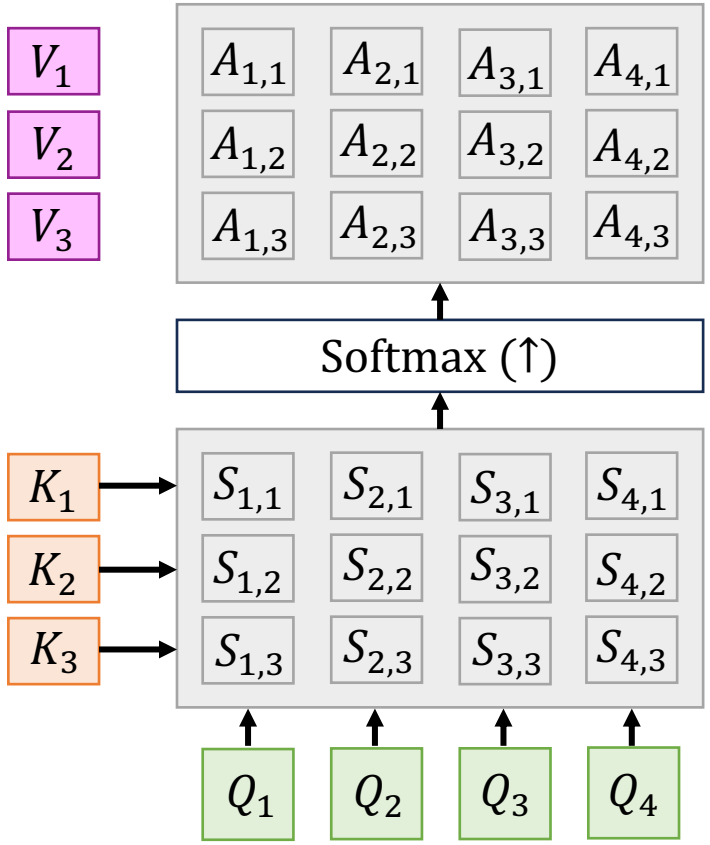
Computations:

Similarities: $S = \frac{QK^T}{\sqrt{D_h}}$ (Shape: $N_Q \times N_K$)

i.e., $S_{i,j} = \frac{Q_i \cdot K_j}{\sqrt{D_h}}$

Attention weights: $A = \mathrm{softmax}(S, \ \mathrm{dim} = 1)$ (Shape: $N_Q \times N_K$)

i.e., $A_{i,j} = \frac{\exp(S_{i,j})}{\sum_k \exp(S_{i,k})}$



(QKV) Attention Layer

Inputs:

Key vectors: K (Shape: $N_K \times D_h$)

Value vectors: V (Shape: $N_K \times D_h$)

Query vectors: Q (Shape: $N_Q \times D_h$)

Computations:

Similarities: $\mathrm{S} = \frac{\mathrm{Q}\mathrm{K}^T}{\sqrt{D_h}}$ (Shape: $N_Q \times N_K$)

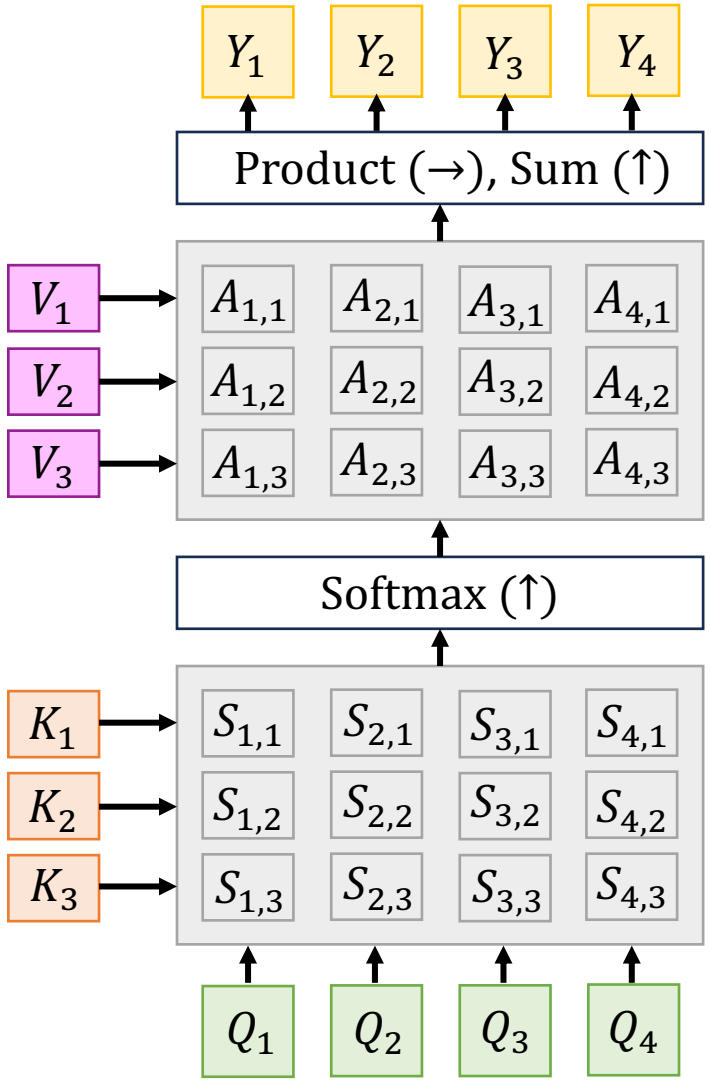
i.e., $\mathrm{S_{i,j}} = \frac{Q_i \cdot K_j}{\sqrt{D_h}}$

Attention weights: $\mathrm{A} = \mathrm{softmax}(\mathrm{S},\ \mathrm{dim} = 1)$ (Shape: $N_Q \times N_K$)

i.e., $A_{i,j} = \frac{\exp(S_{i,j})}{\sum_k \exp(S_{i,k})}$

Output vectors: $\mathrm{Y} = \mathrm{AV}$ (Shape: $N_Q \times D_h$)

i.e., $Y_i = \sum_j A_{i,j} V_j$



Cross-Attention Layer

Inputs:

1st input vectors: X (Shape: $N_X \times D$)

2nd input vectors: X' (Shape: $N_{X'} \times D$)

Computations:

X_1

X_2

X_3

X'_1 X'_2 X'_3 X'_4

Cross-Attention Layer

Inputs:

1st input vectors: X (Shape: $N_X \times D$)

2nd input vectors: X' (Shape: $N_{X'} \times D$)

Key matrix: W_K (Shape: $D \times D_h$)

Value matrix: W_V (Shape: $D \times D_h$)

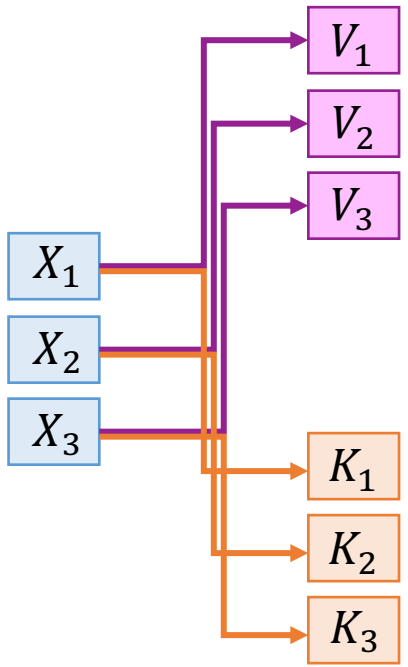
Query matrix: W_Q (Shape: $D \times D_h$)

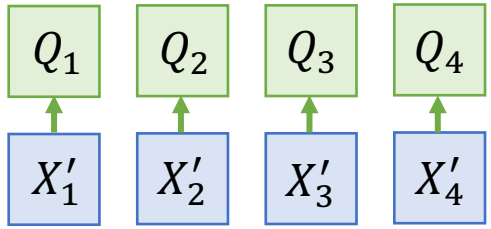
Computations:

Key vectors: $\mathrm{K} = \mathrm{X}\mathrm{W}_K$ (Shape: $N_X \times D_h$)

Value vectors: $\mathrm{V} = \mathrm{X}\mathrm{W}_V$ (Shape: $N_X \times D_h$)

Query vectors: $\mathrm{Q} = \mathrm{X}'\mathrm{W}_Q$ (Shape: $N_{X'} \times D_h$)





Cross-Attention Layer

Inputs:

1st input vectors: X (Shape: $N_X \times D$)

2nd input vectors: X' (Shape: $N_{X'} \times D$)

Key matrix: W_K (Shape: $D \times D_h$)

Value matrix: W_V (Shape: $D \times D_h$)

Query matrix: W_Q (Shape: $D \times D_h$)

Computations:

Key vectors: $\mathrm{K} = \mathrm{X}\mathrm{W}_K$ (Shape: $N_X \times D_h$)

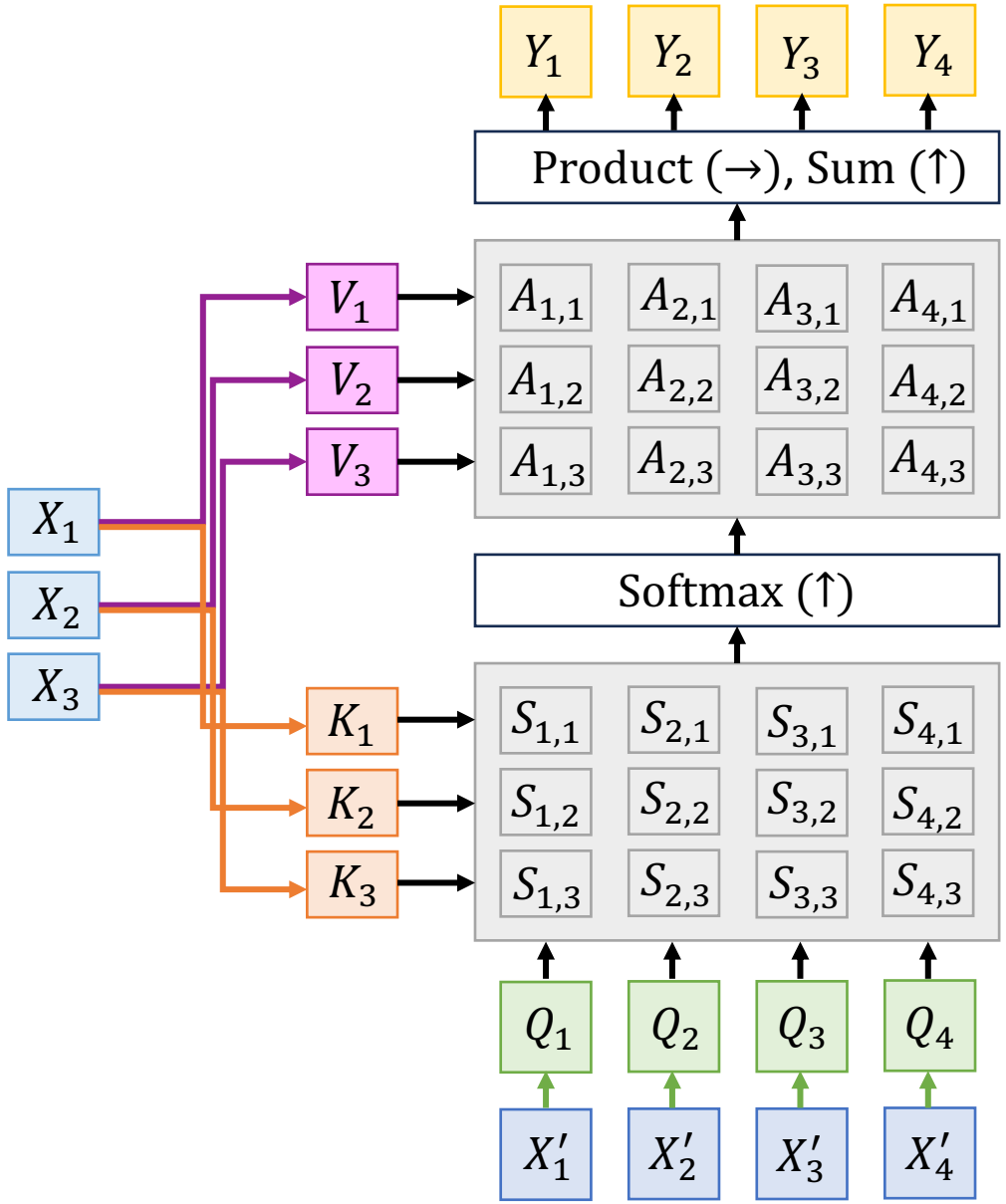
Value vectors: $\mathrm{V} = \mathrm{X}\mathrm{W}_V$ (Shape: $N_X \times D_h$)

Query vectors: $\mathrm{Q} = \mathrm{X}'\mathrm{W}_Q$ (Shape: $N_{X'} \times D_h$)

Similarities: $\mathrm{S} = \frac{\mathrm{Q}\mathrm{K}^T}{\sqrt{D_h}}$ (Shape: $N_{X'} \times N_X$)

Attention weights: $\mathrm{A} = \mathrm{softmax}(\mathrm{S},\ \mathrm{dim} = 1)$ (Shape: $N_{X'} \times N_X$)

Output vectors: $\mathrm{Y} = \mathrm{A}\mathrm{V}$ (Shape: $N_{X'} \times D_h$)



Self-Attention Layer

Inputs:

Input vectors: X (Shape: $N \times D$)

Computations:

X_1

X_2

X_3

Self-Attention Layer

Inputs:

Input vectors: X (Shape: $N \times D$)

Key matrix: W_K (Shape: $D \times D_h$)

Value matrix: W_V (Shape: $D \times D_h$)

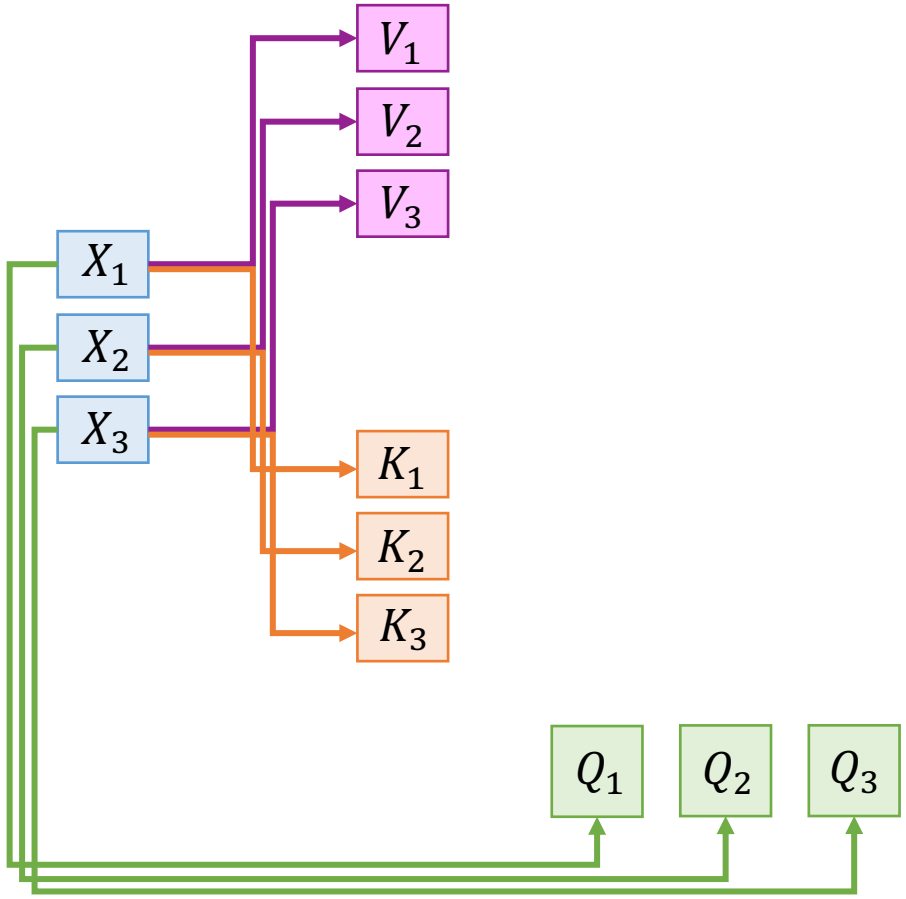
Query matrix: W_Q (Shape: $D \times D_h$)

Computations:

Key vectors: $\mathrm{K} = \mathrm{X}\mathrm{W}_K$ (Shape: $N \times D_h$)

Value vectors: $\mathrm{V} = \mathrm{X}\mathrm{W}_V$ (Shape: $N \times D_h$)

Query vectors: $\mathrm{Q} = \mathrm{X}\mathrm{W}_Q$ (Shape: $N \times D_h$)



Self-Attention Layer

Inputs:

Input vectors: X (Shape: $N \times D$)

Key matrix: W_K (Shape: $D \times D_h$)

Value matrix: W_V (Shape: $D \times D_h$)

Query matrix: W_Q (Shape: $D \times D_h$)

Computations:

Key vectors: $\mathrm{K} = \mathrm{X}\mathrm{W}_K$ (Shape: $N \times D_h$)

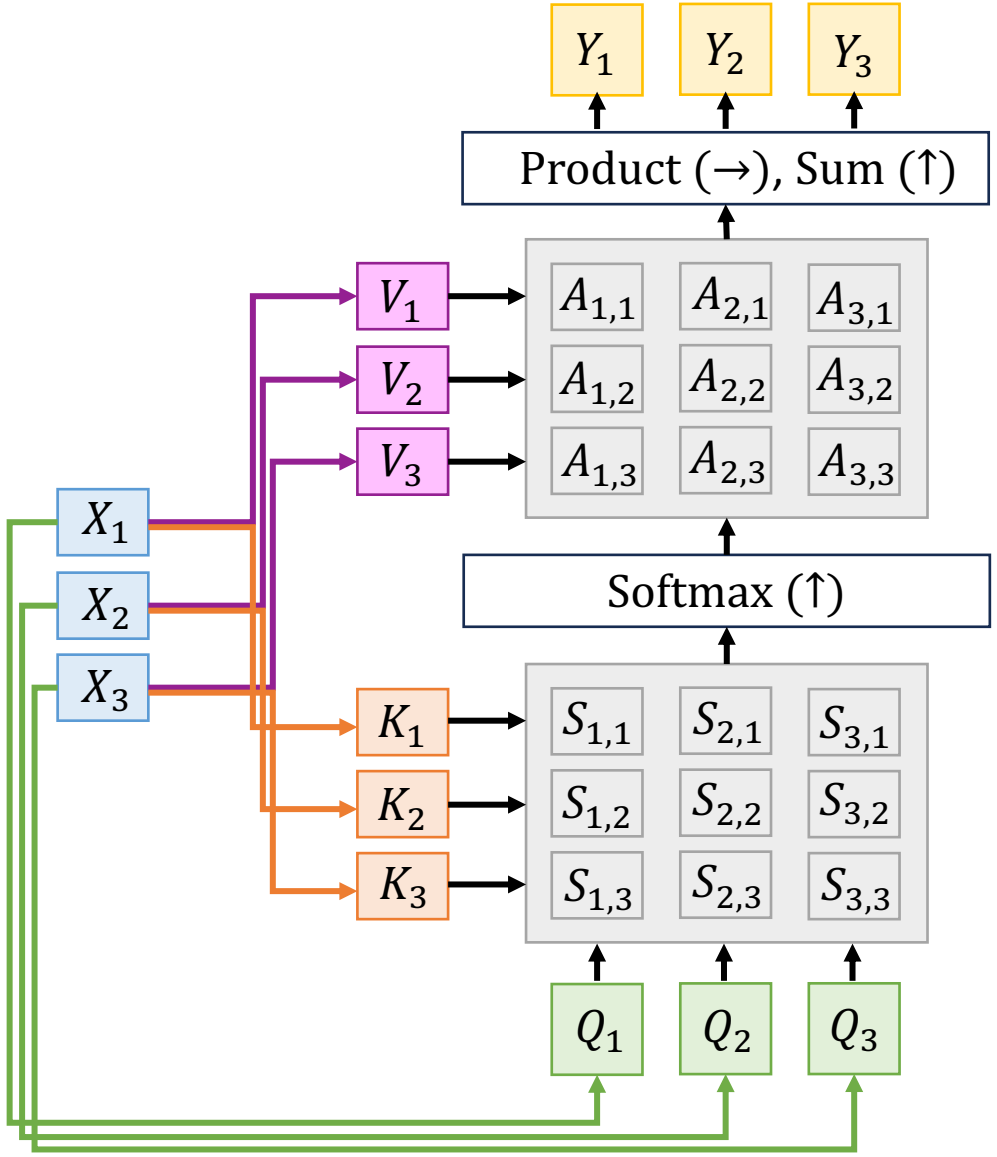
Value vectors: $\mathrm{V} = \mathrm{X}\mathrm{W}_V$ (Shape: $N \times D_h$)

Query vectors: $\mathrm{Q} = \mathrm{X}\mathrm{W}_Q$ (Shape: $N \times D_h$)

Similarities: $\mathrm{S} = \frac{\mathrm{Q}\mathrm{K}^T}{\sqrt{D_h}}$ (Shape: $N \times N$)

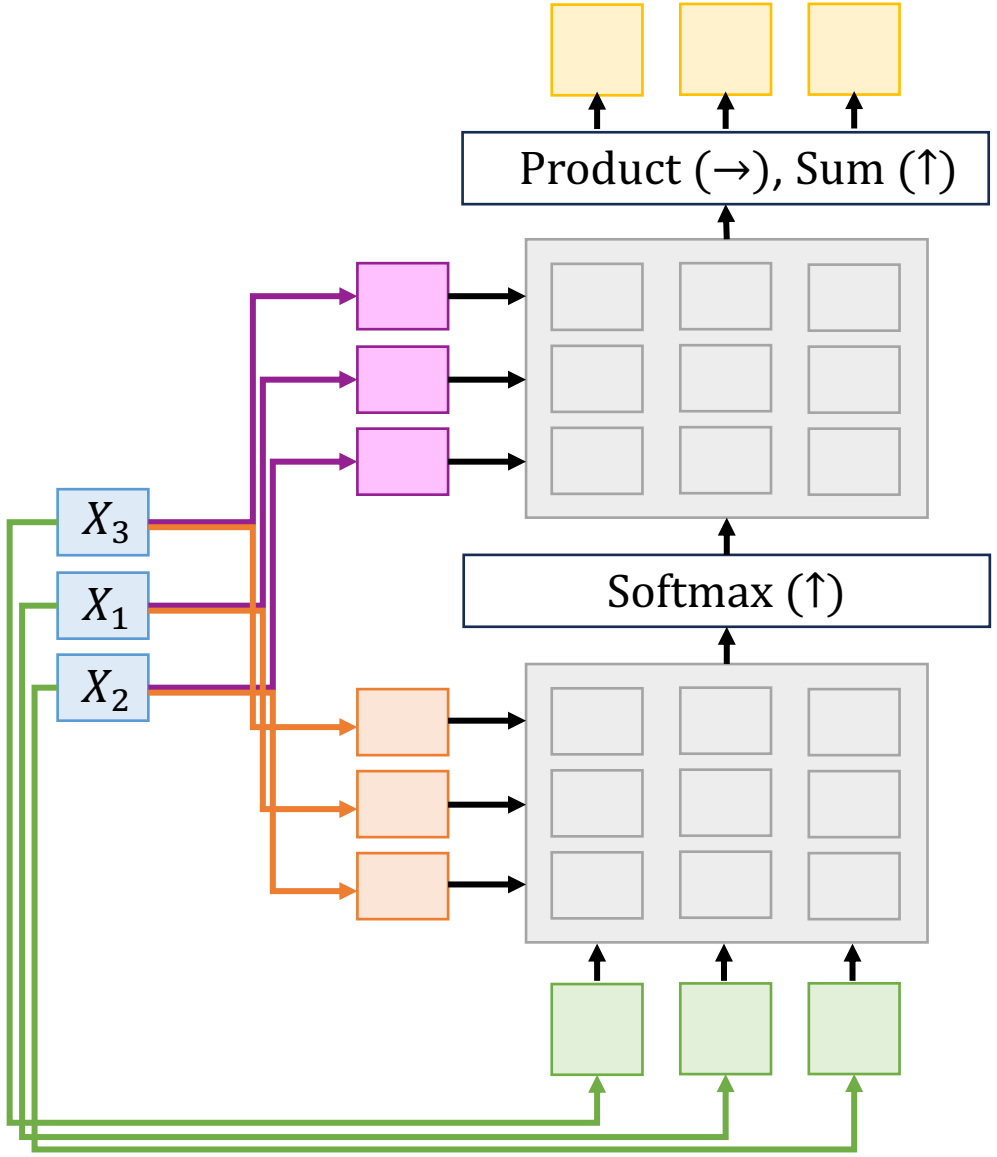
Attention weights: $\mathrm{A} = \mathrm{softmax}(\mathrm{S},\ \mathrm{dim} = 1)$ (Shape: $N \times N$)

Output vectors: $\mathrm{Y} = \mathrm{A}\mathrm{V}$ (Shape: $N \times D_h$)



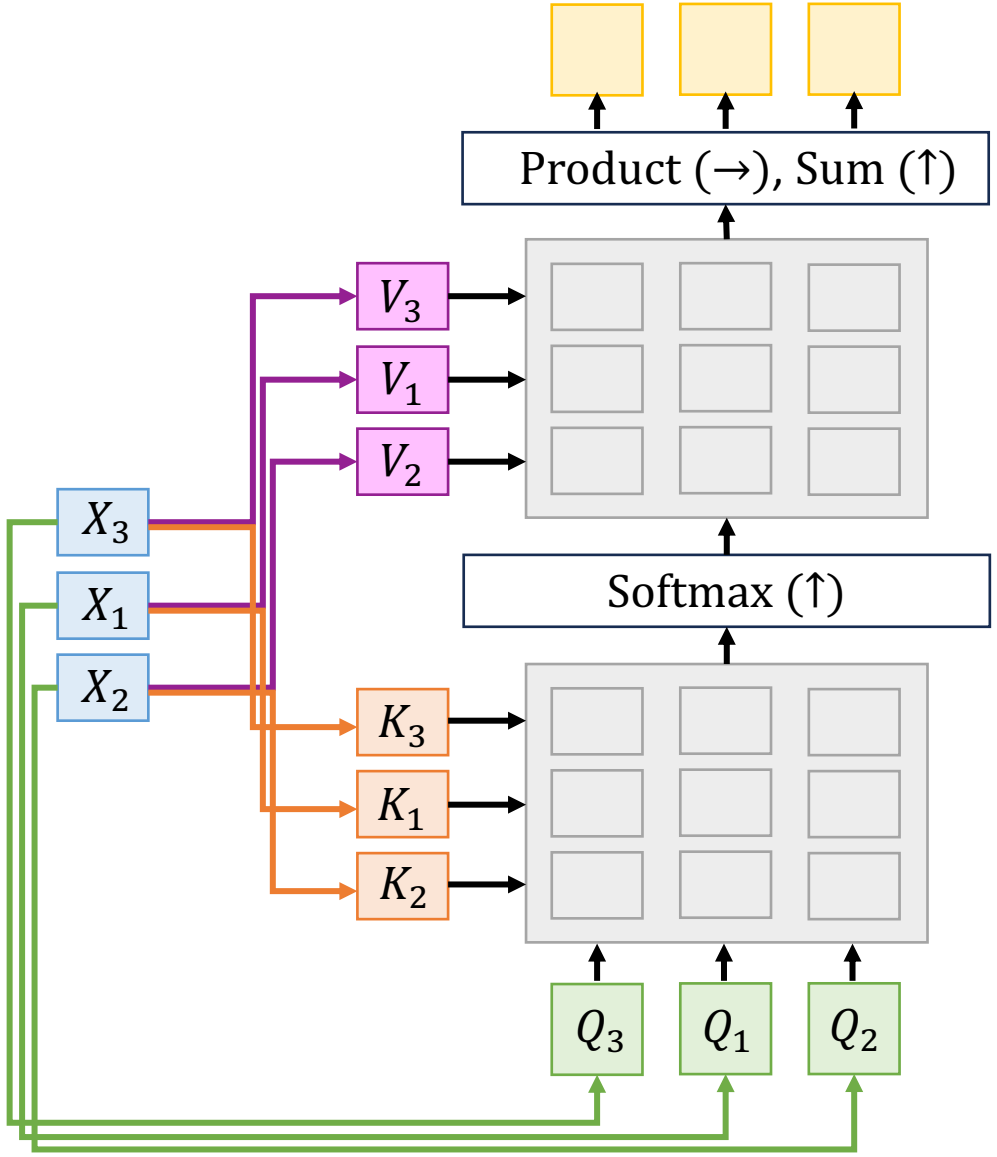
Attention layers are permutation equivariant

- What happens if we **permute** the rows in X?



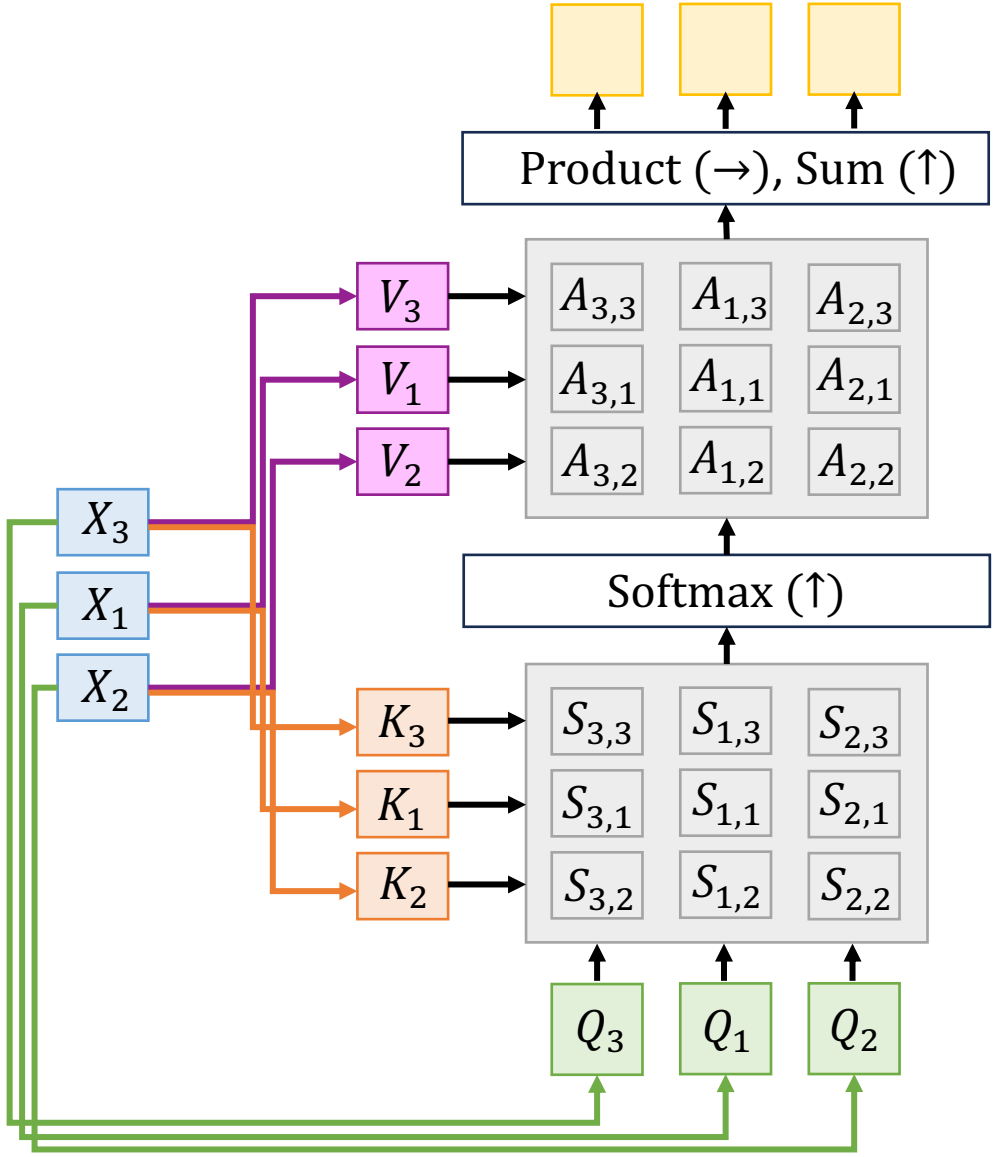
Attention layers are permutation equivariant

- What happens if we **permute** the rows in X?
- The rows of Q, K, V are permuted



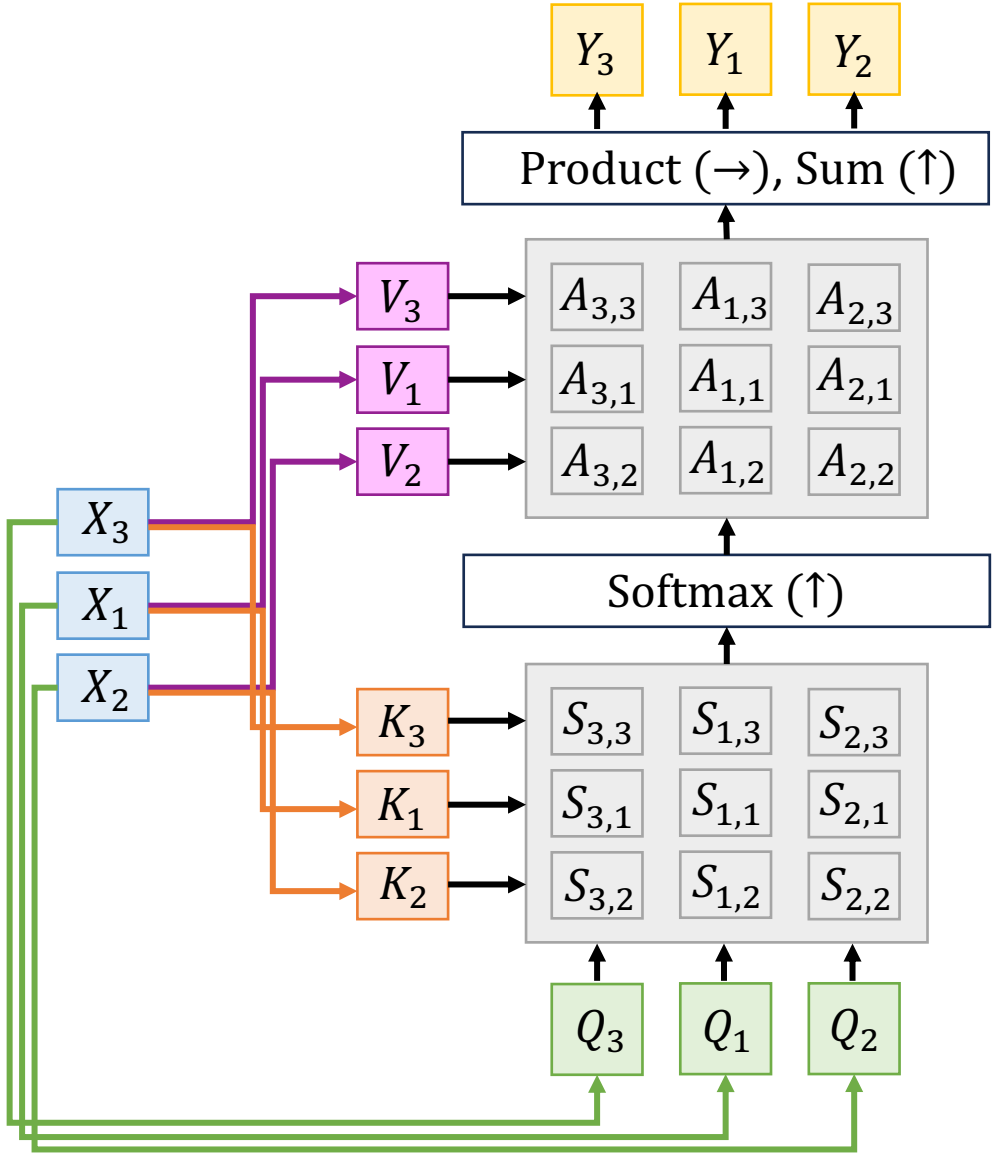
Attention layers are permutation equivariant

- What happens if we **permute** the rows in X?
- The rows of Q, K, V are permuted
- The similarity and attention coefficients are permuted in S and A



Attention layers are permutation equivariant

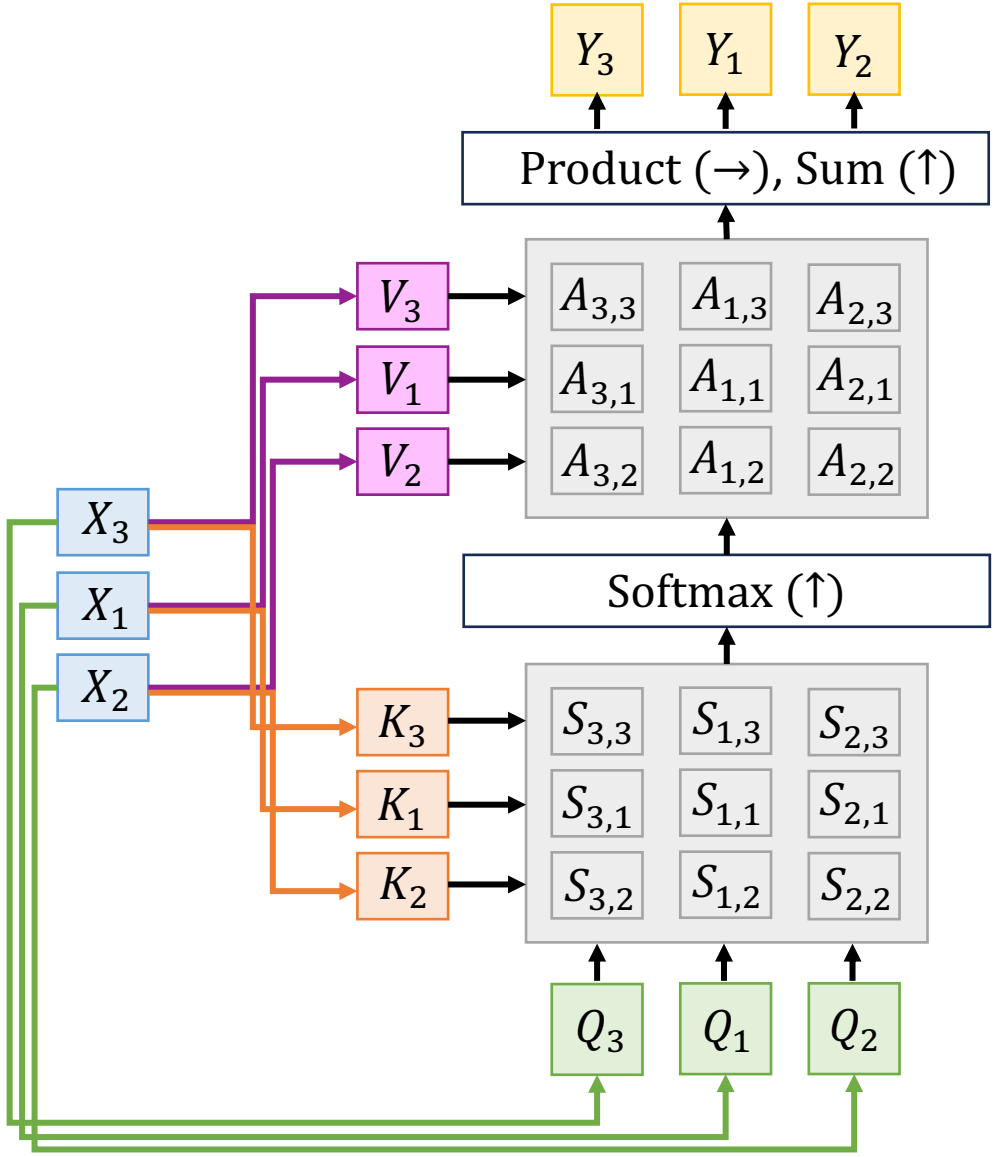
- What happens if we **permute** the rows in X?
- The rows of Q, K, V are permuted
- The similarity and attention coefficients are permuted in S and A
- The output vectors Y are permuted
  - But each token X_i has the same output Y_i as before



Attention layers are permutation equivariant

- What happens if we **permute** the rows in X?
- The rows of Q, K, V are permuted
- The similarity and attention coefficients are permuted in S and A
- The output vectors Y are permuted
  - But each token X_i has the same output Y_i as before

→ **Permutation equivariance**



Problem: SA doesn't know the order of the tokens it is processing!

Sentence 1: "The man in black caught a brown lion"

Sentence 2: "The black lion caught a man in brown"

Figure 1

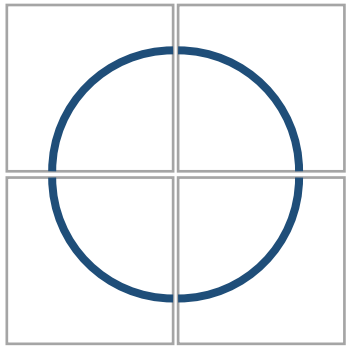
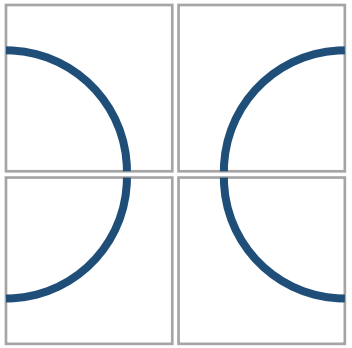


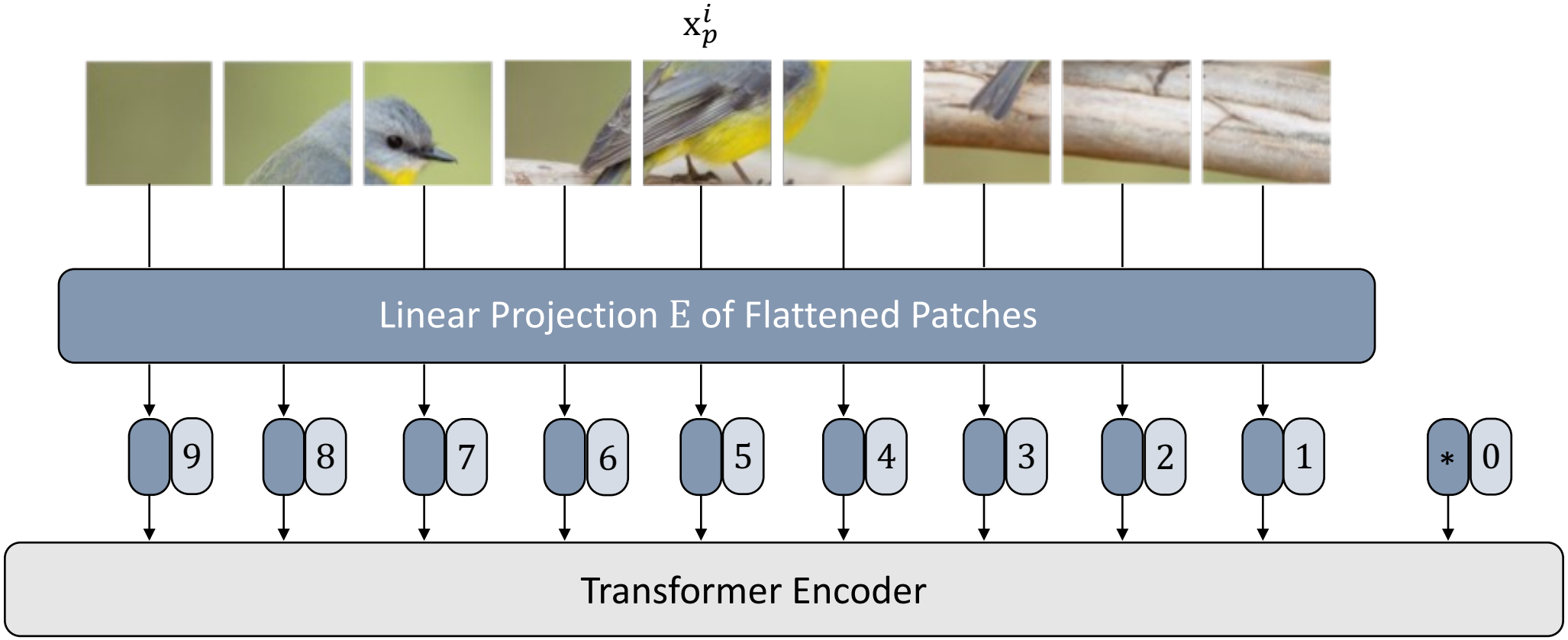
Figure 2



Same token embeddings but the **semantics are completely different**

Solution: Positional Embeddings

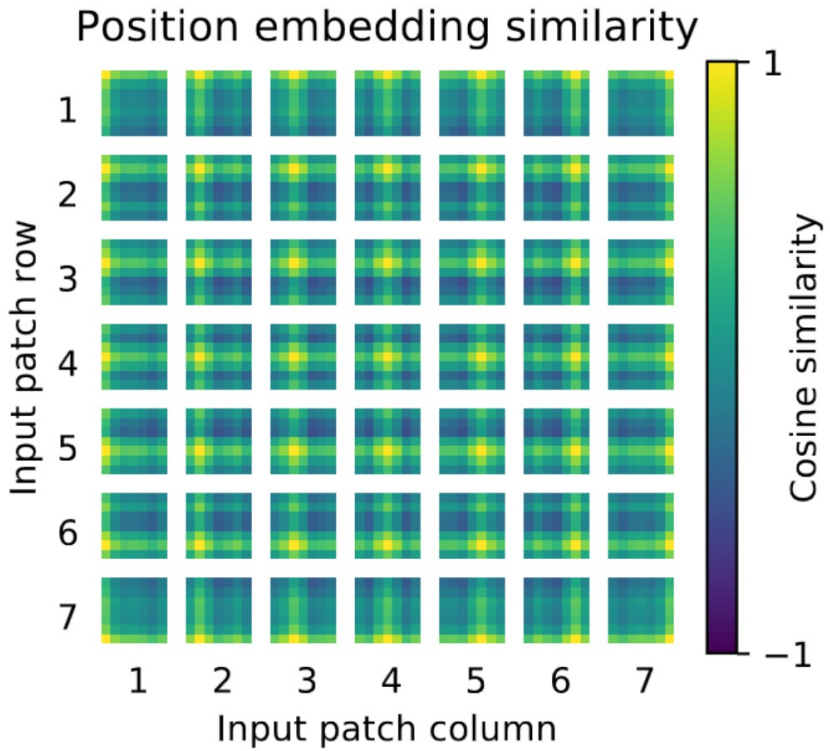
- We add learnable **positional embeddings** in $\mathbb{R}^D$ to the token embeddings: $X_i = \mathrm{E}^T \mathrm{x}_{\mathrm{p}}^i + \mathrm{E}_{\mathrm{pos}}^i$



Solution: Positional Embeddings

- The learnable positional embeddings in $\mathbb{R}^D$ are not straightforward to visualize…

- … but for any one of them, we can visualize their similarities with all others !
  - The pos. embeddings vary smoothly as a function of the patch location in the grid



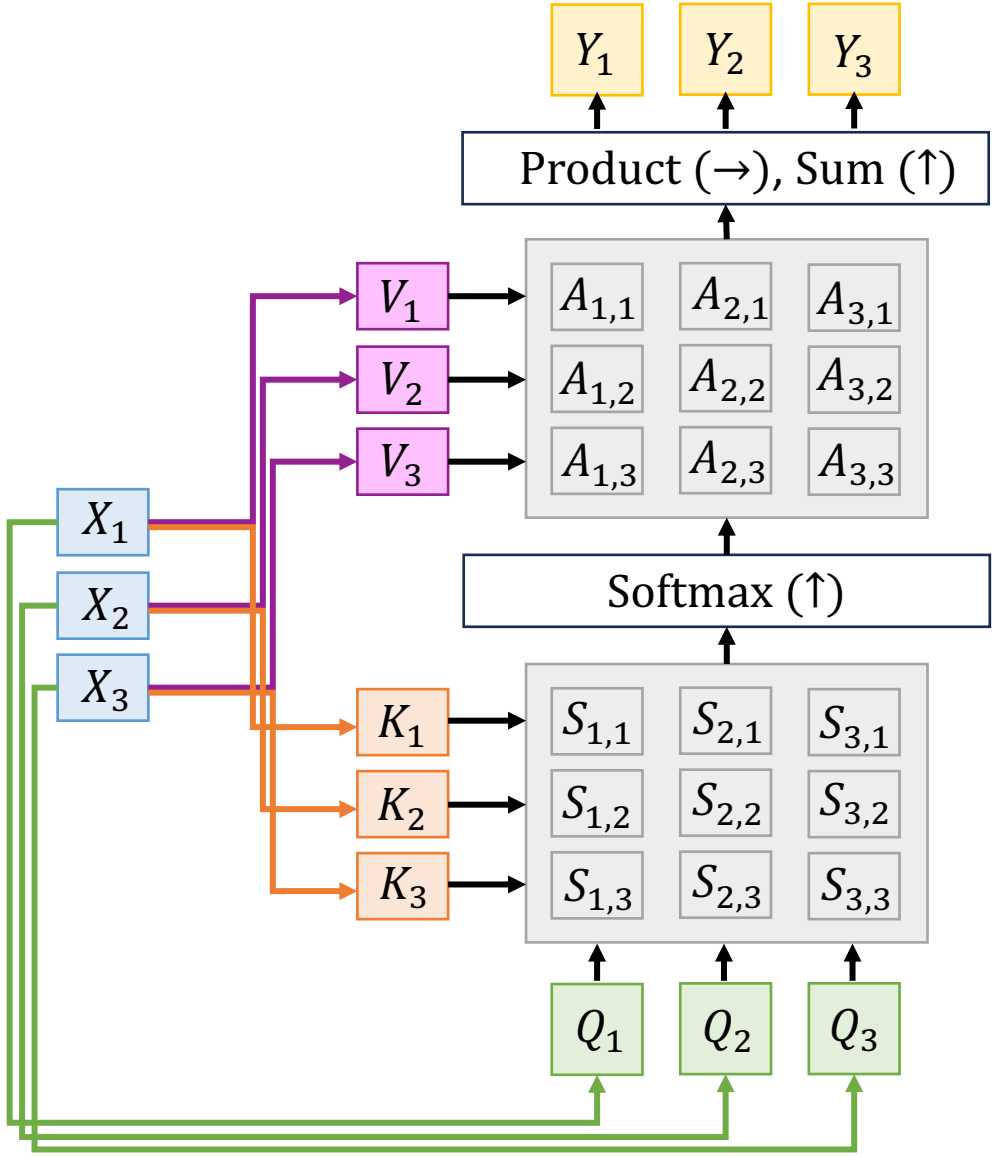
[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

Solution: Positional Embeddings

- X_i incorporates positional information
- Q_i, K_i, V_i incorporate positional information
- $S_{i,j} = Q_i \cdot K_j / \sqrt{D_h}$ and $A_{i,j}$ are modulated by the positions of i and j

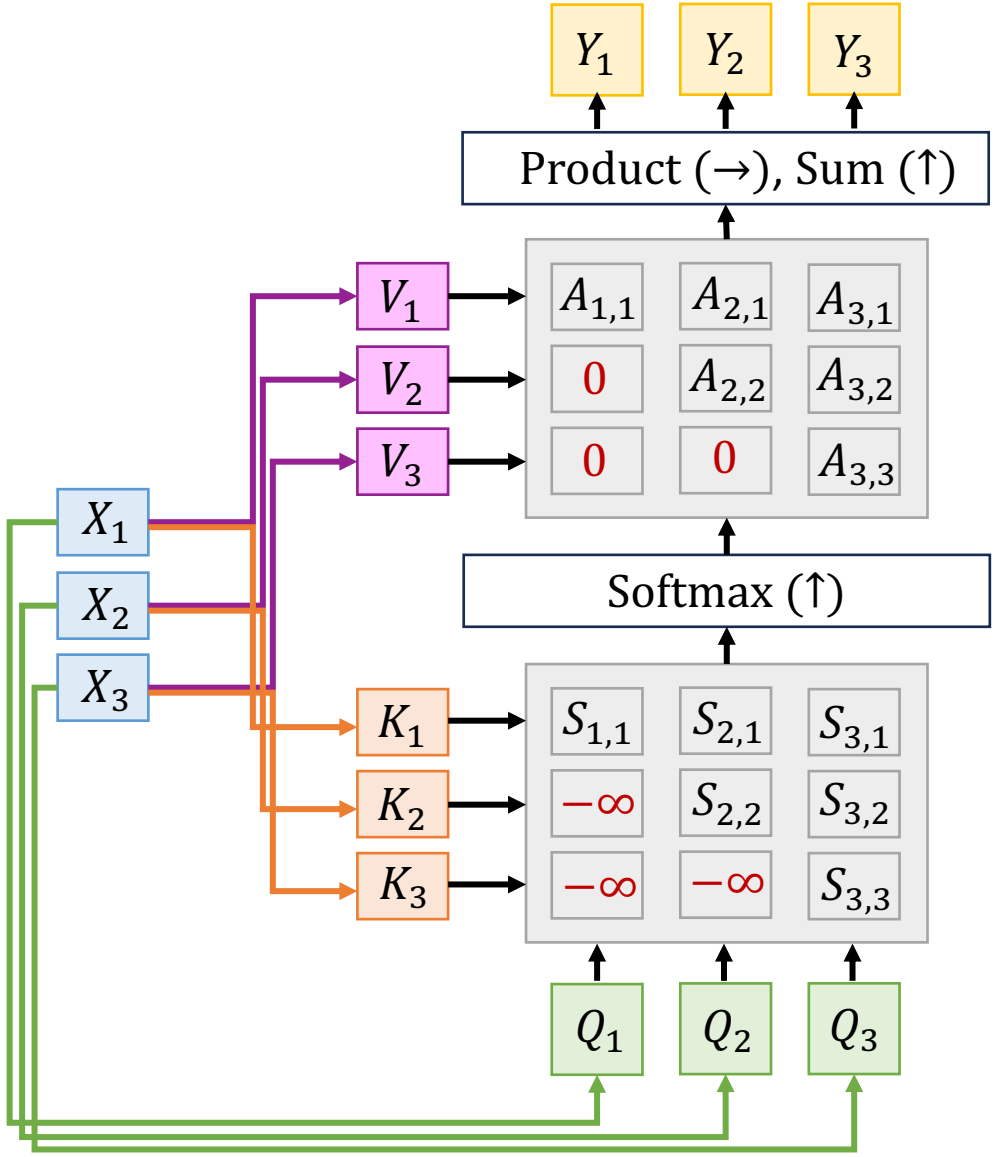
→ The **position of the tokens** in the sequence **modulates attention weights**

→ The **semantics are captured**



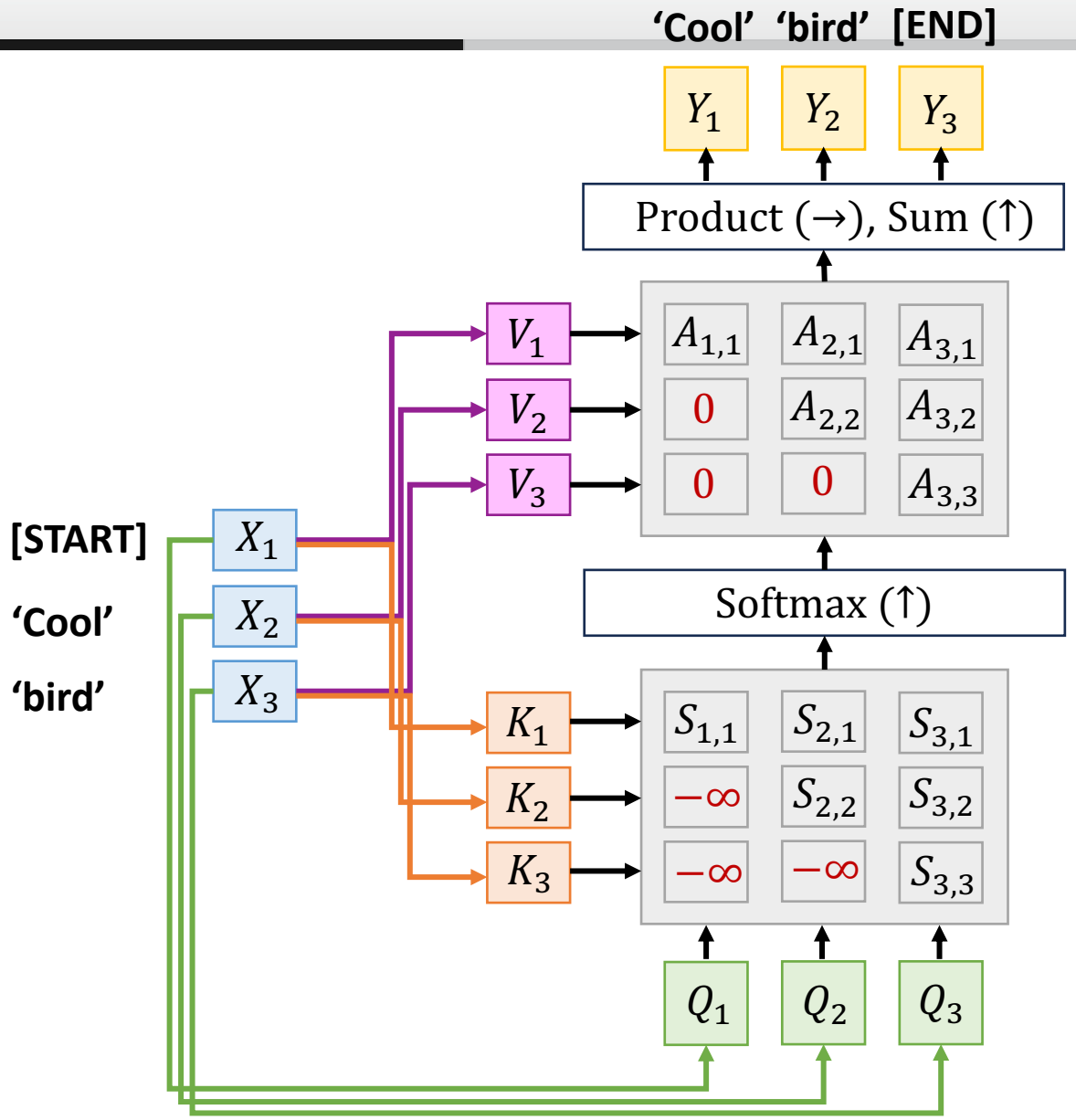
Masked Self-Attention Layer in NLP

- Subsequent tokens can only **attend to previous tokens**
  - $S_{i,j} := -\infty$ for $j > i$, hence:
  - $A_{i,j} := 0$ for $j > i$
- Don't let vectors "look ahead" in the sequence
- Useful when training a transformer to predict the next token in NLP (see later)



Masked Self-Attention Layer in NLP

- Subsequent tokens can only **attend to previous tokens**
  - $S_{i,j} := -\infty$ for $j > i$, hence:
  - $A_{i,j} := 0$ for $j > i$
- Don't let vectors "look ahead" in the sequence
- Useful when training a transformer to predict the next token in NLP (see later)

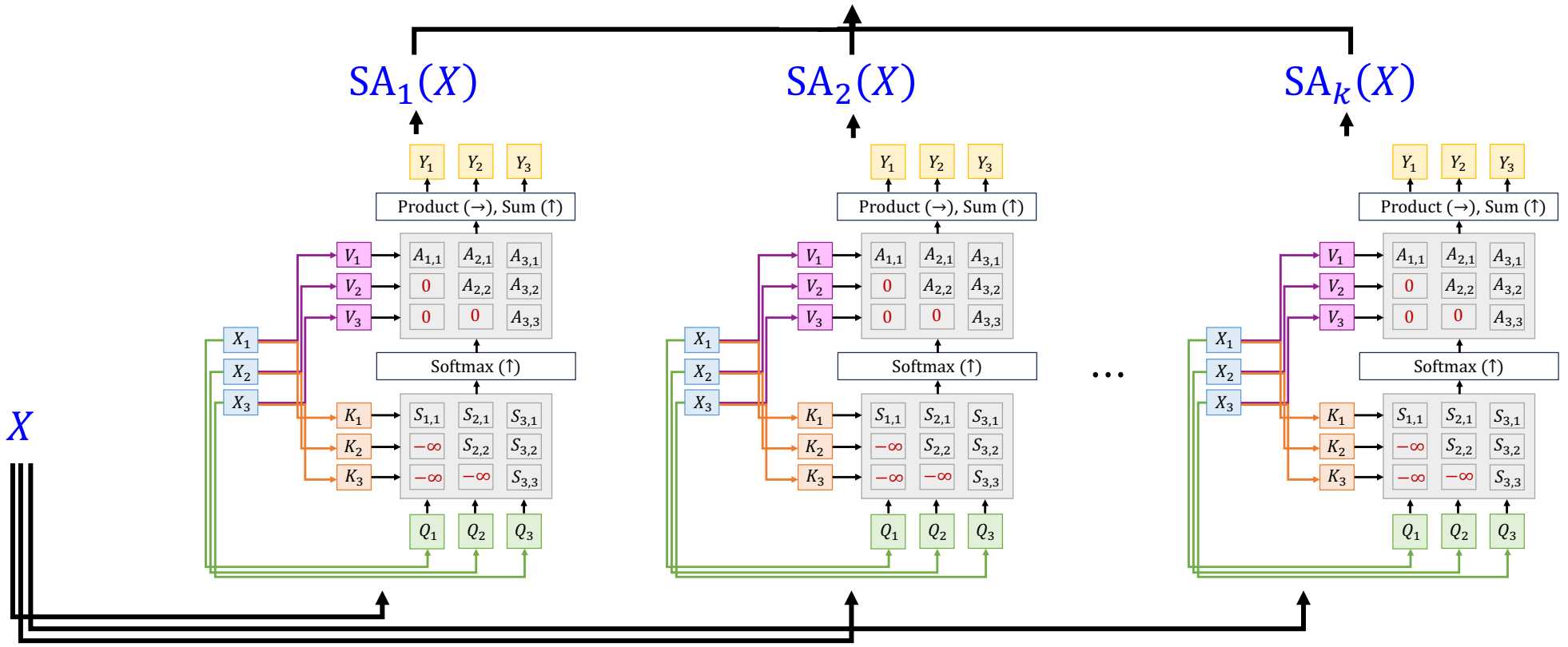


Multi-Head Self-Attention

- k **independent "attention heads"** SA_k ($\neq \mathrm{W}_K, \mathrm{W}_V, \mathrm{W}_Q$) process **in parallel** the token embeddings

- Their outputs $\mathrm{SA}_k(X)$ (each of size $N \times D_h$) are **concatenated** (size $N \times (k \cdot D_h)$) then **linearly projected** ($\mathrm{U}_{\mathrm{msa}} \in \mathbb{R}^{(k \cdot D_h) \times D}$) to the input dimension ($N \times D$); generally, $D_h := D/k$

$$\mathrm{MSA}(\mathrm{X}) = [\mathrm{SA}_1(X); \ldots; \mathrm{SA}_k(X)]\mathrm{U}_{\mathrm{msa}}$$



The Vision Transformer Encoder

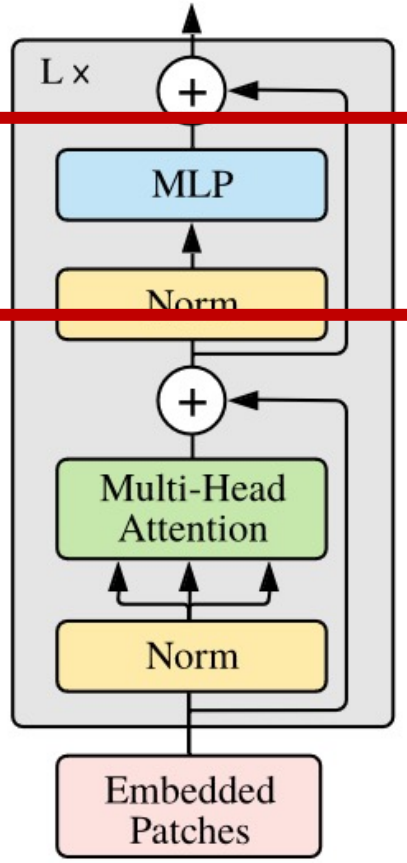
- The encoder is a succession of L identical blocks
- Each block modifies the token embeddings ($\mathbb{R}^{N\times D} \rightarrow \mathbb{R}^{N\times D}$), relying on an MSA layer and an MLP



Image adapted from: **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

The Vision Transformer Encoder

- The encoder is a succession of L identical blocks
- Each block modifies the token embeddings ($\mathbb{R}^{N\times D} \rightarrow \mathbb{R}^{N\times D}$), relying on an MSA layer and an MLP

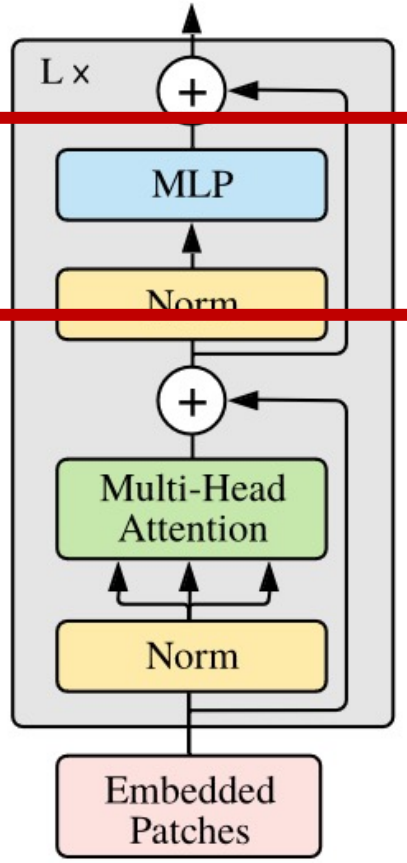


An **MLP** to **mix channel information** (i.e., from the D dimensions) within each token, without mixing information across tokens

Image adapted from: **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

The Vision Transformer Encoder

- The encoder is a succession of L identical blocks
- Each block modifies the token embeddings ($\mathbb{R}^{N\times D} \rightarrow \mathbb{R}^{N\times D}$), relying on an MSA layer and an MLP



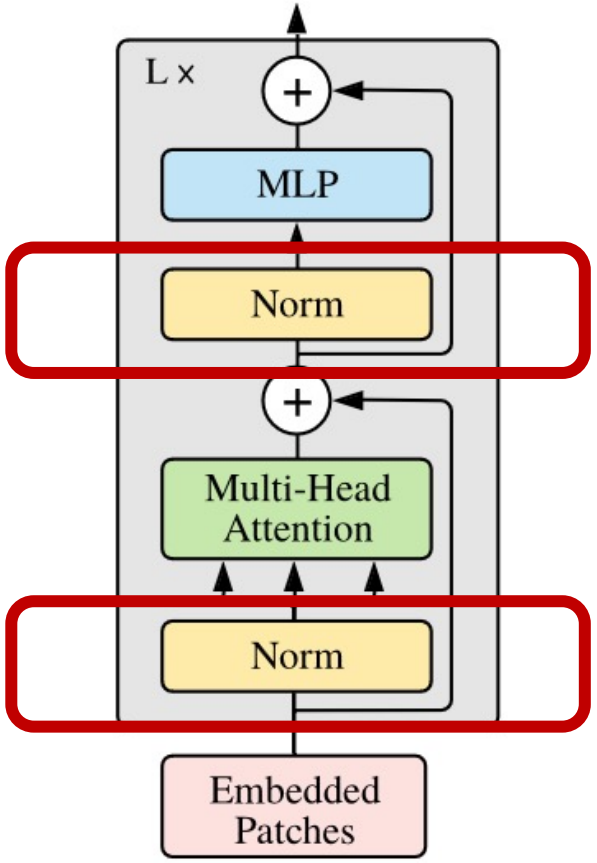
An **MLP** to **mix channel information** (i.e., from the D dimensions) within each token, without mixing information across tokens

- Unlike the MSA, the MLP modifies each token embedding without looking at other tokens
- Generally, 1 hidden layer MLP with a GeLU non-linearity, and an "inverted bottleneck" design:
  - Each token embedding in the hidden layer is projected to dimension $4\times D$

Image adapted from: **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

The Vision Transformer Encoder

- The encoder is a succession of L identical blocks
- Each block modifies the token embeddings ($\mathbb{R}^{N\times D} \rightarrow \mathbb{R}^{N\times D}$), relying on an MSA layer and an MLP

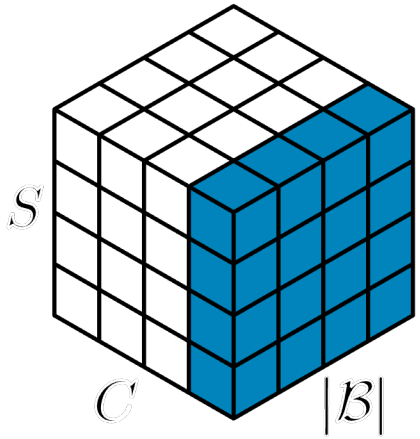


Layer Normalization

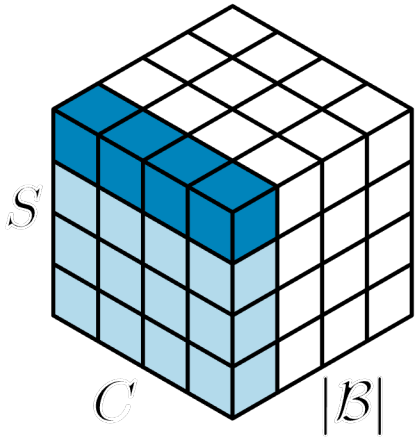
Image adapted from: **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

Recall: normalization types

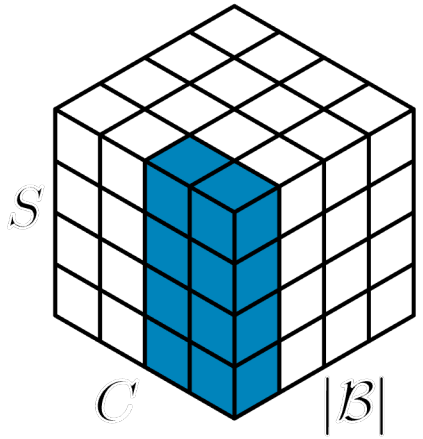
S: signal size ($H \times W$ for images, or N tokens for transformers), C: number of features/channels, $|B|$: minibatch size
Blue colors indicate the dimensions over which the averages are computed for the normalization



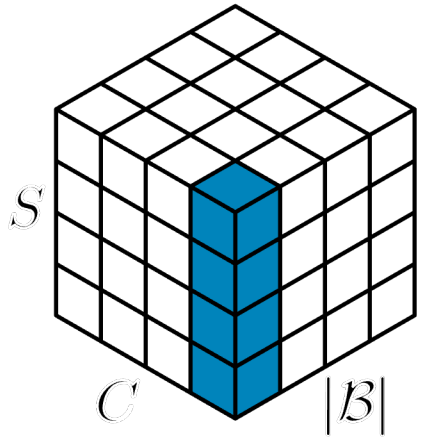
Batch Normalization



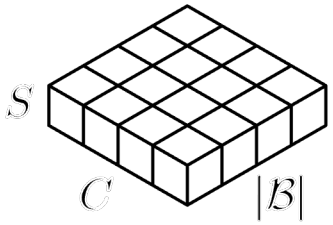
Layer Normalization



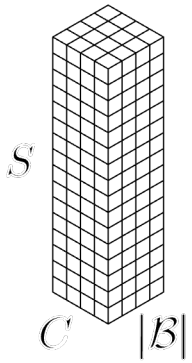
Group Normalization



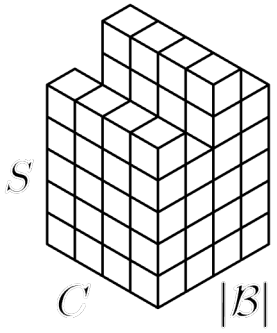
Instance Normalization



Tabular



Images



Sequential

Figures from: this source

The Vision Transformer Encoder

- The encoder is a succession of L identical blocks
- Each block modifies the token embeddings ($\mathbb{R}^{N\times D} \rightarrow \mathbb{R}^{N\times D}$), relying on an MSA layer and an MLP

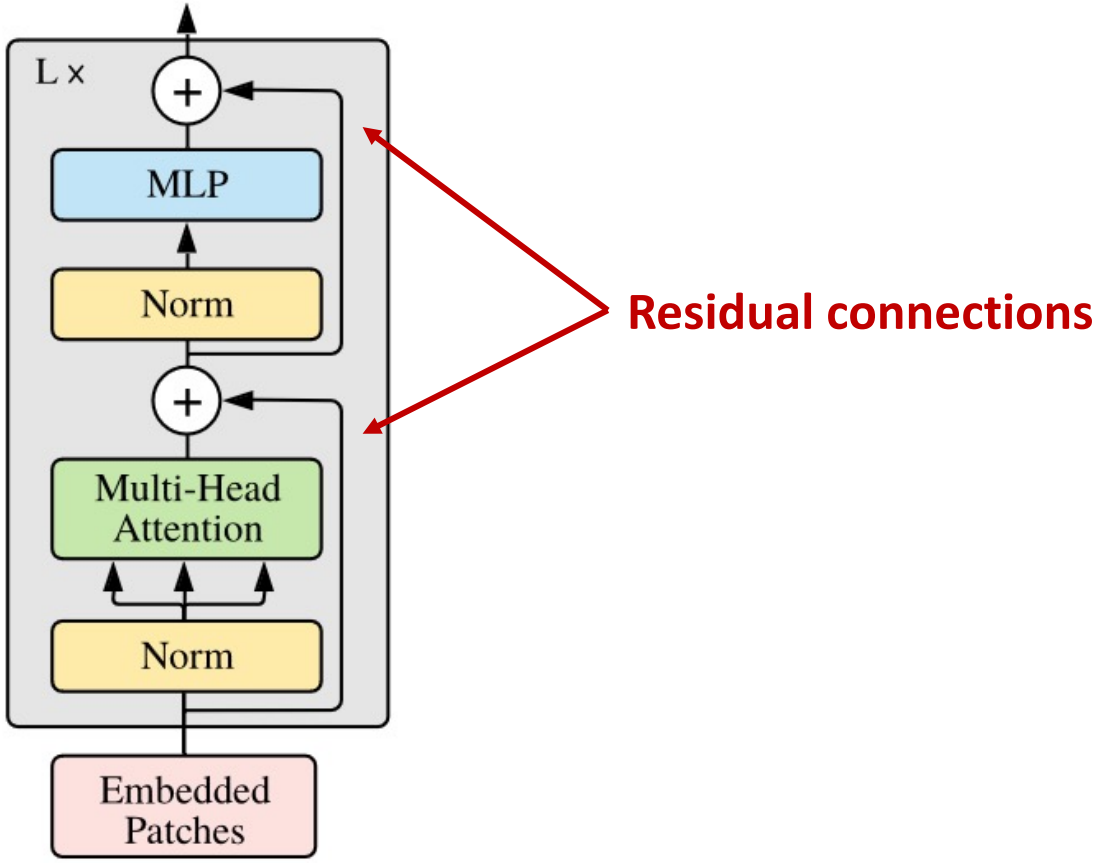
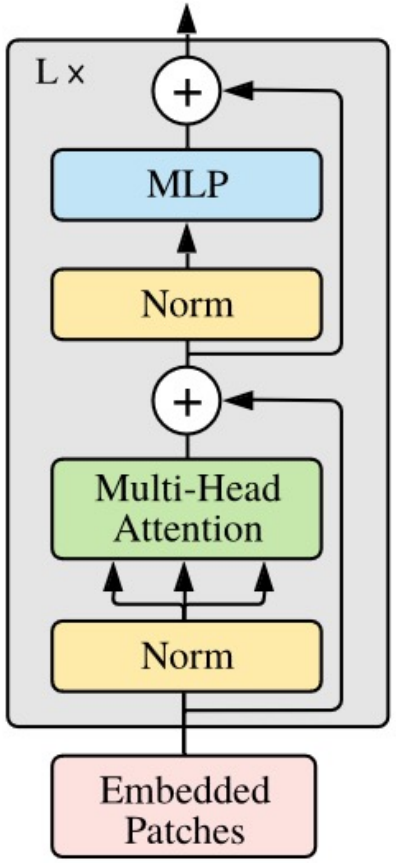


Image adapted from: **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

The Vision Transformer Encoder

- The encoder is a succession of L identical blocks
- Each block modifies the token embeddings ($\mathbb{R}^{N\times D} \rightarrow \mathbb{R}^{N\times D}$), relying on an MSA layer and an MLP



To summarize:

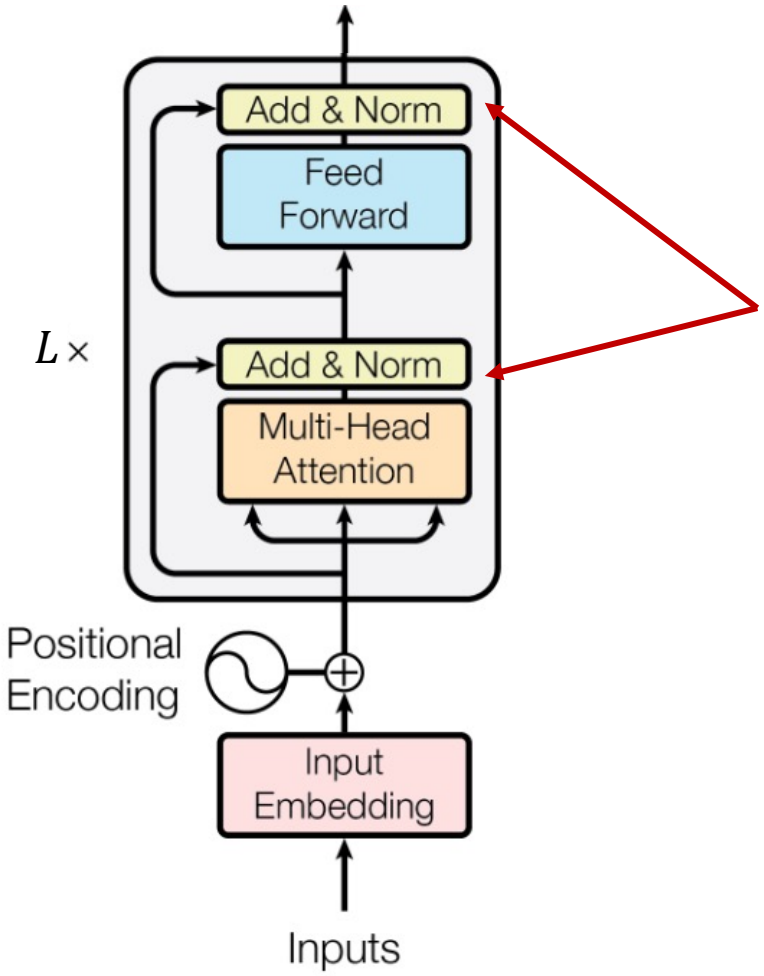
$$\text{for } l = 1 \ldots L\text{:}$$

$$X'_l = \text{MSA}\big(\text{LN}(X_{l-1})\big) + X_{l-1}$$

$$X_l = \text{MLP}\big(\text{LN}(X'_l)\big) + X'_l$$

Image adapted from: **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

A variant: the original transformer block [optional]



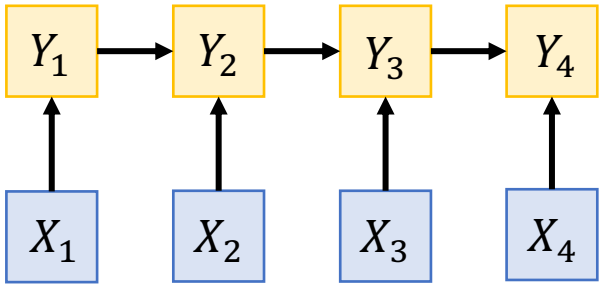
"post-normalization" outside of the residual connection

- Less common nowadays than "pre-normalization" inside the residual block

[Vaswani, 2017] **Attention is all you need**, Vaswani, A. et al., *NeuRIPS*, *2017*

Processing sequences

Recurrent Neural Network

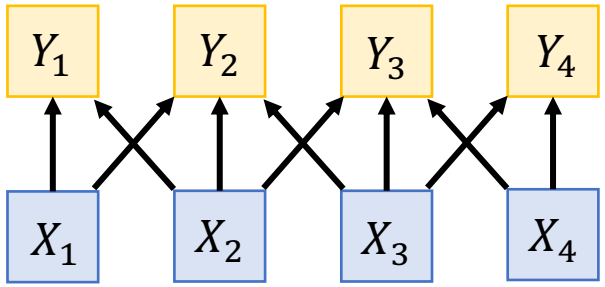


Works on **ordered sequences**

(+) **Good at long sequences**: after one RNN layer, Y_T "sees" the whole sequence

(−) **Not parallelizable**: need to compute hidden states sequentially

1D Convolution

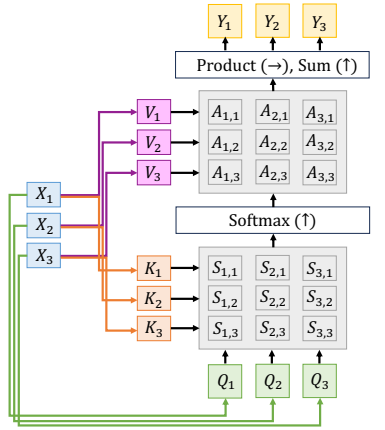


Works on **multidimensional grids**

(−) **Bad at long sequences**: need to stack many conv layers for outputs to "see" the whole sequence

(+) **Highly parallel**: each output is computed in parallel

Self-Attention



Works on **sets of vectors**

(+) **Good at long sequences**: after one self-attention layer, each output "sees" all inputs

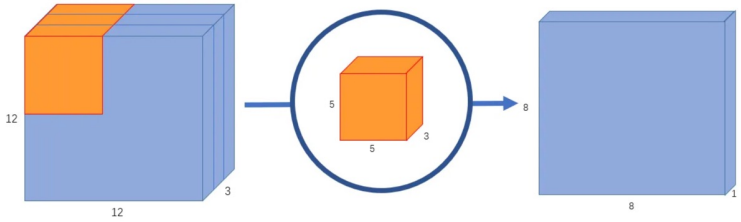
(+) **Highly parallel**: each output is computed in parallel

(−) **Very memory intensive**

Slide directly adapted from Justin Johnson's lecture

Processing images

2D Convolution

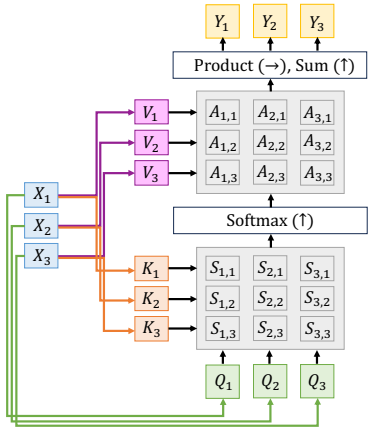


Works on **multidimensional grids**

(−) **Slow at capturing context**: need to stack many conv layers for output to "see" the whole image

(+) **High inductive bias**: CNNs don't need to learn translation equivariance from data

Self-Attention



Works on **sets of vectors**

(+) **Good at capturing context**: after one self-attention layer, each output "sees" the whole image

(+) **Highly scalable**

(−) **Little inductive bias**: needs to learn translation equivariance from data

(−) **Less memory efficient**

Parameter count: ViT-Base example

- $L \coloneqq 12$ layers, $D \coloneqq 16{\times}16{\times}3 = 768$ embedding dimension, $k \coloneqq 12$ heads, $D_{\text{mlp}} \coloneqq 768{\times}4 = 3072$ for the MLP hidden layer
  - $D_h \coloneqq D/k = 64$ dimension of each query/key/value

- How many parameters in a transformer layer?
  - MSA: $D{\times}D_h{\times}3{\times}k = 1{,}769{,}472$ for W_Q, W_K, W_V of all heads, $k{\times}D_h{\times}D = 589{,}824$ for U_{msa}
  - MLP: $\left(D{\times}D_{\text{mlp}} + D_{\text{mlp}}\right) + \left(D_{\text{mlp}}{\times}D + D\right) = 4{,}722{,}432$
  - Total: about 7.1 million parameters per block

- For 12 transformer layers, about 85 million parameters

- For the complete ViT, add the [class] and positional embeddings ($152{,}064$ params for $N \coloneqq 14{\times}14$ patches) plus the linear classification head (768000 params for ImageNet-1K)

$\rightarrow$ about 86M parameters

Scaling Vision Transformers

Model	Depth (L)	Width (D)	Heads (k)	Params ($\times 10^6$)
ViT-Ti/16	12	192	3	6
ViT-S/16	12	384	6	22
ViT-B/16	12	768	12	86
ViT-L/16	24	1024	16	307
ViT-H/14	32	1280	16	632

- As a point of comparison, ResNet-18: $11.5M$, ResNet-34: $21.8M$, ResNet-50: $25.6M$, ResNet-101: $44.5M$, ResNet-152: $60.2M$
- ViTs can be scaled to huge models with very high capacity → very data hungry, but potentially very high performance

[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*
[Touvron, 2021] **Training data-efficient image transformers & distillation through attention**, Touvron, H. et al., *ICML*, *2021*

Scaling Vision Transformers

Maintain $D_h := {}^{D}/_{k} = 64$ constant

Model	Depth (L)	Width (D)	Heads (k)	Params ($\times 10^6$)
ViT-Ti/16	12	192	3	6
ViT-S/16	12	384	6	22
ViT-B/16	12	768	12	86
ViT-L/16	24	1024	16	307
ViT-H/14	32	1280	16	632

- As a point of comparison, ResNet-18: $11.5M$, ResNet-34: $21.8M$, ResNet-50: $25.6M$, ResNet-101: $44.5M$, ResNet-152: $60.2M$
- ViTs can be scaled to huge models with very high capacity → very data hungry, but potentially very high performance

[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR, 2021*
[Touvron, 2021] **Training data-efficient image transformers & distillation through attention**, Touvron, H. et al., *ICML, 2021*

Scaling Vision Transformers

Maintain $D_h := {}^{D}/_{k} = 64$ constant

Model	Depth (L)	Width (D)	Heads (k)	Params ($\times 10^6$)
ViT-Ti/16	12	192	3	6
ViT-S/16	12	384	6	22
ViT-B/16	12	768	12	86
ViT-L/16	24	1024	16	307
ViT-H/14	32	1280	16	632

When doing so: #params $=$ $cst.$ (k), but memory still heavily depends on k through attention maps!

- As a point of comparison, ResNet-18: $11.5M$, ResNet-34: $21.8M$, ResNet-50: $25.6M$, ResNet-101: $44.5M$, ResNet-152: $60.2M$
- ViTs can be scaled to huge models with very high capacity → very data hungry, but potentially very high performance

[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR, 2021*
[Touvron, 2021] **Training data-efficient image transformers & distillation through attention**, Touvron, H. et al., *ICML, 2021*

Scaling Vision Transformers

Model	Depth (L)	Width (D)	Heads (k)	Params ($\times 10^6$)
ViT-Ti/16	12	192	3	6
ViT-S/16	12	384	6	22
ViT-B/16	12	768	12	86
ViT-L/16	24	1024	16	307
ViT-H/14	32	1280	16	632

When D doubles, params quadruple

- As a point of comparison, ResNet-18: $11.5M$, ResNet-34: $21.8M$, ResNet-50: $25.6M$, ResNet-101: $44.5M$, ResNet-152: $60.2M$
- ViTs can be scaled to huge models with very high capacity → very data hungry, but potentially very high performance

[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*
[Touvron, 2021] **Training data-efficient image transformers & distillation through attention**, Touvron, H. et al., *ICML*, *2021*

Scaling Vision Transformers

- ViTs can also be scaled by **changing patch size**: smaller patch size → more compute
  - This (almost) does not change the # of parameters but it changes the # of Floating Point Operations!

Model	Depth (L)	Width (D)	MLP (D_{mlp})	Heads (k)	Params ($\times 10^6$)	GFLOPs (224^2)
s/16	6	256	$4D$	8	5	2.2
S/32	12	384	$4D$	6	22	2.3
Ti/16	12	192	$4D$	3	6	2.5
B/32	12	768	$4D$	12	87	8.7
S/16	12	384	$4D$	6	22	9.2
B/16	12	768	$4D$	12	86	35.1
L/16	24	1024	$4D$	16	307	122.9
H/14	32	1280	$4D$	16	632	?
g/14	40	1408	6144	16	1011	533.1
G/14	48	1664	8192	16	1843	965.3

[Zhai, 2022] **Scaling vision transformers**, Zhai, X. et al., *CVPR, 2022*

Scaling Vision Transformers

- ViTs can also be scaled by **changing patch size**: smaller patch size → more compute
  - This (almost) does not change the # of parameters but it changes the # of Floating Point Operations!

Model	Depth (L)	Width (D)	MLP (D_{mlp})	Heads (k)	Params ($\times 10^6$)	GFLOPs (224^2)
s/16	6	256	$4D$	8	5	2.2
S/32	12	384	$4D$	6	22	2.3
Ti/16	12	192	$4D$	3	6	2.5
B/32	12	768	$4D$	12	87	8.7
S/16	12	384	$4D$	6	22	9.2
B/16	12	768	$4D$	12	86	35.1
L/16	24	1024	$4D$	16	307	122.9
H/14	32	1280	$4D$	16	632	?
g/14	40	1408	6144	16	1011	533.1
G/14	48	1664	8192	16	1843	965.3

Patch size has a huge effect on compute & memory, incl. $O(N^2)$ for attention maps of SA layers!

Scaling (NLP) Transformers

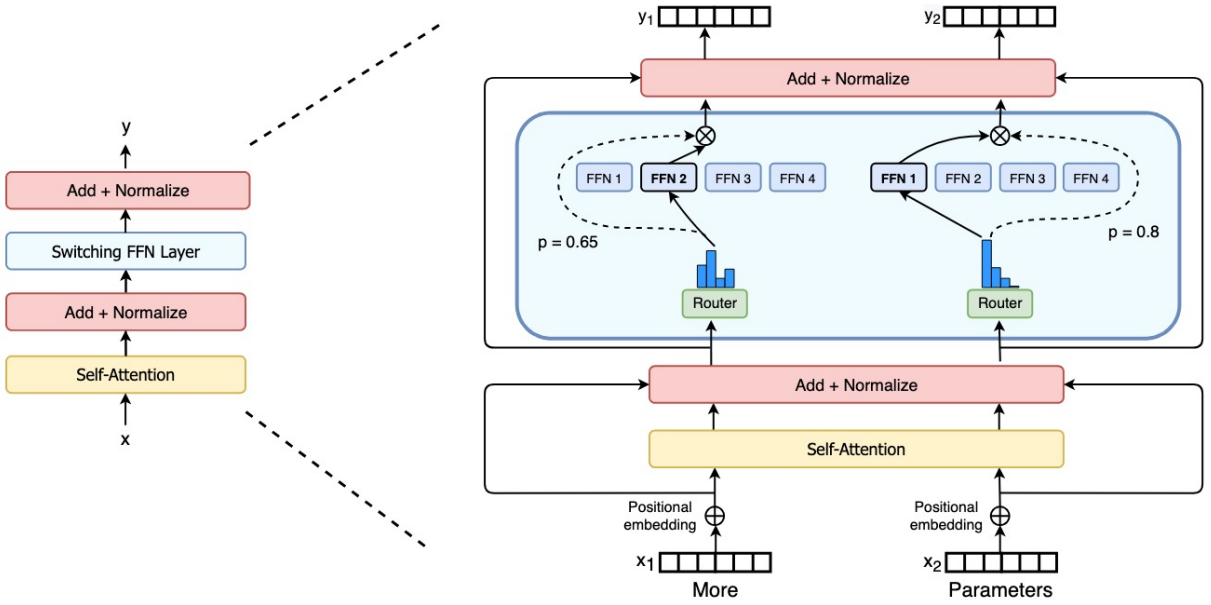
Model	Layers	Width	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	
BERT-Large	24	1024	16	340M	13 GB	
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)
Turing-NLG	78	4256	28	17B	?	256x V100 GPU
GPT-3	96	12,288	96	175B	694GB	?
Gopher	80	16,384	128	280B	10.55 TB	4096x TPUv3 (38 days)
GPT-4				**1.76T ?**		
GPT-5				**?**		

Table from Justin Johnson's lecture slides

Further scaling up

- To further scale the size of models, one can train **Mixtures Of Experts** (MoEs) which route inputs adaptively to subnetworks
  - This increases the parameter count while **keeping the FLOPs per example constant**

- **Switch Transformers** introduced Switch layers with up to 2048 experts, scaling to 1T-parameter language models (back in 2022!) that beat dense models for equivalent FLOPs



[Shazeer, 2017] **Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer**, Shazeer, N. et al., *ICLR*, *2017*
[Fedus, 2022] **Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity**, Fedus, W. et al., *JMLR*, *2022*

Space complexity (memory)

- The space complexity of multi-head self attention is usually in $\mathcal{O}(kN^2)$
  - The attention maps are the bottleneck
  - N is the number of tokens in the sequence i.e., the **context window size**

Model	Company	Parameters	Context window size
BERT-Large	Google	340M	512
Turing-NLG	Microsoft	17B	512
GPT-2	OpenAI	1.5B	1024
Llama	Meta	70B	2048
Gopher	DeepMind	280B	2048
GPT-3	OpenAI	175B	2048
Llama 2	Meta	70B	4096
GPT-3.5	OpenAI	175B	8,192

Data from Wikipedia and IBM

Space complexity (memory)

- The space complexity of multi-head self attention is usually in $\mathcal{O}(kN^2)$
  - The attention maps are the bottleneck
  - N is the number of tokens in the sequence i.e., the **context window size**

Model	Company	Parameters	Context window size
Mistral Large 2	Mistral AI	123B	128k
Llama-3.2	Meta	90B (vision) or 3B (text)	128k
GPT-4	OpenAI	1.76T ?	128k
Claude 3.5 Sonnet	Anthropic	?	200k
GPT-5	OpenAI	?	400k
Gemini 1.5 Pro	Google	120B ?	**10M**

Data from Wikipedia and IBM

- Storing attention maps in $\mathcal{O}(N^2)$ is clearly prohibitive for 128k+ context window sizes

- Various techniques are available to **reduce the space complexity** (it is often not known what techniques commercial models actually use)

Improving the space complexity of attention

- **FlashAttention** avoids the full materialization of the similarity/softmax matrix, exploiting a tiling-and-recomputation strategy, to compute exact attention with better space complexity
  - FlashAttention-2, FlashAttention-3, FlashAttention-4 bring many improvements
  - Blockwise Parallel Transformers include the feedforward layer in the blockwise computation
  - See `torch.nn.functional.scaled_dot_product_attention`

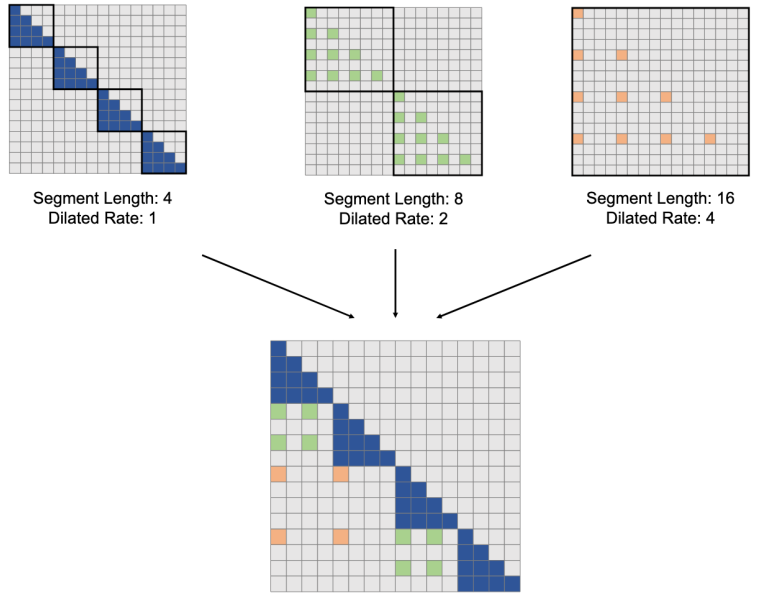
- **Ring Attention** allows the context length to scale linearly with the number of devices while maintaining performance and eliminating memory bottlenecks

[Dao, 2022] **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness**, Dao, T. et al., *NeuRIPS*, *2022*
[Liu, 2024] **RingAttention with Blockwise Transformers for Near-Infinite Context**, Liu, H. et al., *ICLR*, *2024*

Improving the space complexity of attention

- **LongNet** proposes dilated attention as a replacement of standard attention, with an attention pattern exponentially sparser as the distance in the sequence increases.

- **Infini-Attention** incorporates a compressive memory into the vanilla attention; it builds in both masked local attention and long-term linear attention mechanisms in a transformer block



dilated attention from [Ding, 2023]

[Ding, 2023] **LongNet: Scaling Transformers to 1,000,000,000 Tokens**, Ding, J. et al., *arXiv*, *2023*
[Munkhdalai, 2024] **Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention**, Munkhdalai, T. et al., *arXiv*, *2024*

Large Reasoning Models

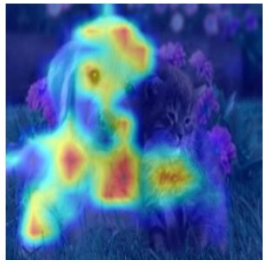
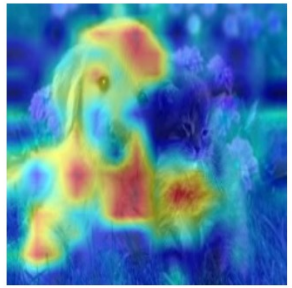
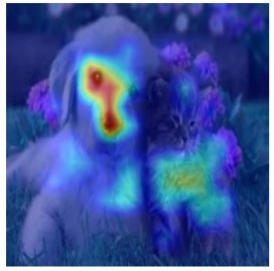
- Recently, the focus has shifted to endowing LLMs with **better reasoning capabilities**, including "chain-of-thought" answers, including via RLHF/RLVR

Model	Company	Parameters	Context window size
Mistral Large 2	Mistral AI	123B	128k
Llama-3.2	Meta	90B (vision) or 3B (text)	128k
GPT-4	OpenAI	1.76T ?	128k
Claude 3.5 Sonnet	Anthropic	?	200k
GPT-5	OpenAI	?	**400k**
Gemini 1.5 Pro	Google	120B ?	**10M**

Data from Wikipedia and IBM

Visualizing ViT's focus

- How to visualize the regions on which ViTs focus when classifying an image?
- Some techniques developed for CNNs like Saliency Maps [Simonyan, 2013] or GradCAM [Selvaraju, 2017] still apply

Category	Image	GradCAM	AblationCAM	ScoreCAM
Dog				
Cat				

Source: github

[Simonyan, 2013] **Deep inside convolutional networks: Visualising image classification models and saliency maps**, Simonyan, K. et al., *arXiv 2013*
[Selvaraju, 2017] **Grad-cam: Visual explanations from deep networks via gradient-based localization**, Selvaraju, R. R. et al., *ICCV 2017*

Visualizing ViT's focus

- How to visualize the regions on which ViTs focus when classifying an image?
- Some techniques developed for CNNs like Saliency Maps [Simonyan, 2013] or GradCAM [Selvaraju, 2017] still apply



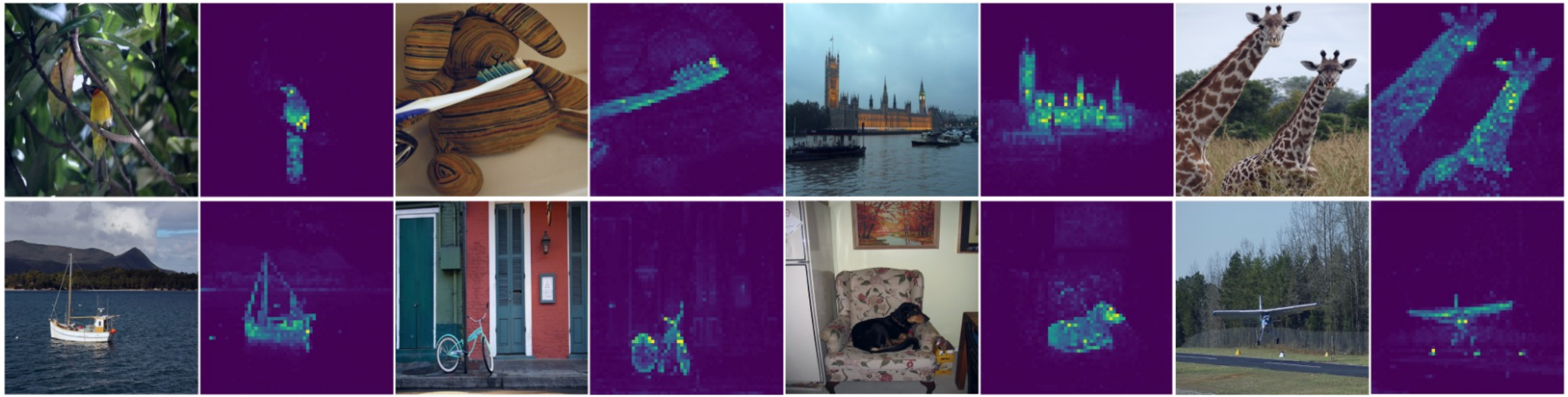
But we can do much simpler!

[Simonyan, 2013] **Deep inside convolutional networks: Visualising image classification models and saliency maps**, Simonyan, K. et al., *arXiv 2013*
[Selvaraju, 2017] **Grad-cam: Visual explanations from deep networks via gradient-based localization**, Selvaraju, R. R. et al., *ICCV 2017*

Attention maps: what does the [cls] token attend to?

- In transformers, only self-attention layers mix information between different tokens
  - And in particular between patch tokens and the [cls] token

- Attention map = the **row of the [cls] token** in the **last layer's attention matrix, rearranged spatially**
  - For MHSA, sum attention matrices from all heads first



Source: **Emerging properties in self-supervised vision transformers**, Caron, M. et al., *ICCV 2021*

Attention maps

- Attention map = the **row of the [cls] token** in the **last layer's attention matrix, rearranged spatially**
  - For MHSA, sum attention matrices from all heads first

- Thresholding attention maps is a simple approach to weakly-supervised/unsupervised segmentation

Thresholded attention maps for ViT trained on ImageNet:



Improved attention maps when ViT trained in self-supervised fashion:



Figure adapted from: **Emerging properties in self-supervised vision transformers**, Caron, M. et al., *ICCV 2021*

Attention maps

- Attention map = the **row of the [cls] token** in the **last layer's attention matrix, rearranged spatially**
- We can also display each head in a different color



Figure adapted from: **Emerging properties in self-supervised vision transformers**, Caron, M. et al., *ICCV 2021*

Attention Rollout

- The previous approach **ignores** that the [cls] token (or other tokens) can attend to other tokens in **previous layers**, rather than in the last layer

- Solution [Abnar, 2020]: **Multiply all layers' attention matrices** to account for the flow of information, then retrieve the [cls] row of the resulting matrix

recursively: $\tilde{A}(l) = A(l)\tilde{A}(l-1)$, then retrieve $\tilde{A}_{[\text{cls}],.}(L)$



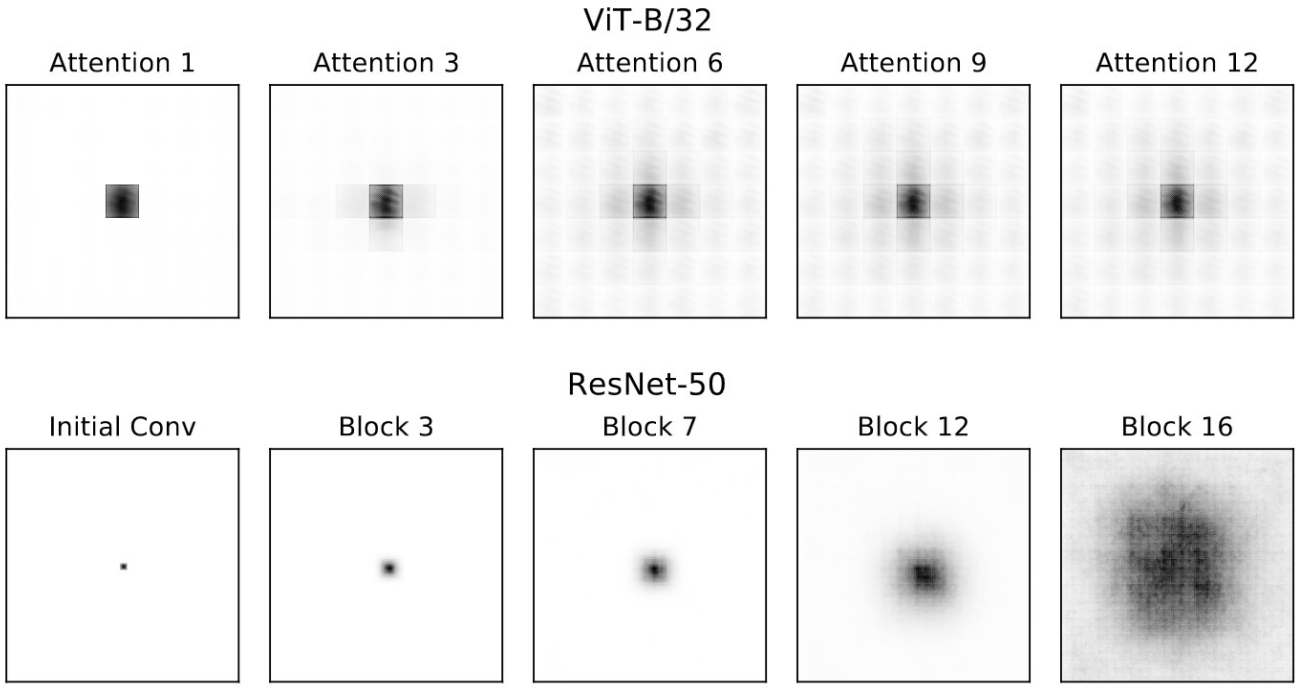
Source: [Dosovitskiy, 2021]

[Abnar, 2020] **Quantifying attention flow in transformers**, Abnar, S. et al., *arXiv 2020*
[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR, 2021*

ViT vs. ResNet: Effective Receptive Field (ERF)

- Quantify the dependency of a given location in a given layer w.r.t. all locations in the input layer, by computing partial derivatives averaged over many input images

- ResNet's ERF grows gradually – ViT's ERF shifts midway through from local to global
  - ViT's ERF retains strong dependence on the center patch due to their strong residual connections

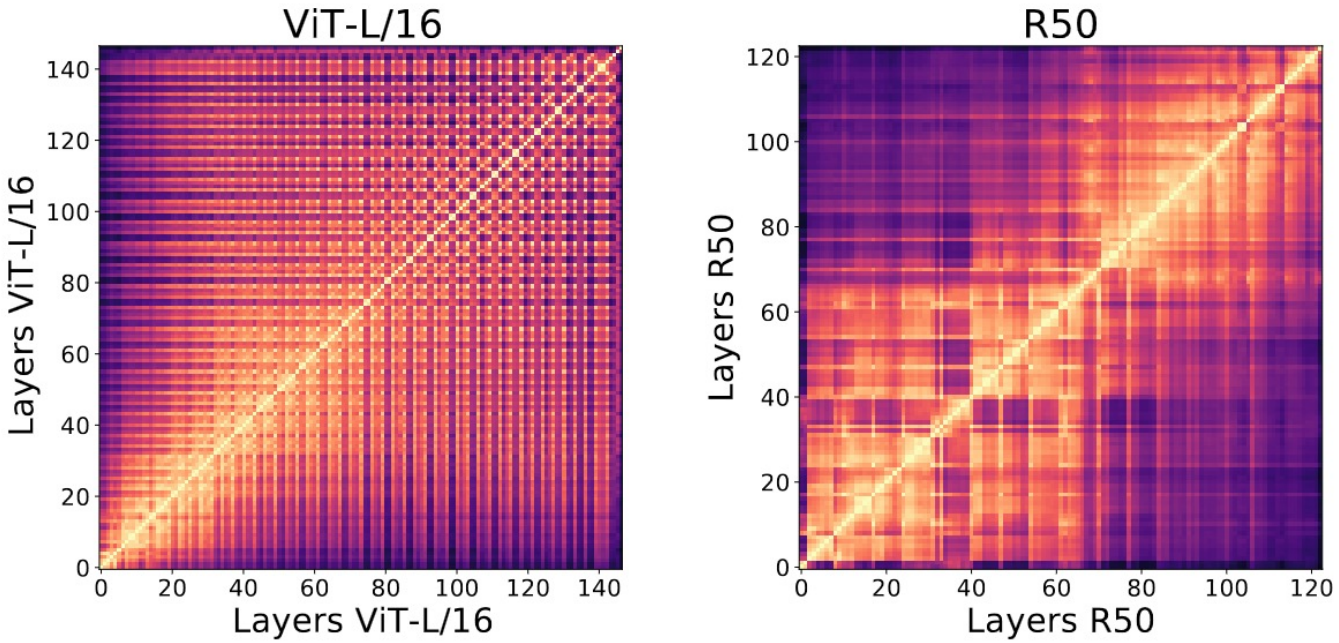


[Raghu, 2021] **Do vision transformers see like convolutional neural networks?**, Raghu, M. et al., *NeuRIPS*, *2021*

ViT vs. ResNet: representation similarity across layers

- [Raghu, 2021] proposes to quantify the **representation similarity of various layers**, using Centered Kernel Alignment (CKA)
  - Quantifies the similarity of every pair of layers within ViT or ResNet

- ViT's representation remains similar throughout the model / is quite uniform…
- … whereas ResNet representation changes abruptly across several stages / is more hierarchical

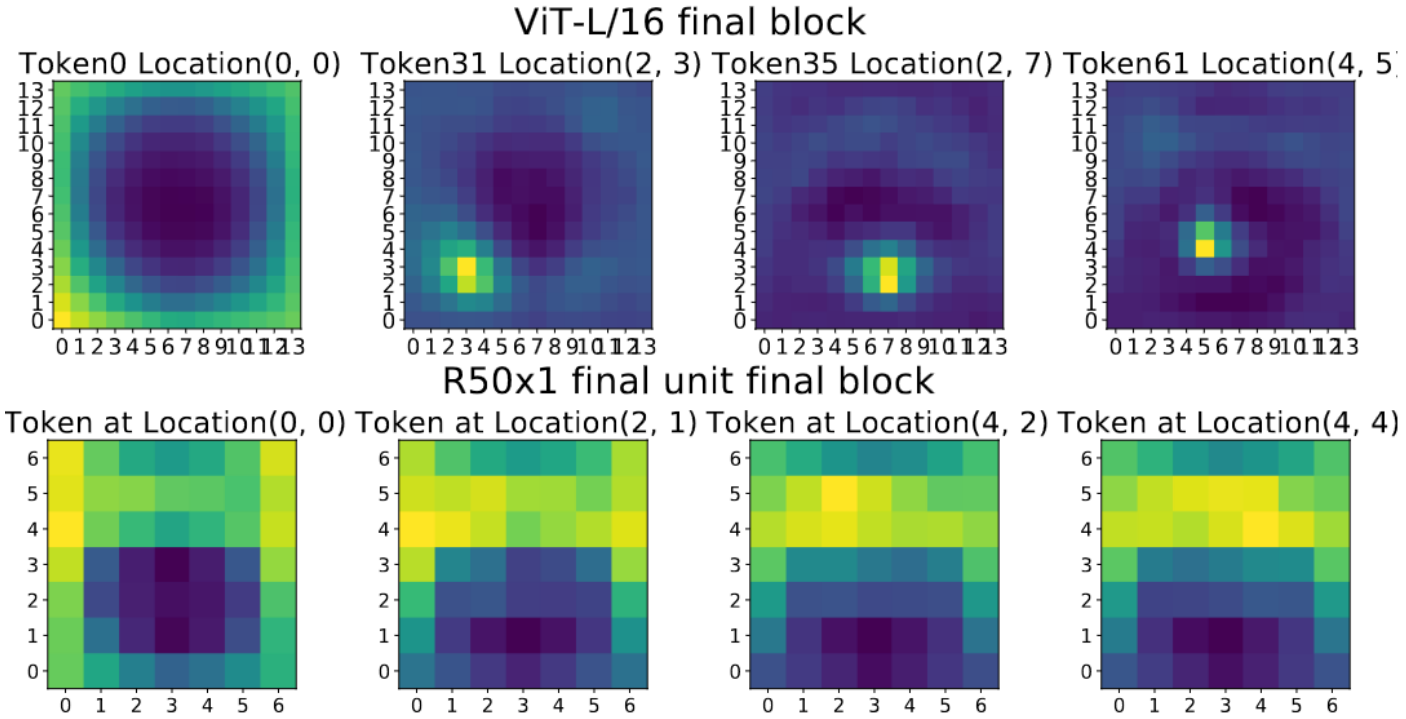


[Raghu, 2021] **Do vision transformers see like convolutional neural networks?**, Raghu, M. et al., *NeuRIPS*, *2021*

ViT vs. ResNet: spatial localization

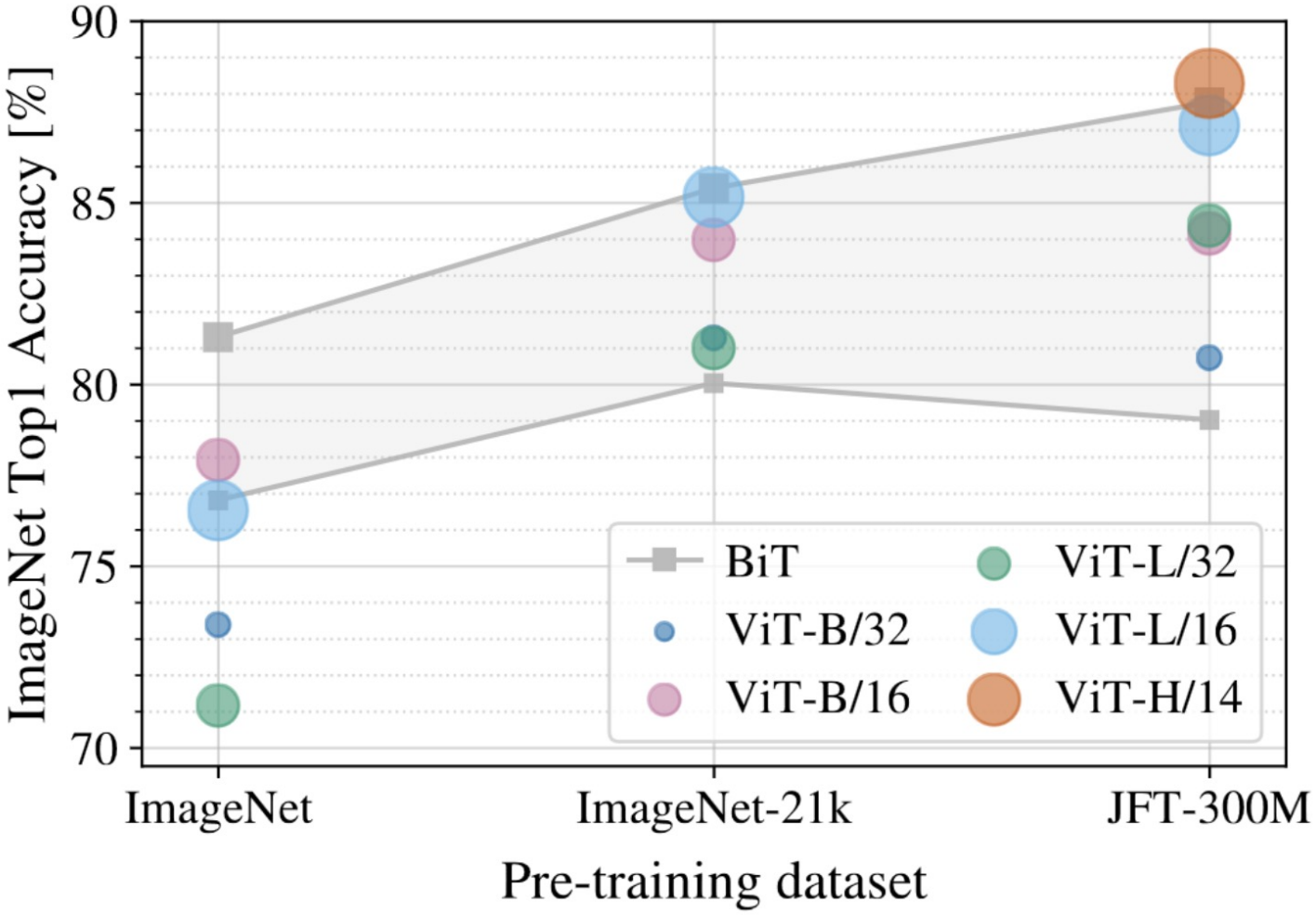
- We can evaluate **how much location information persists** in final ViT spatial token embeddings by computing (CKA) similarity with input patches

- ViT's spatial tokens retain much more spatial location information than ResNet's feature maps
  - The [cls] token oversees the aggregation of all the spatial context



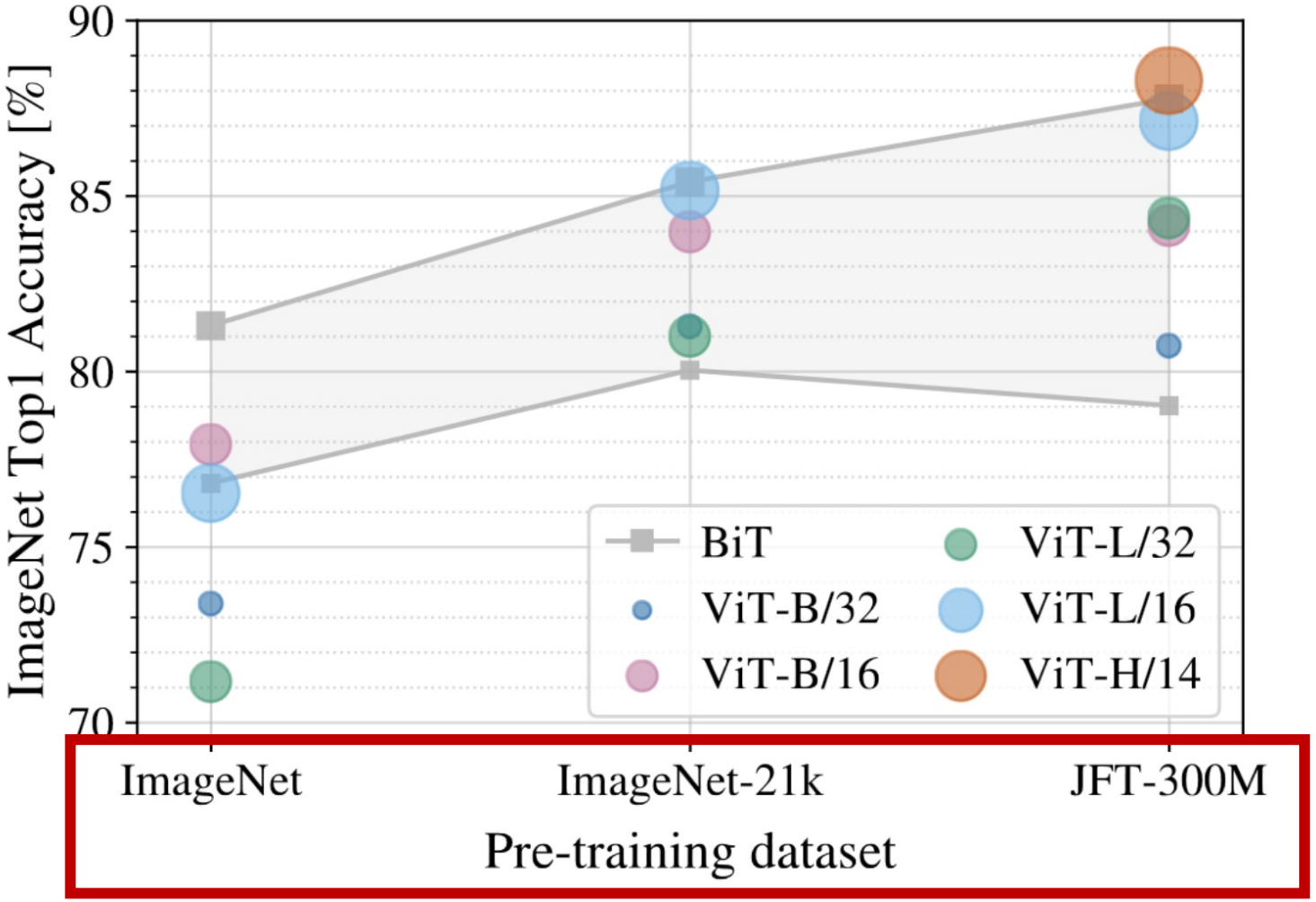
[Raghu, 2021] **Do vision transformers see like convolutional neural networks?**, Raghu, M. et al., *NeuRIPS*, *2021*

ViT vs. ResNet: Performance



[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

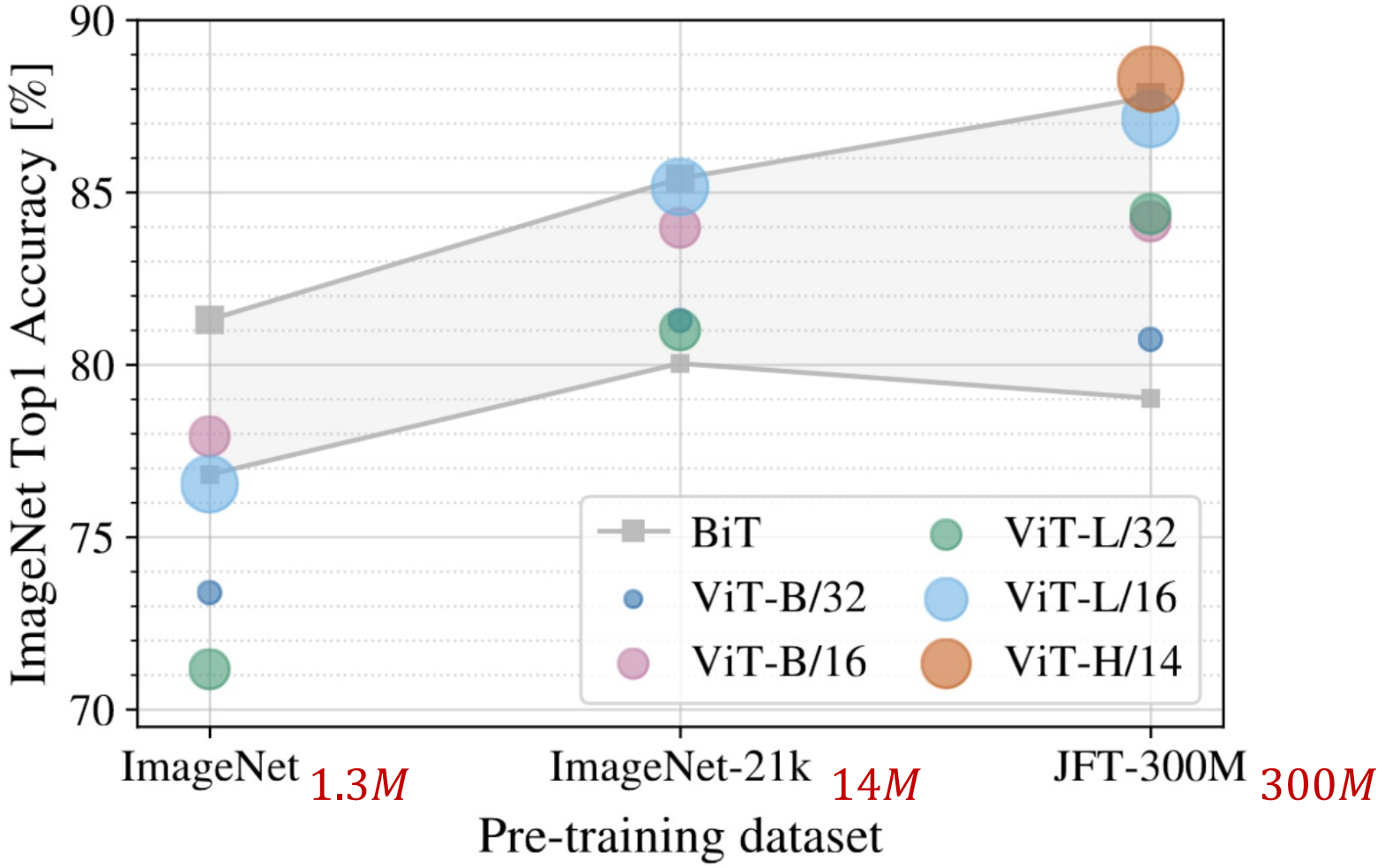
ViT vs. ResNet: Performance



Pre-trained on X, fine-tuned on ImageNet-1K

[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

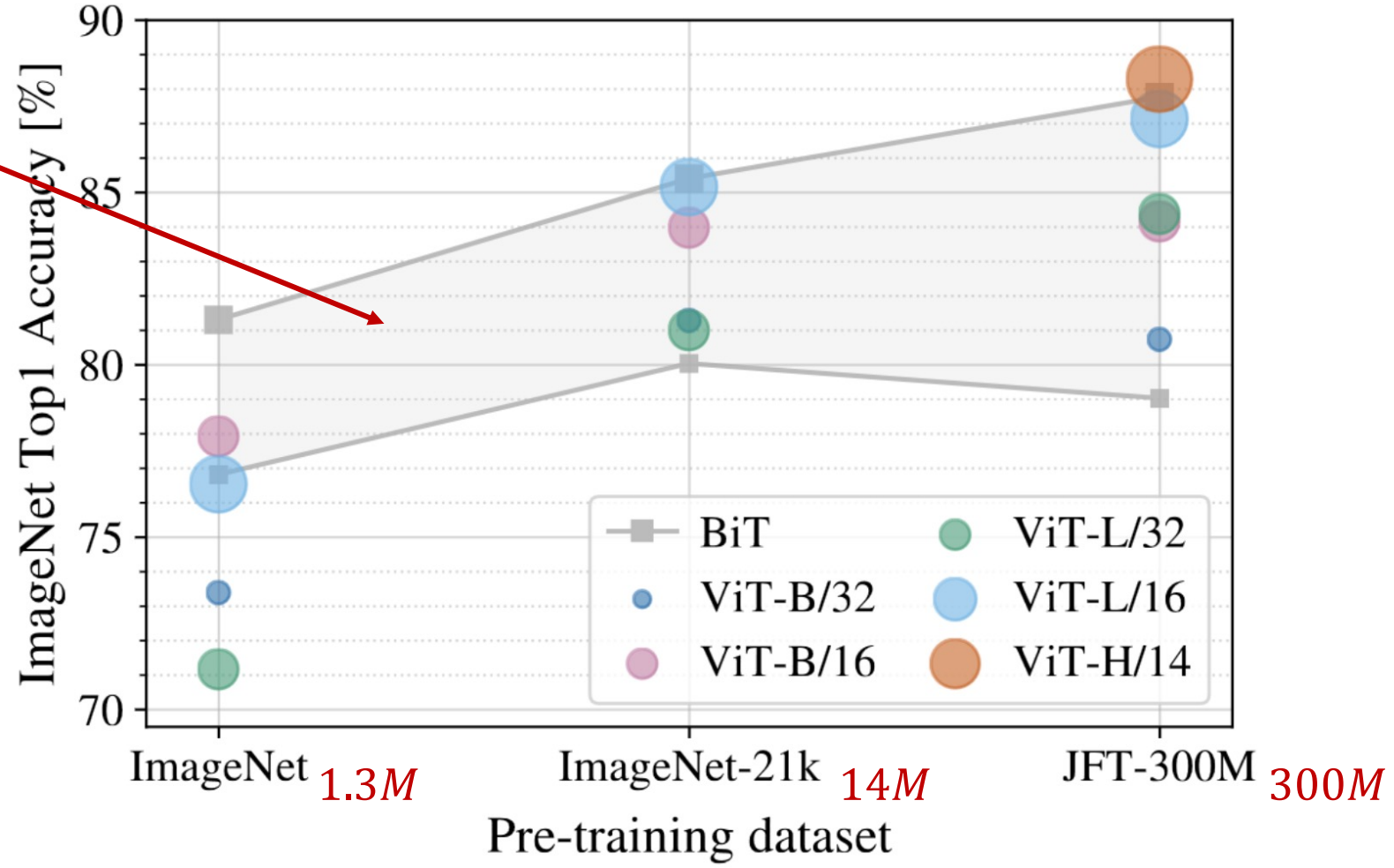
ViT vs. ResNet: Performance



[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

ViT vs. ResNet: Performance

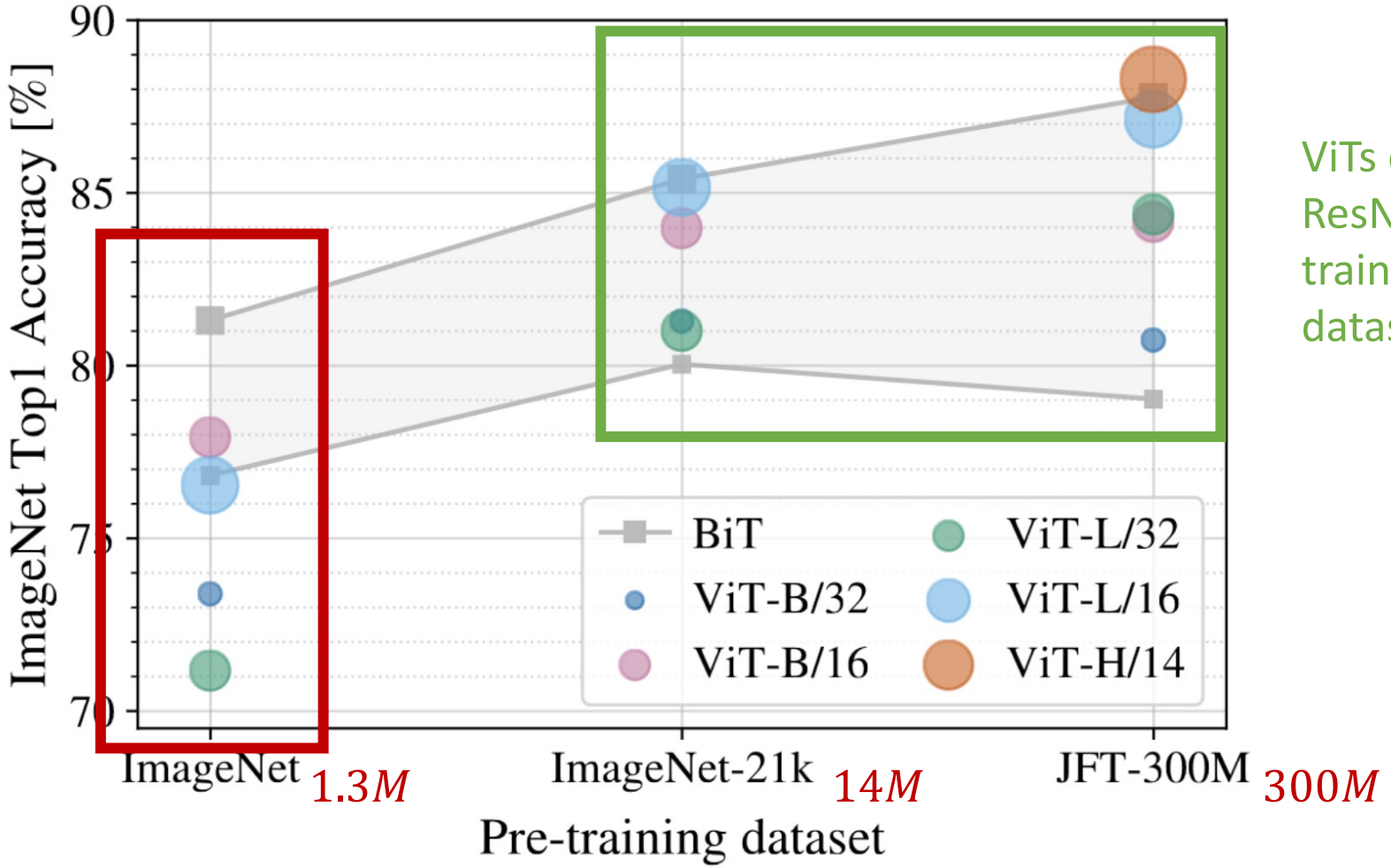
Accuracy range of various ResNet models (from ResNet50x1 up to ResNet200x3) trained with “Big Transfer” [Kolesnikov, 2020] recipe



[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*
[Kolesnikov, 2020] **Big transfer (bit): General visual representation learning**, Kolesnikov, A. et al., *ECCV*, *2020*

ViT vs. ResNet: Performance

ViTs do not match ResNets when trained on ImageNet-1K

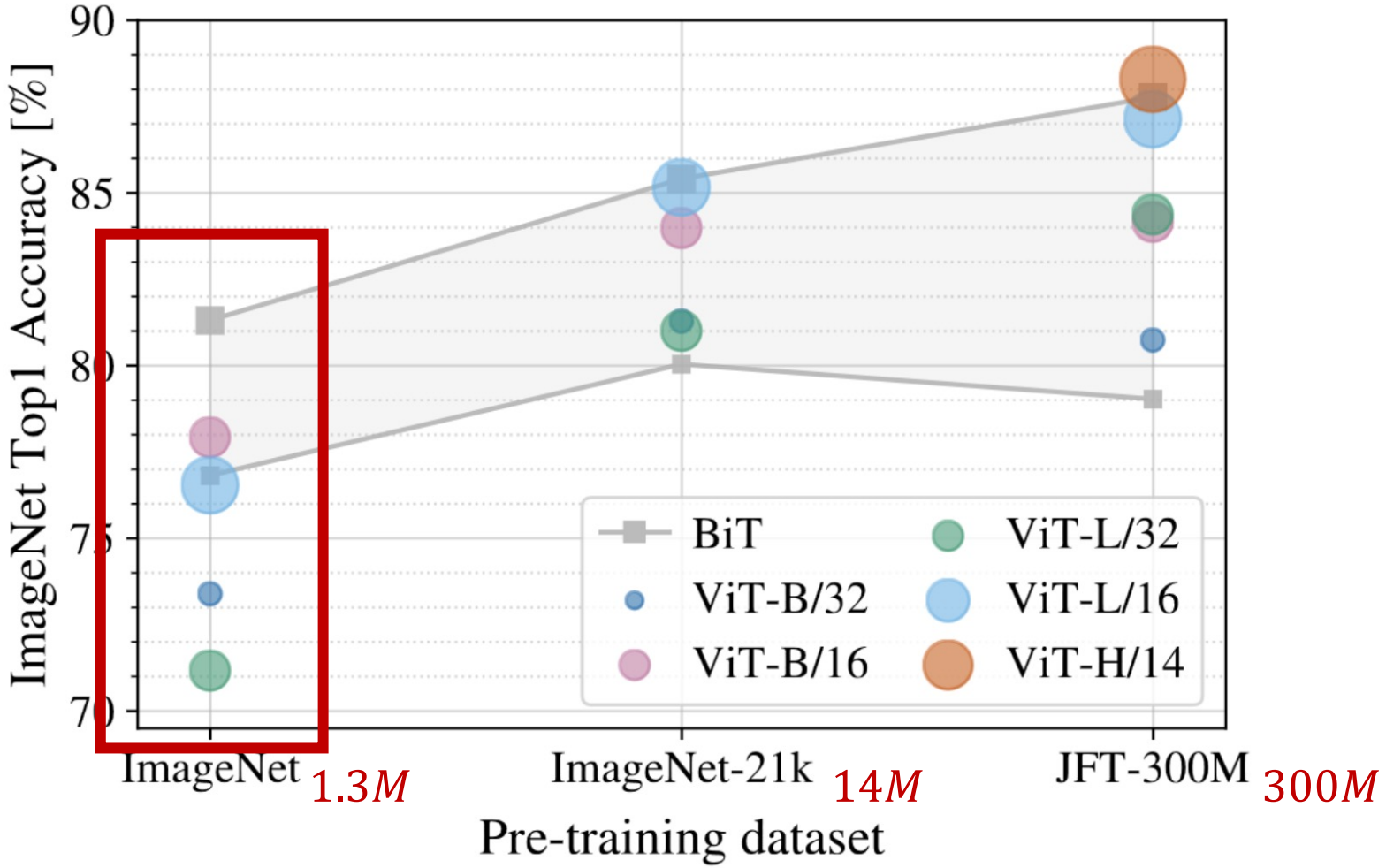


ViTs overtake ResNets when trained on massive datasets

[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

ViT vs. ResNet: Performance

How to further improve ViTs in this regime?



[Dosovitskiy, 2021] **An image is worth 16x16 words: Transformers for image recognition at scale**, Dosovitskiy, A. et al., *ICLR*, *2021*

Improving Vision Transformers

Strong augmentations and regularization

- Use **data augmentation**:
  - MixUp
  - RandAugment

- Use **regularization**:
  - Dropout (in MLP layers)
  - Weight Decay
  - Stochastic Depth

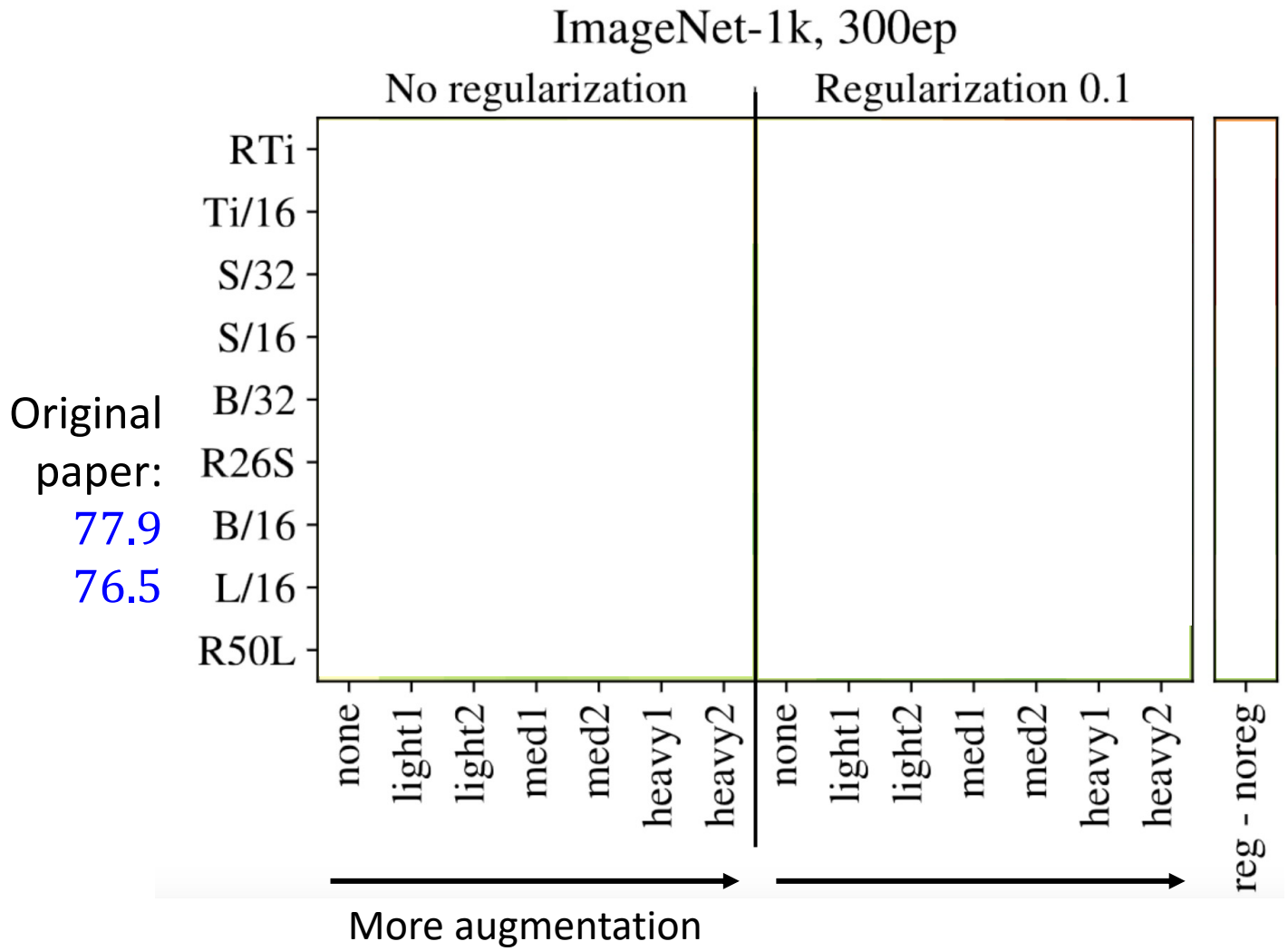
[Steiner, 2022] **How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers**, Steiner, A. P. et al., *TMLR*, *2022*

Slide inspired by Justin Johnson's lecture

Strong augmentations and regularization

- Use **data augmentation**:
  - MixUp
  - RandAugment

- Use **regularization**:
  - Dropout (in MLP layers)
  - Weight Decay
  - Stochastic Depth

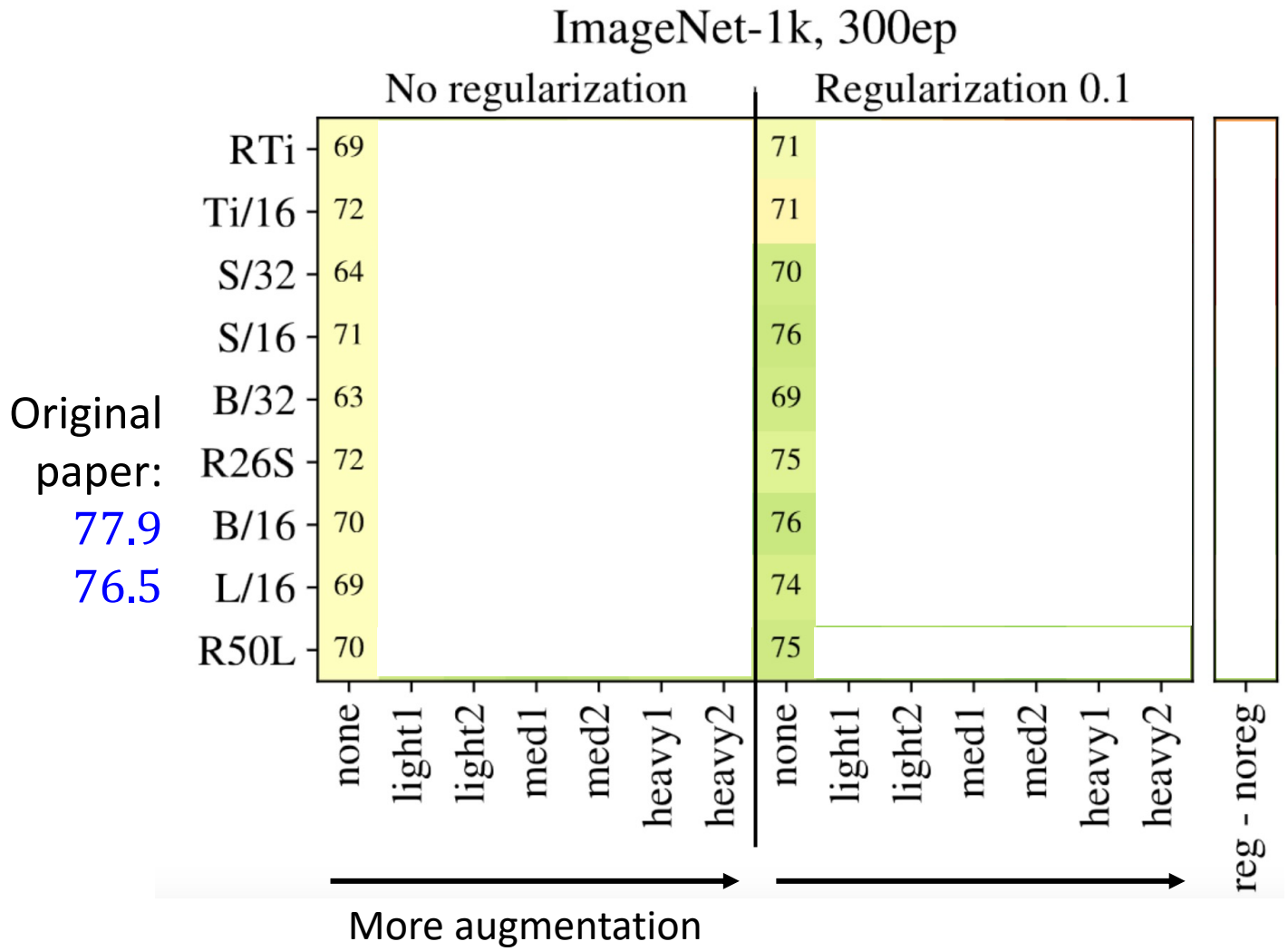


[Steiner, 2022] **How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers**, Steiner, A. P. et al., *TMLR*, *2022*

Slide inspired by Justin Johnson's lecture

Strong augmentations and regularization

- Use **data augmentation**:
  - MixUp
  - RandAugment
- Use **regularization**:
  - Dropout (in MLP layers)
  - Weight Decay
  - Stochastic Depth

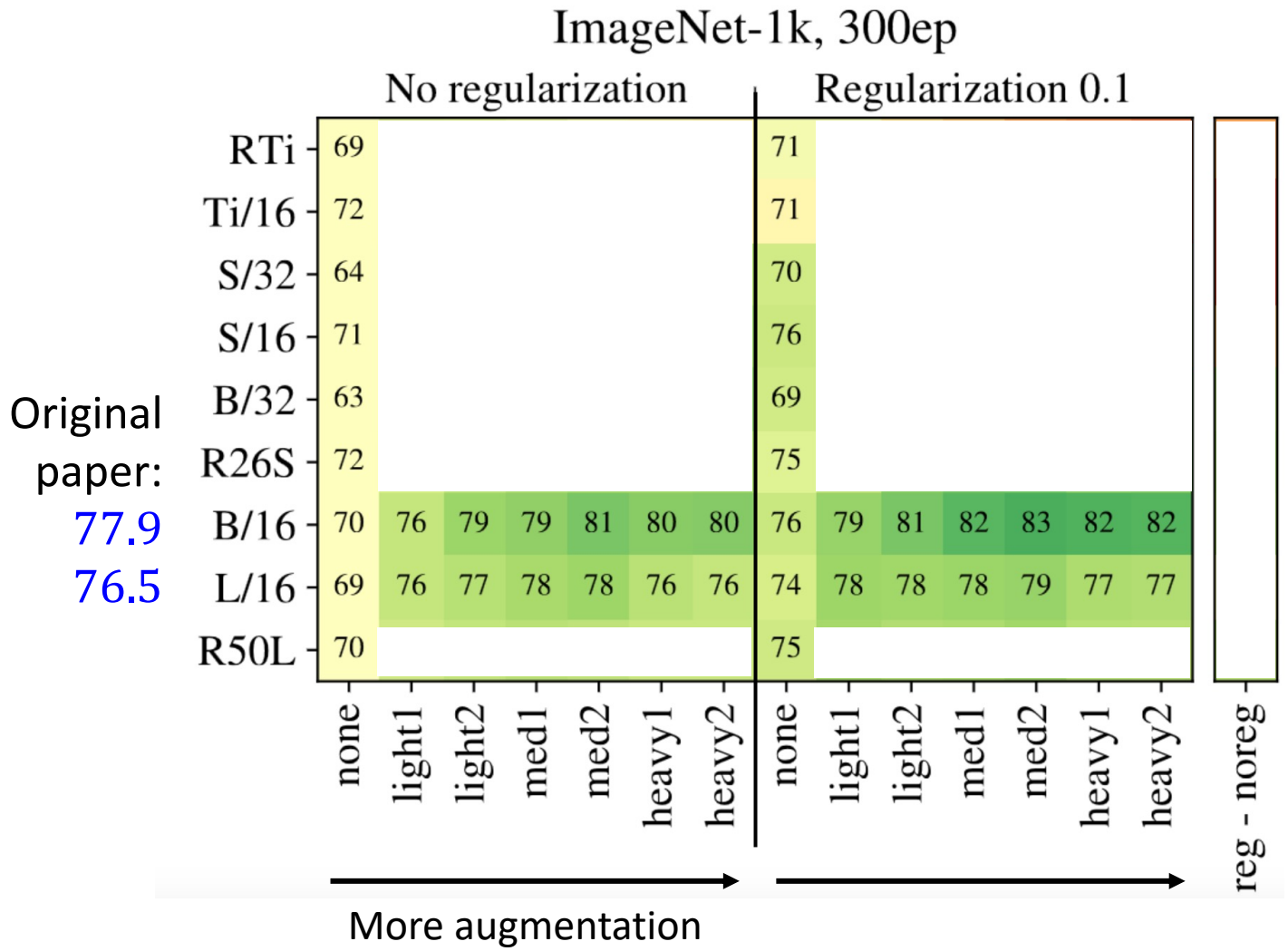


[Steiner, 2022] **How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers**, Steiner, A. P. et al., *TMLR*, *2022*

Slide inspired by Justin Johnson's lecture

Strong augmentations and regularization

- Use **data augmentation**:
  - MixUp
  - RandAugment
- Use **regularization**:
  - Dropout (in MLP layers)
  - Weight Decay
  - Stochastic Depth



Original paper:
77.9
76.5

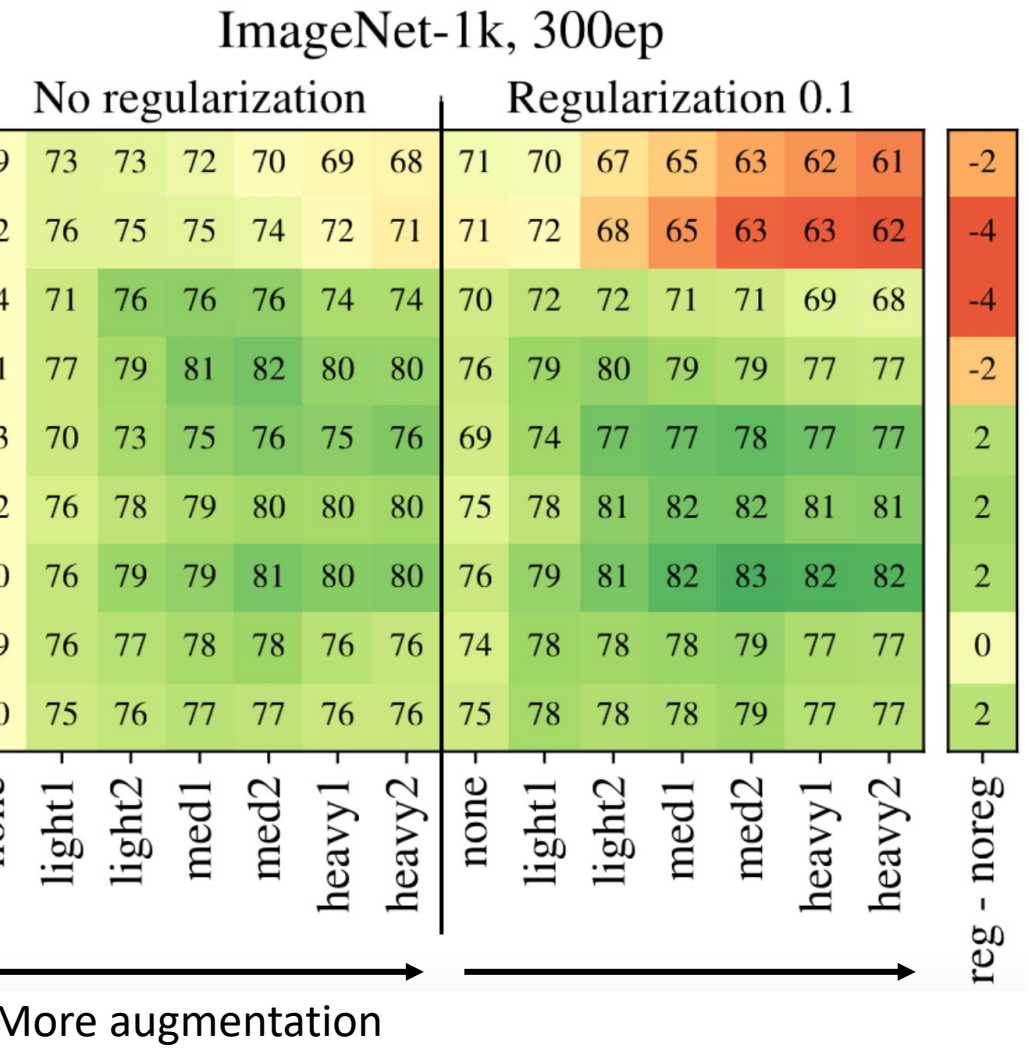
[Steiner, 2022] **How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers**, Steiner, A. P. et al., *TMLR*, *2022*

Slide inspired by Justin Johnson's lecture

Strong augmentations and regularization

- Use **data augmentation**:
  - MixUp
  - RandAugment
- Use **regularization**:
  - Dropout (in MLP layers)
  - Weight Decay
  - Stochastic Depth

Original paper:
77.9
76.5

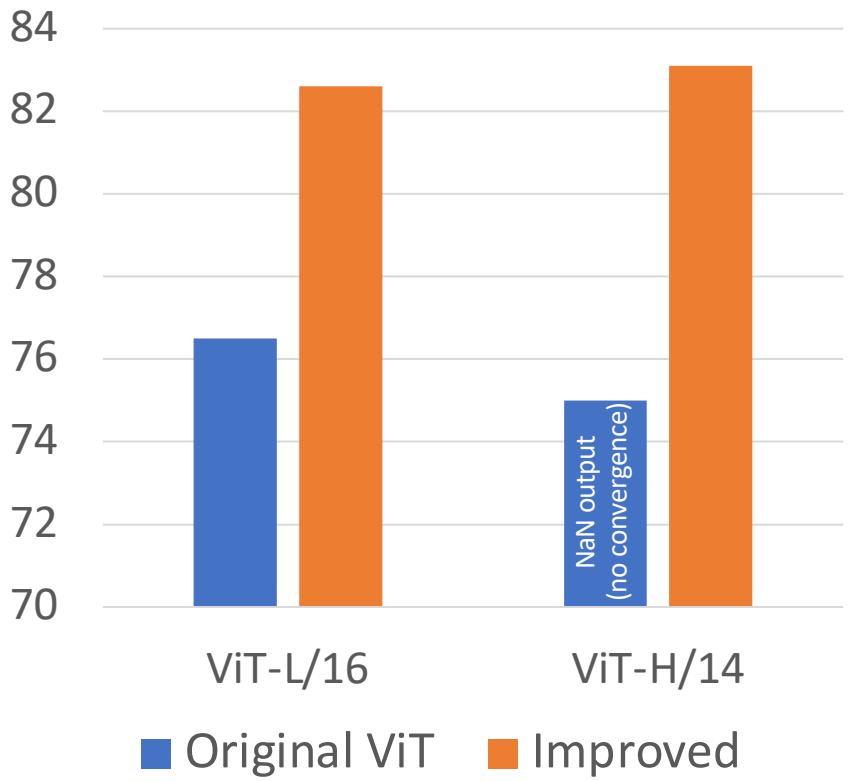


[Steiner, 2022] **How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers**, Steiner, A. P. et al., *TMLR*, *2022*

Slide inspired by Justin Johnson's lecture

Similar recipe from [He, 2022]

config	value
optimizer	AdamW
base learning rate	1e-4
weight decay	0.3
optimizer momentum	β_1, β_2=0.9, 0.95
batch size	4096
learning rate schedule	cosine decay
warmup epochs	20
training epochs	300 (B), 200 (L/H)
augmentation	RandAug (9, 0.5) [12]
label smoothing [52]	0.1
mixup [69]	0.8
cutmix [68]	1.0
drop path [30] = Stoch. Depth	0.1 (B), 0.2 (L/H)
exp. moving average (EMA)	0.9999



[He, 2022] **Masked autoencoders are scalable vision learners**, He, K. et al., *CVPR*, *2022*

DeiT

- **DeiT** [Touvron, 2021] **adds (hard) knowledge distillation** to improve the training of ViTs

Step 1: Train a teacher model (CNN) on ground-truth labels



CNN

$P(\text{bird}) = 0.95$
$P(\text{plane}) = 0.05$

→ Cross-Entropy Loss ← $y = \text{'bird'}$

Step 2: Train a student model (ViT) to match (a) predictions from the teacher; and (b) ground-truth labels



CNN

$P(\text{bird}) = 0.2$
$P(\text{plane}) = 0.8$

→ $y_T = \text{'plane'}$ → Cross-Entropy Loss (with Label Smoothing)



ViT

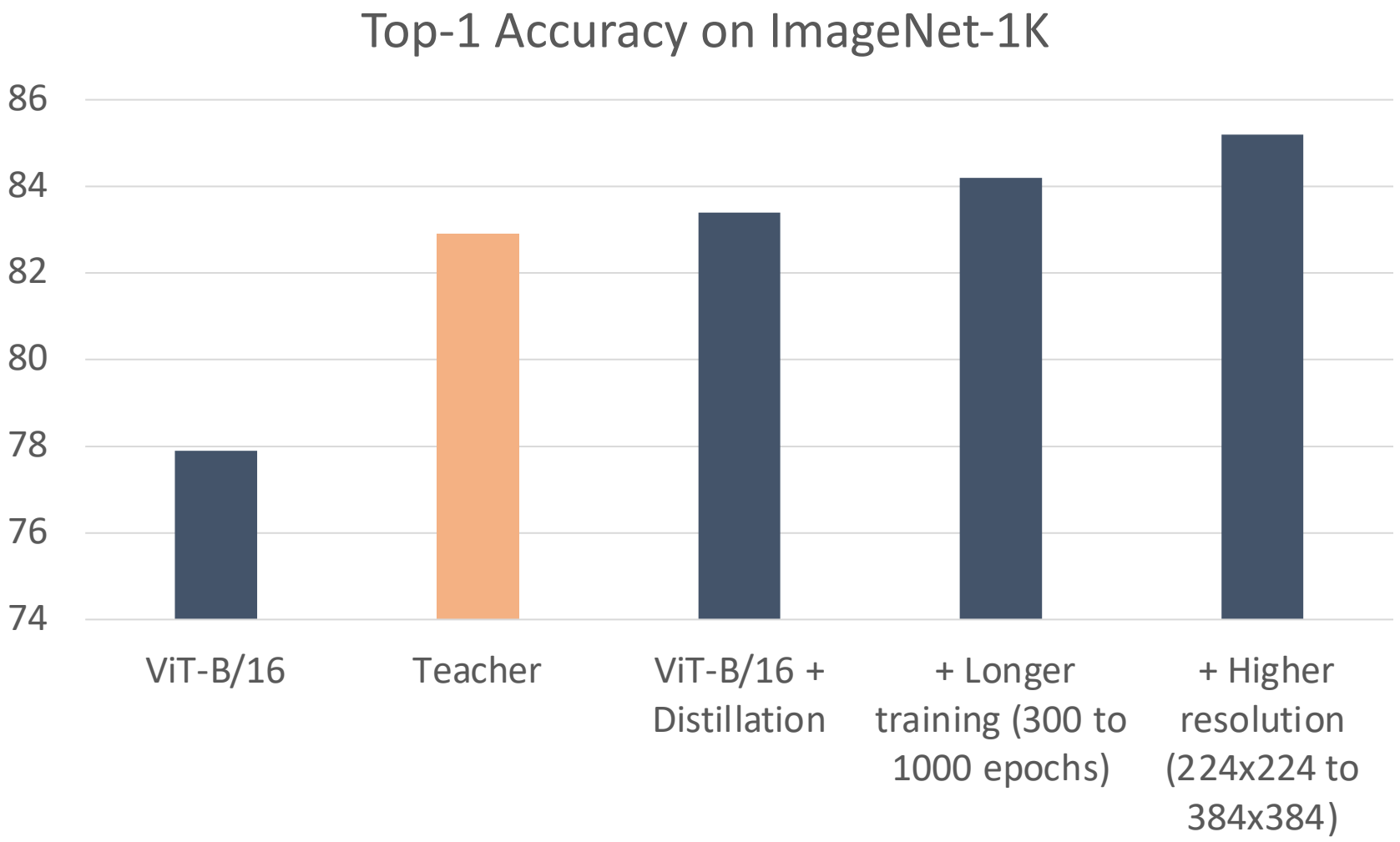
$P(\text{bird}) = 0.25$
$P(\text{plane}) = 0.75$

→ Cross-Entropy Loss ← $y = \text{'plane'}$

[Touvron, 2021] **Training data-efficient image transformers & distillation through attention**, Touvron, H. et al., *ICML*, *2021*

Bird & Plane images from wikipedia

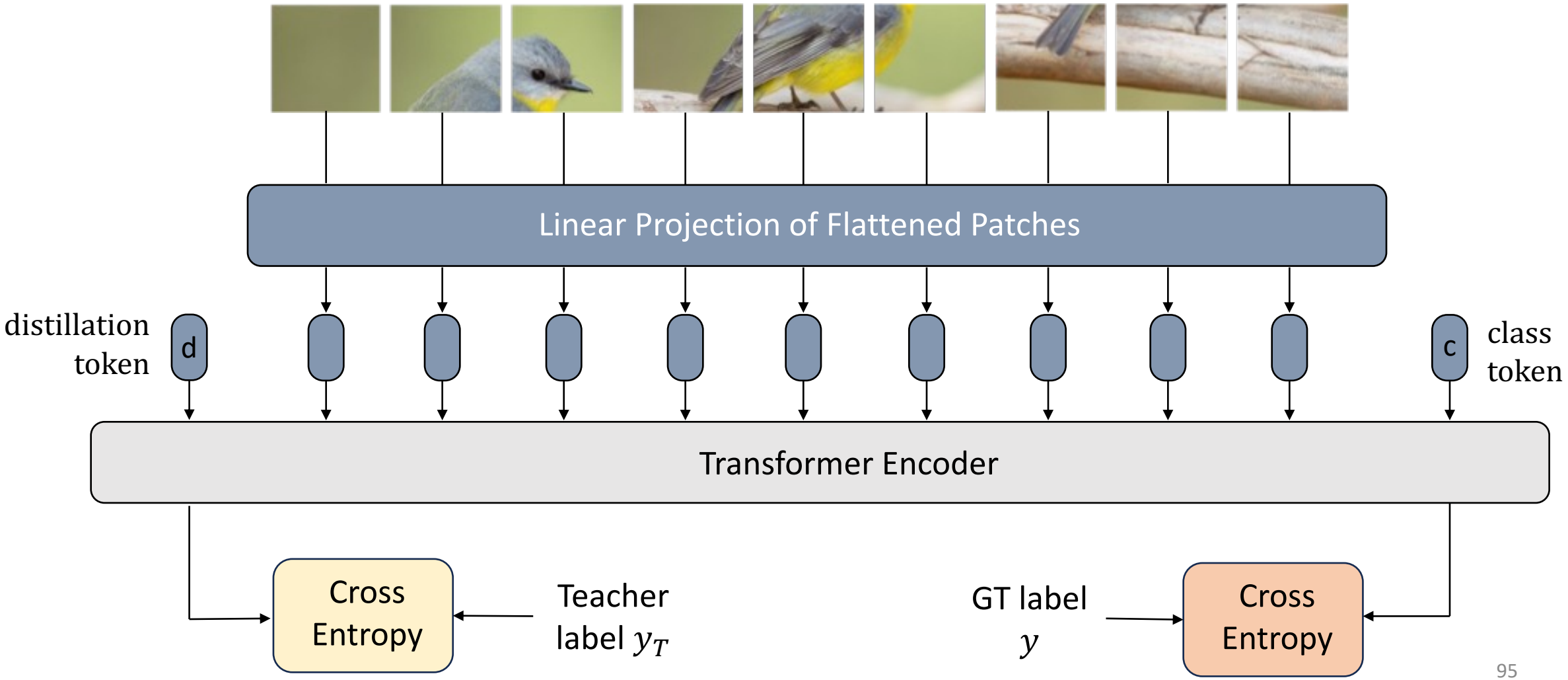
DeiT



[Touvron, 2021] **Training data-efficient image transformers & distillation through attention**, Touvron, H. et al., *ICML*, *2021*

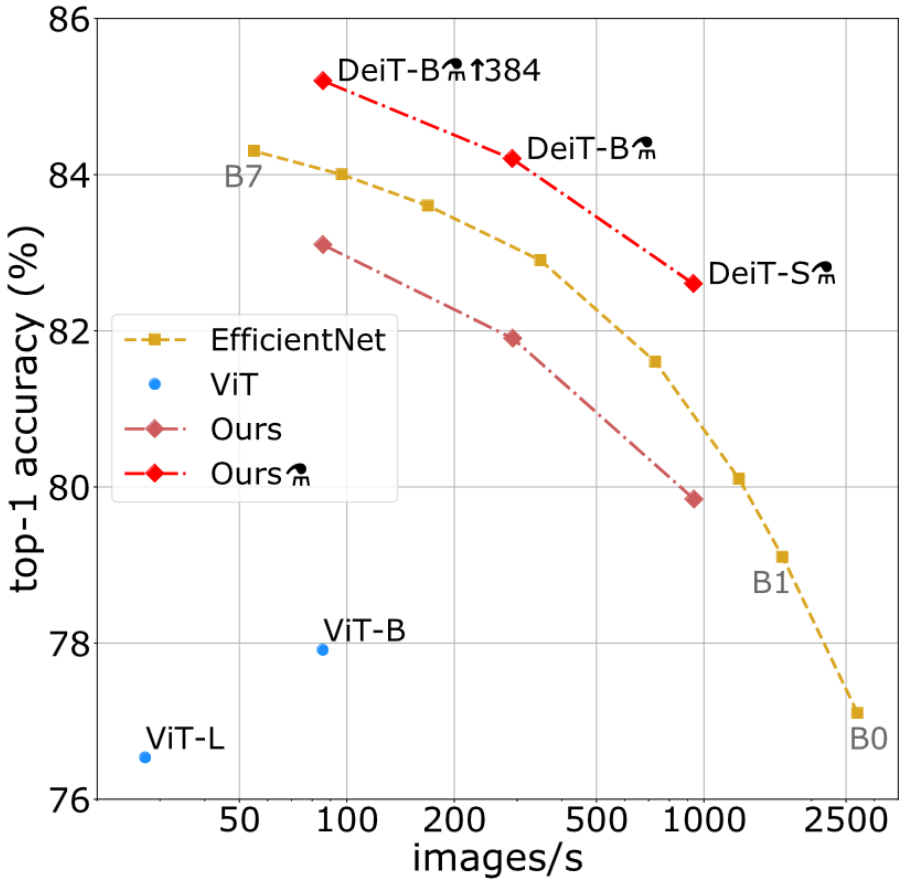
DeiT

- DeiT specializes distillation to the ViT framework by adding a **dedicated distillation token**



DeiT

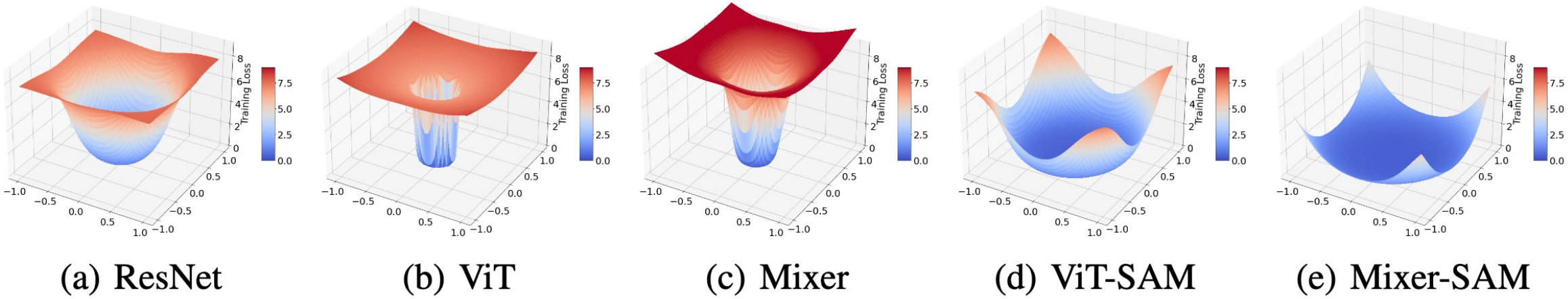
- DeiT is on par with, or slightly surpasses state-of-the-art CNNs (EfficientNet)



[Touvron, 2021] **Training data-efficient image transformers & distillation through attention**, Touvron, H. et al., *ICML*, *2021*

Sharpness-Aware Minimization (SAM)

- [Chen, 2022] observes that standard ViT training **converges to sharper local minima** in parameter space than for ResNets
- It is known that **convergence to flat regions** with small curvature **benefits generalization**
- SAM modifies the training objective to converge to flatter local minima



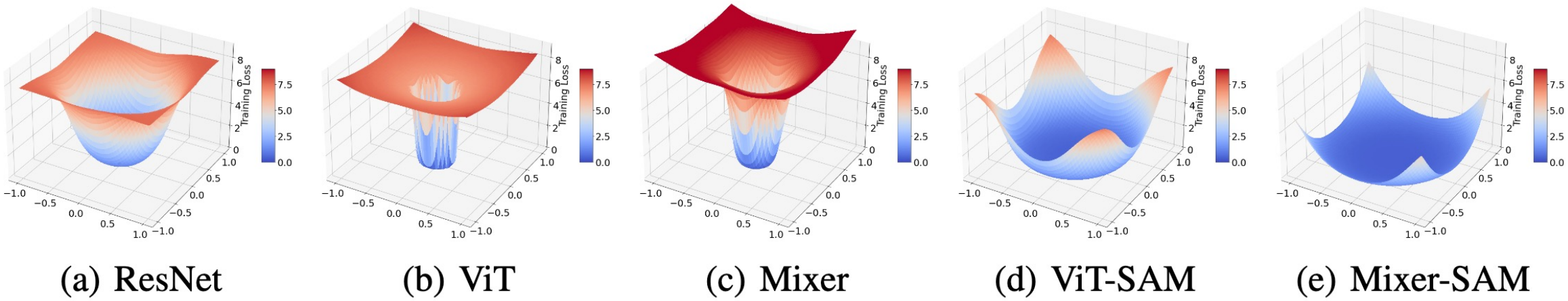
[Chen, 2022] **When vision transformers outperform resnets without pre-training or strong data augmentations**, Chen, X. et al., *ICLR*, *2022*

Sharpness-Aware Minimization (SAM)

- SAM training objective:

$$\underset{w}{\operatorname{argmin}} \max_{\|\epsilon\|_2 \leq \rho} \mathcal{L}_{\text{train}}(w + \epsilon)$$

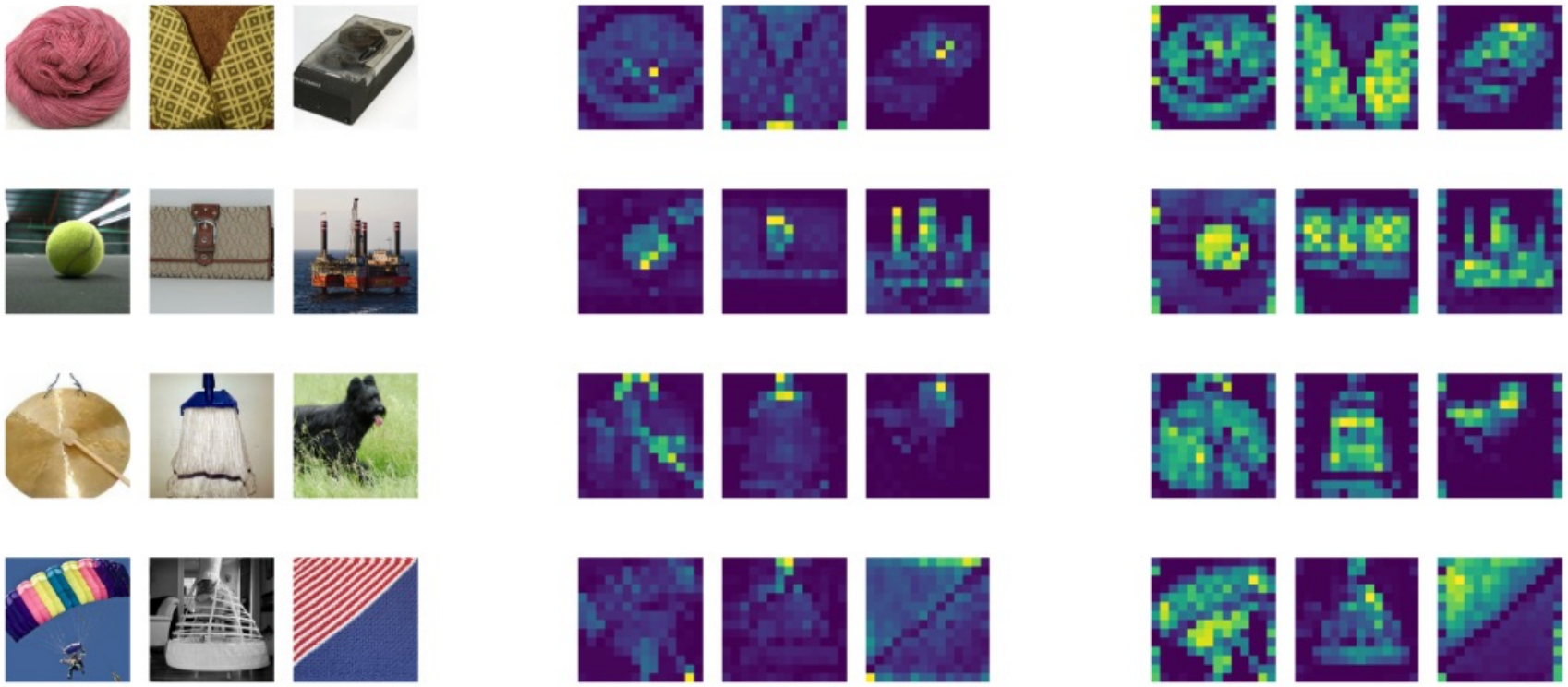
- [Chen, 2022] shows a $+5.3\%$ top-1 accuracy boost on ImageNet with this method, without the need for strong augmentations



[Chen, 2022] **When vision transformers outperform resnets without pre-training or strong data augmentations**, Chen, X. et al., *ICLR*, *2022*

Sharpness-Aware Minimization (SAM)

- ViTs trained with SAM have more interpretable attention maps



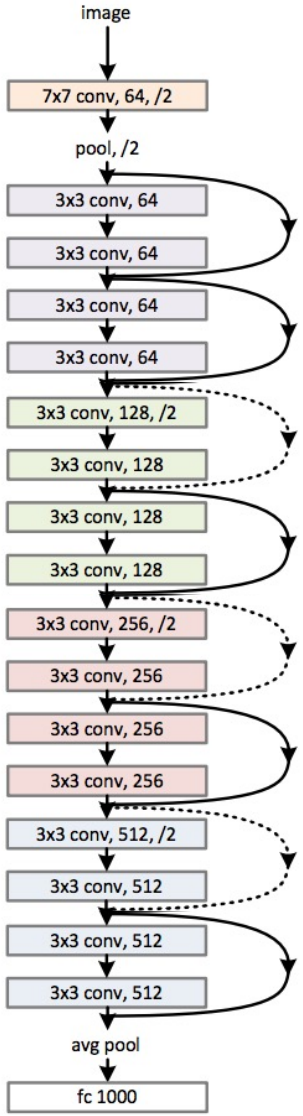
Input images

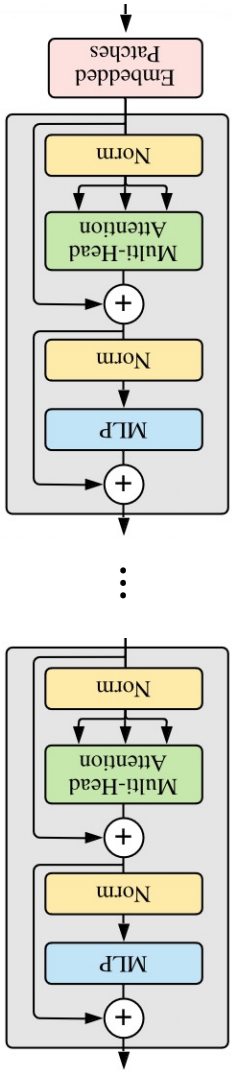
Attention maps, without SAM

Attention maps, with SAM

[Chen, 2022] **When vision transformers outperform resnets without pre-training or strong data augmentations**, Chen, X. et al., *ICLR*, *2022*

ViT vs. ResNet: architecture





ViT vs. ResNet: architecture

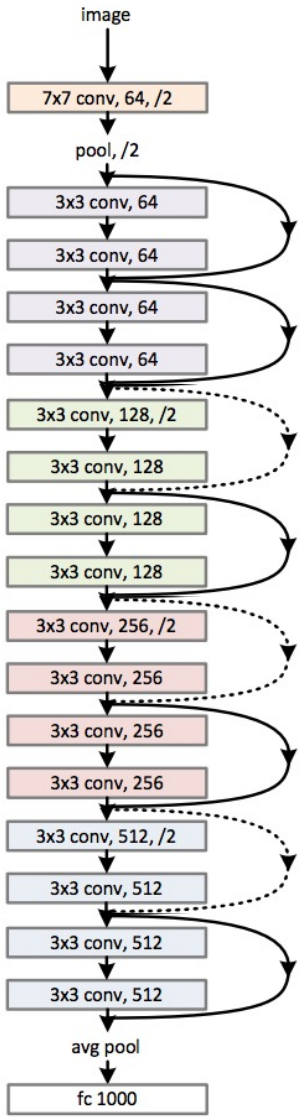
Input:
224×224×3

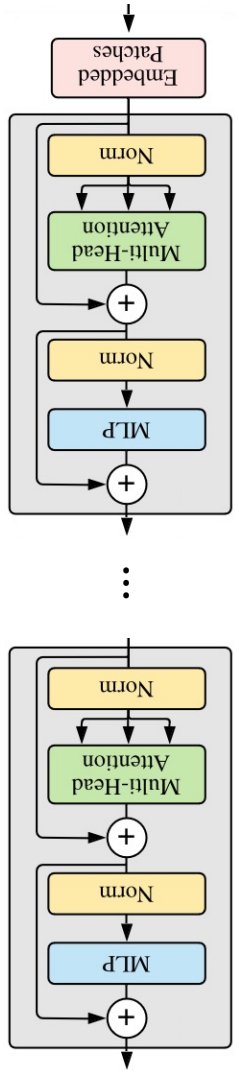
Stage 1:
56×56×64

Stage 2:
28×28×128

Stage 3:
14×14×256

Stage 4:
7×7×512





ViT vs. ResNet: architecture

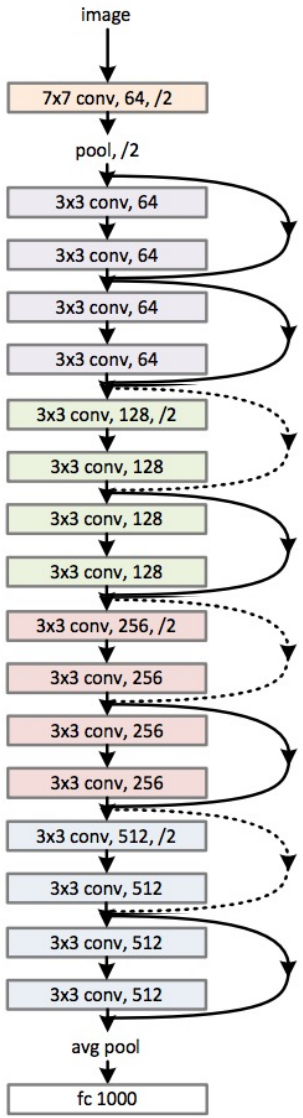
Input:
224×224×3

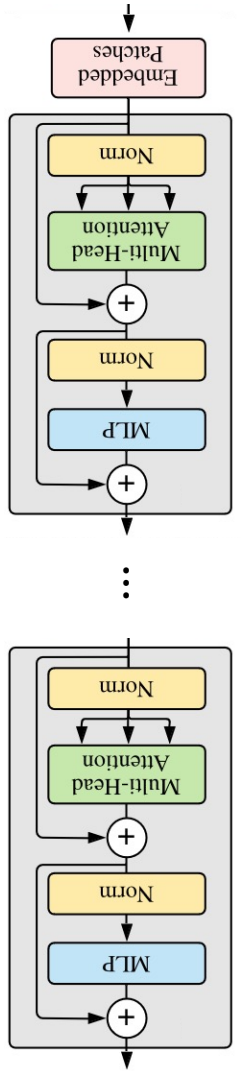
Stage 1:
56×56×64

Stage 2:
28×28×128

Stage 3:
14×14×256

Stage 4:
7×7×512





Input:
224×224×3

Block 1:
14×14×768

Block L:
14×14×768

ViT vs. ResNet: architecture

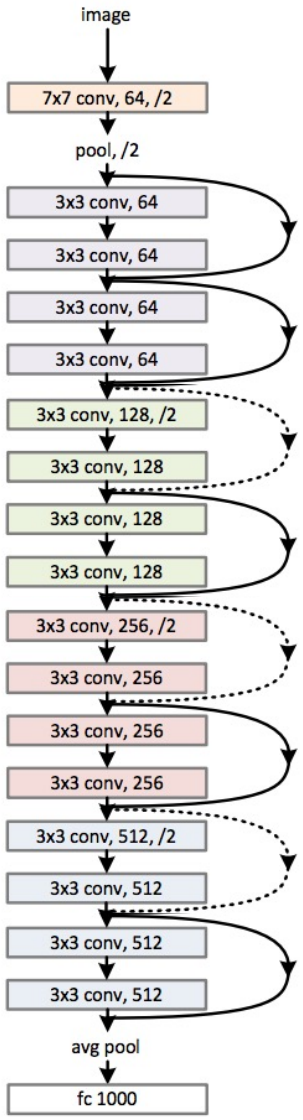
Input:
$224 \times 224 \times 3$

Stage 1:
$56 \times 56 \times 64$

Stage 2:
$28 \times 28 \times 128$

Stage 3:
$14 \times 14 \times 256$

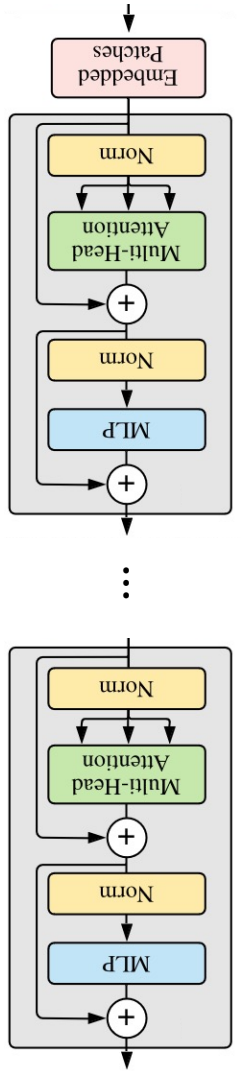
Stage 4:
$7 \times 7 \times 512$



In **ResNets**, decrease resolution and increase channels as we go deeper in the network
→ Hierarchical design

In **ViTs**, all blocks have the same resolution and channel count

Can we build a **hierarchical ViT**?



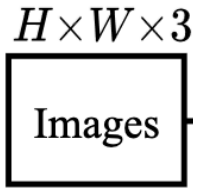
Input:
$224 \times 224 \times 3$

Block 1:
$14 \times 14 \times 768$

Block L:
$14 \times 14 \times 768$

Swin Transformer

- The input image is of size $H \times W \times 3$ (RGB channels)

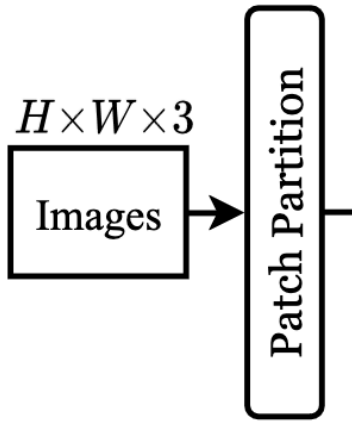


[Liu, 2021] **Swin transformer: Hierarchical vision transformer using shifted windows**, Liu, Z. et al., *ICCV*, *2021*

Swin Transformer

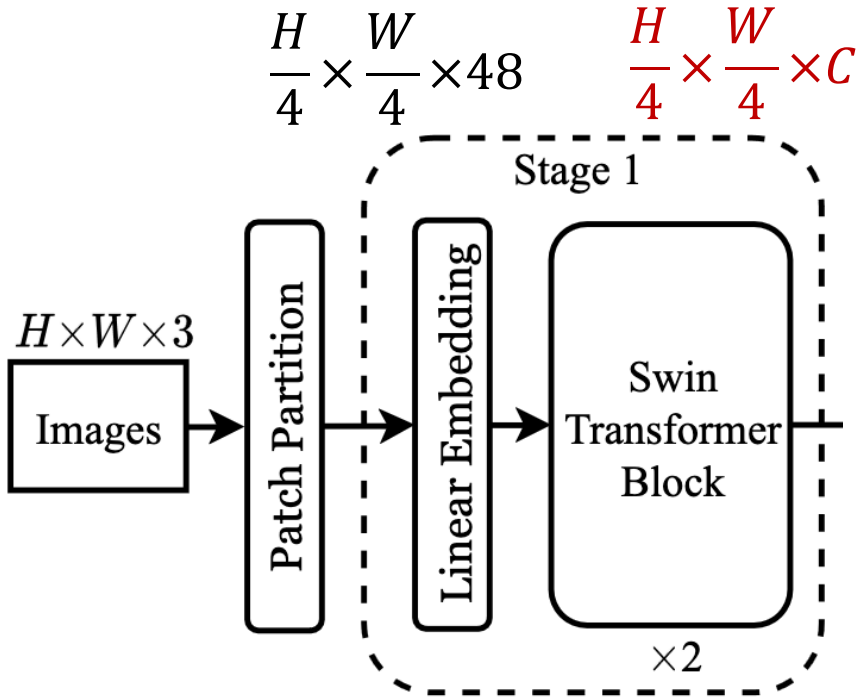
- Concatenate groups of 4×4 pixels to create patches/tokens, each of dimension $4\times4\times3 = 48$

$$\frac{H}{4}\times\frac{W}{4}\times48$$



Swin Transformer

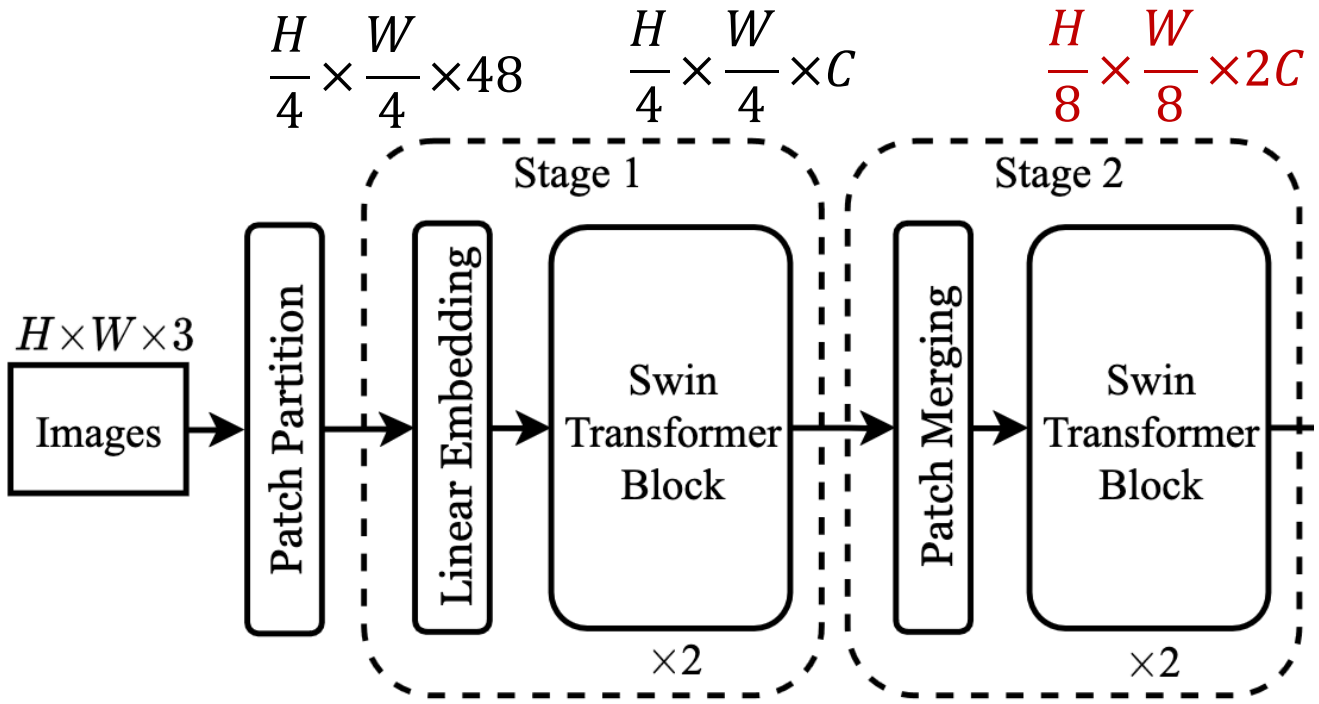
- Project tokens from 48 to C dimensions via a Linear layer (e.g., $C := 96$ for Swin-T/Swin-S)
- Then apply Swin Transformer blocks (more on that in a minute); they don't change the tensor shape



[Liu, 2021] **Swin transformer: Hierarchical vision transformer using shifted windows**, Liu, Z. et al., *ICCV, 2021*

Swin Transformer

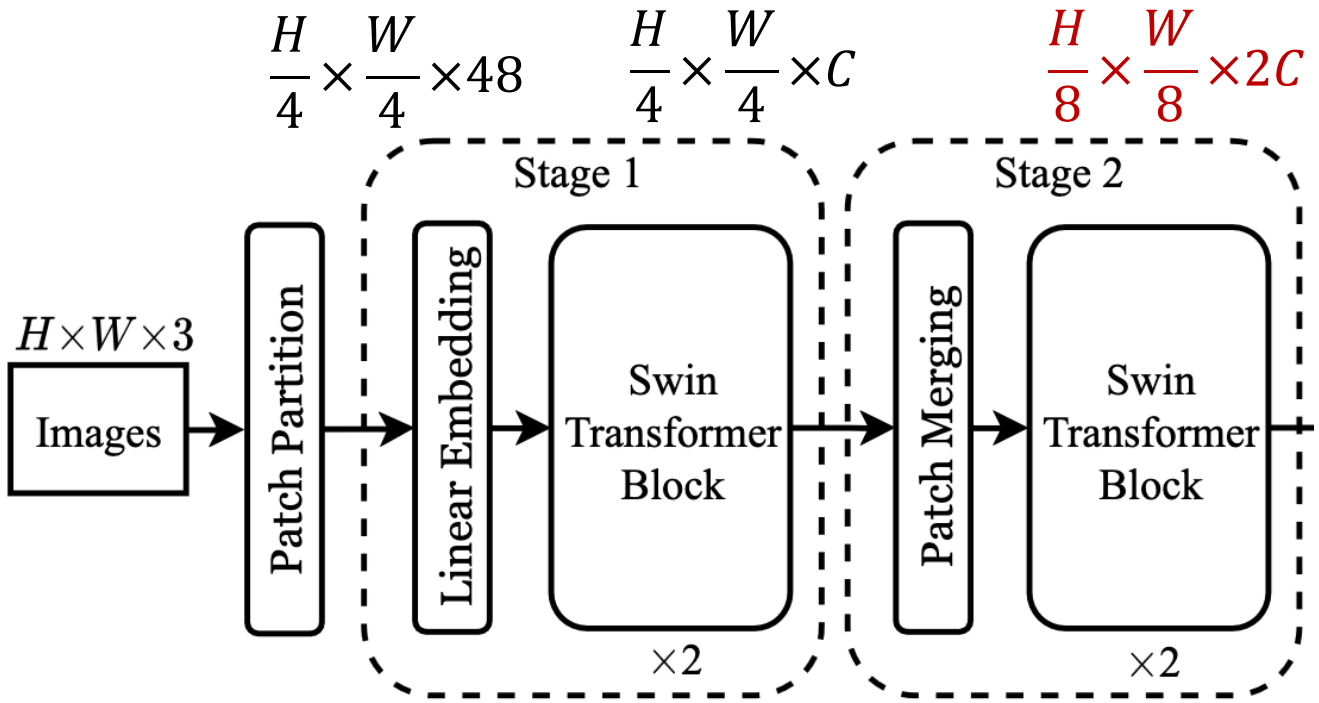
- Merge 2×2 neighborhoods → patches (tokens) are effectively 8×8, with doubled dimension $2C$

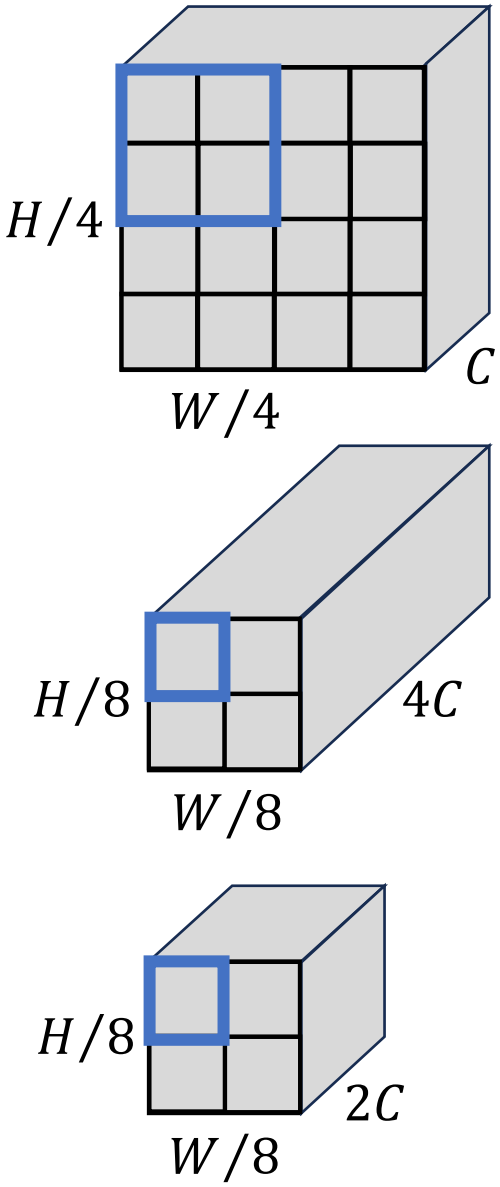


[Liu, 2021] **Swin transformer: Hierarchical vision transformer using shifted windows**, Liu, Z. et al., *ICCV, 2021*

Swin Transformer

- Merge 2×2 neighborhoods → patches (tokens) are effectively 8×8, with doubled dimension $2C$





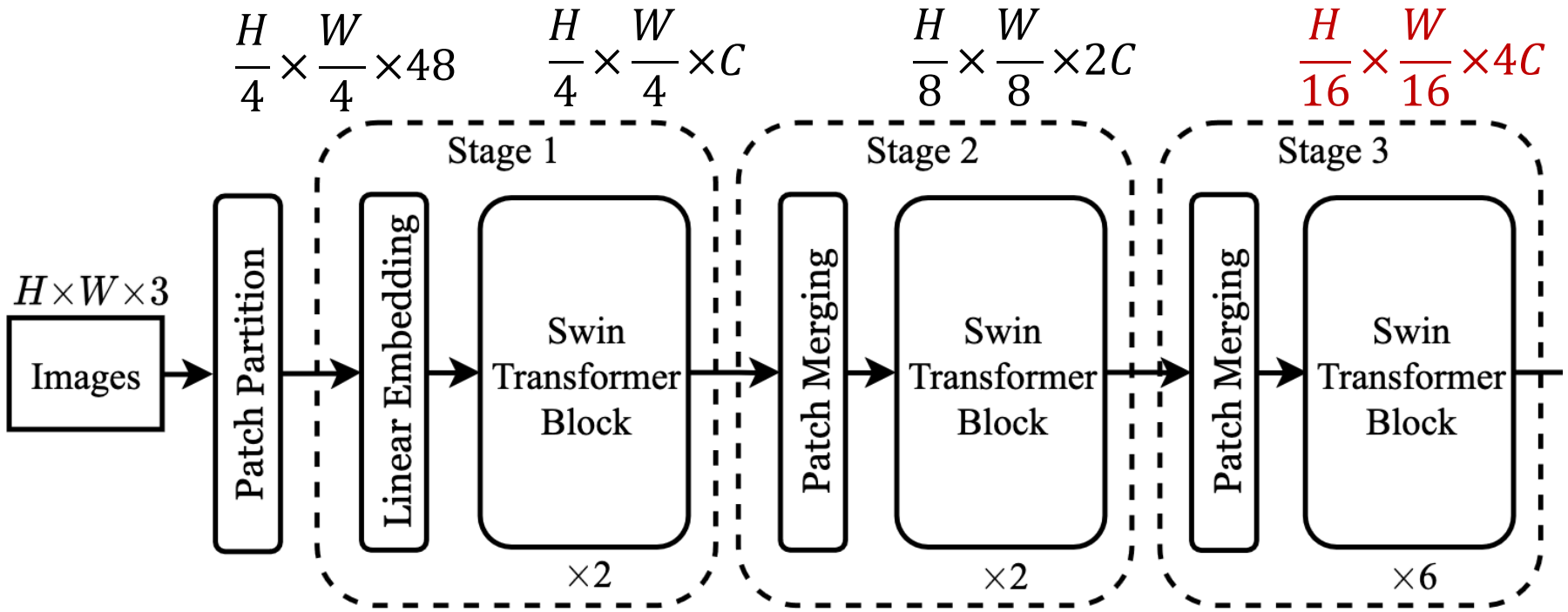
Concatenate groups of 2×2 tokens

Linearly project from $4C$ to $2C$ dimensions

[Liu, 2021] **Swin transformer: Hierarchical vision transformer using shifted windows**, Liu, Z. et al., *ICCV, 2021*

Swin Transformer

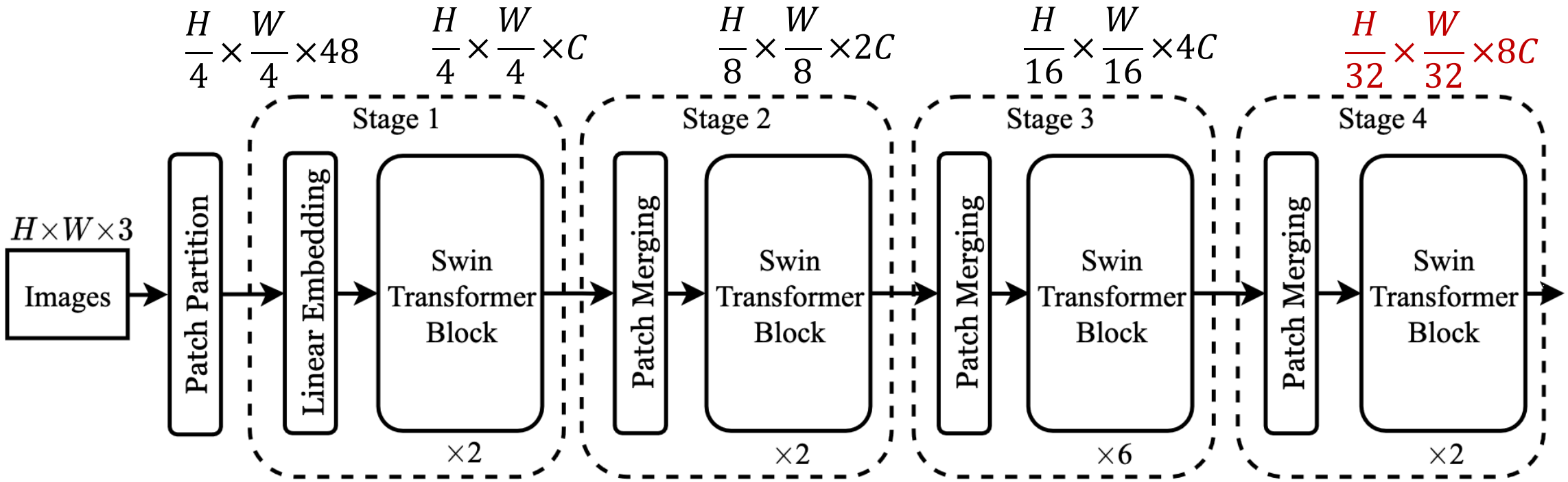
- Merge 2×2 neighborhoods → patches (tokens) are effectively 16×16, with doubled dimension $4C$



[Liu, 2021] **Swin transformer: Hierarchical vision transformer using shifted windows**, Liu, Z. et al., *ICCV, 2021*

Swin Transformer

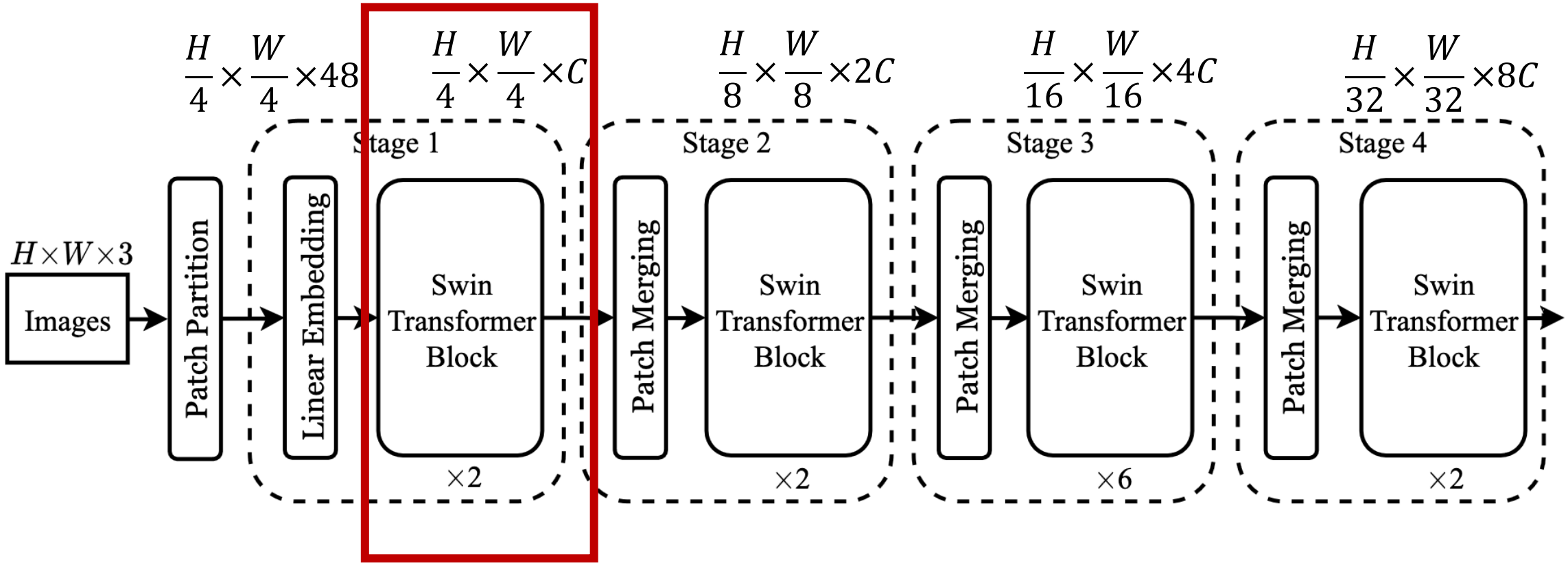
- Merge 2×2 neighborhoods → patches (tokens) are effectively 32×32, with doubled dimension $8C$



[Liu, 2021] **Swin transformer: Hierarchical vision transformer using shifted windows**, Liu, Z. et al., *ICCV, 2021*

Swin Transformer

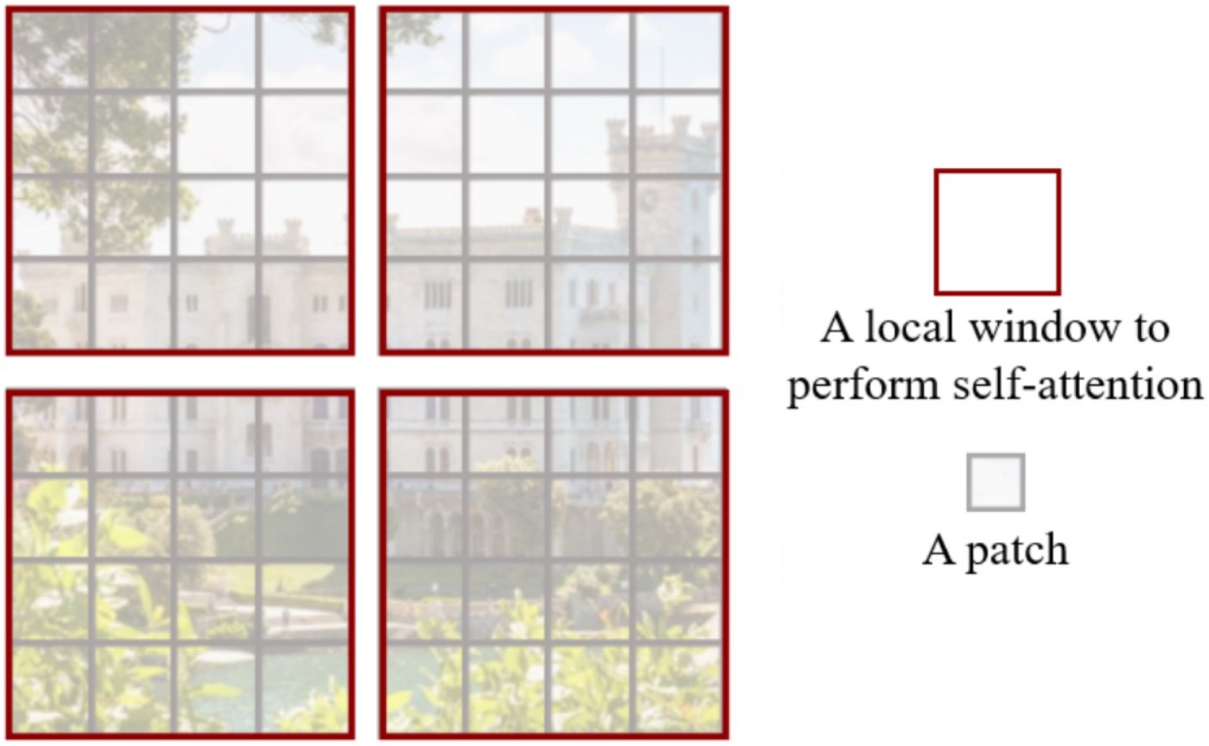
- **Problem**: 224×224 image → 56×56 tokens → attention matrix has $56^4 = 9.8M$ entries



- **Solution**: compute **attention over smaller windows**

Window Attention

- Consider a $H \times W$ grid of patches



- In the original ViT, each attention matrix takes $O((HW)^2)$ in memory

---

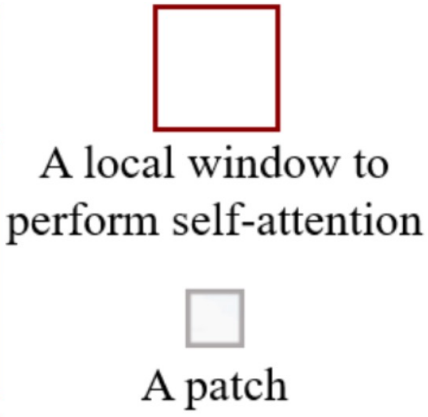
- Instead, **divide** the grid **in $M \times M$ windows**, and **compute attention separately in each window**
  - All attention parameters are shared between windows
  - In the paper, $M := 7$
- In memory, $O\left(M^4 \times \frac{H}{M} \times \frac{W}{M}\right) = O(M^2 HW)$

[Liu, 2021] **Swin transformer: Hierarchical vision transformer using shifted windows**, Liu, Z. et al., *ICCV, 2021*

Window Attention

- Consider a $H \times W$ grid of patches





- In the original ViT, each attention matrix takes $O((HW)^2)$ in memory

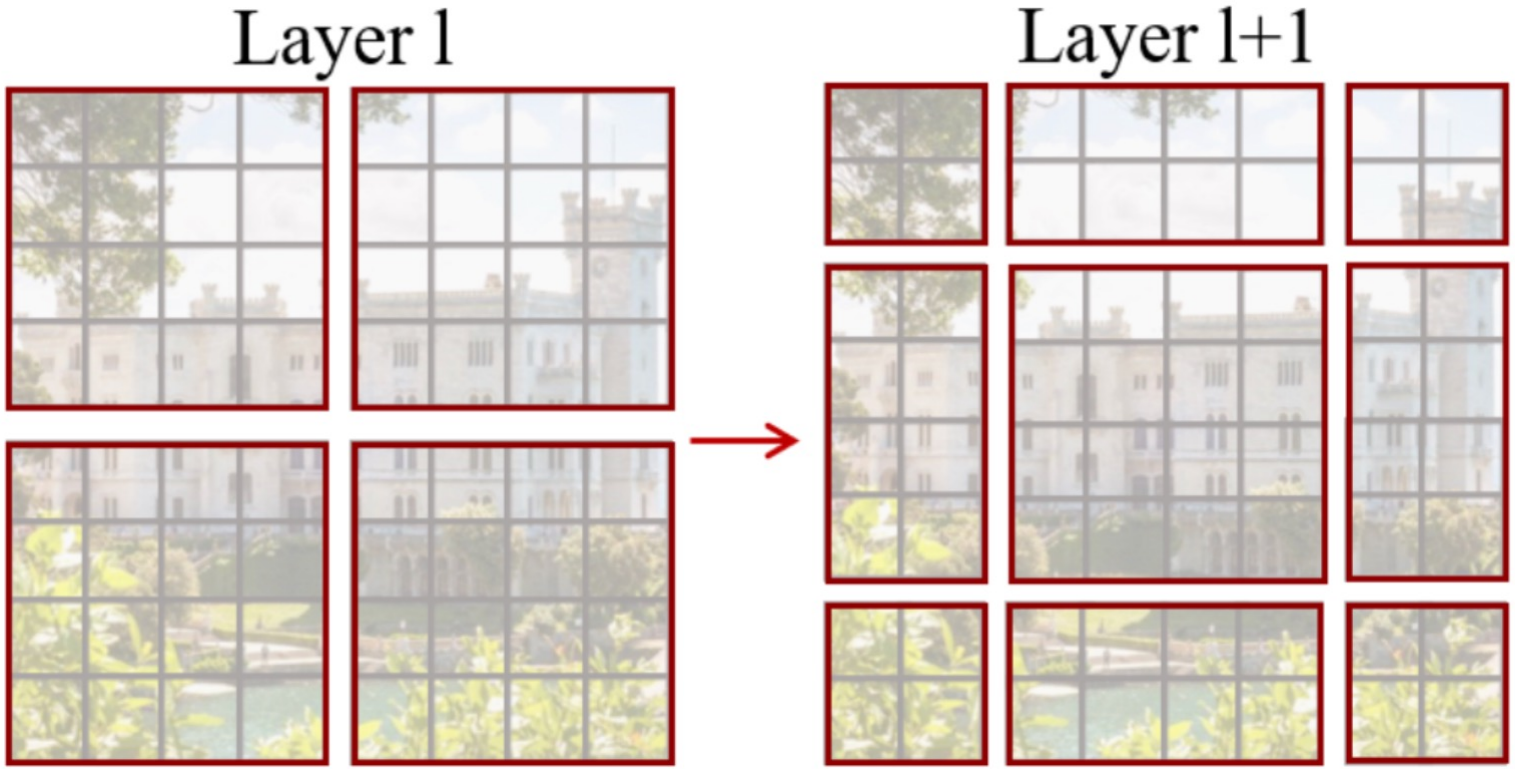
---

- Instead, **divide** the grid **in** $M \times M$ **windows**, and **compute attention separately in each window**
  - All attention parameters are shared between windows
  - In the paper, $M := 7$
- In memory, $O\left(M^4 \times \frac{H}{M} \times \frac{W}{M}\right) = O(M^2 HW)$

- **Problem:** There are **no interactions** between tokens **across windows**

Shifted Window Attention (Swin)

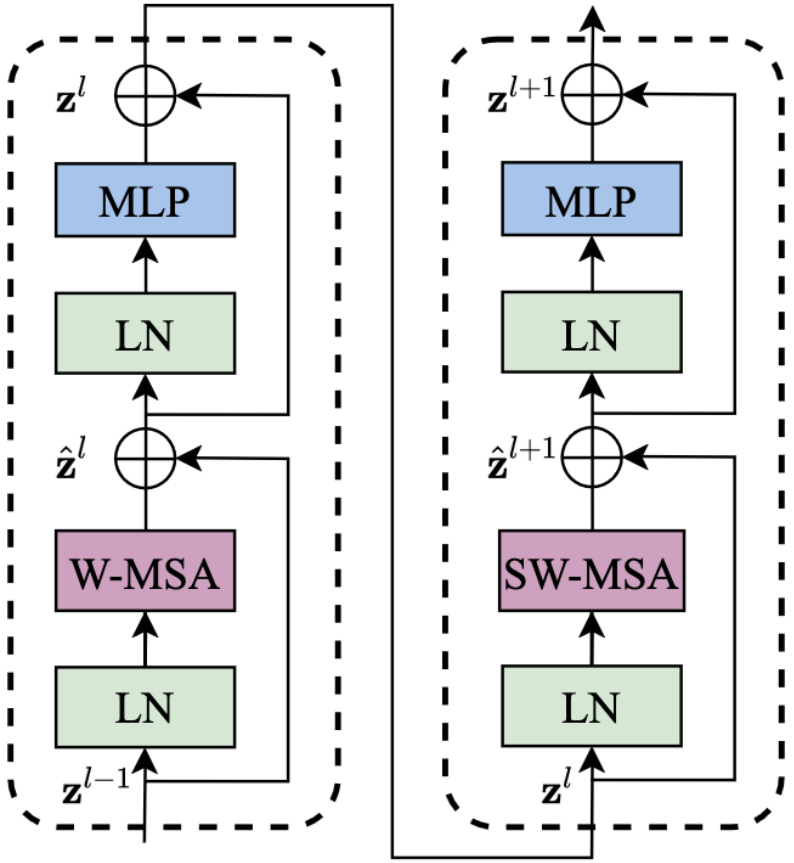
- **Solution:** **alternate** in successive Swin attention blocks **between normal and shifted windows**



[Liu, 2021] **Swin transformer: Hierarchical vision transformer using shifted windows**, Liu, Z. et al., *ICCV*, *2021*

Swin Transformer Blocks

- Nothing changes in the design of a Swin Transformer block compared to a ViT block, except that consecutive blocks have different window patterns



[Liu, 2021] **Swin transformer: Hierarchical vision transformer using shifted windows**, Liu, Z. et al., *ICCV, 2021*

Relative positional bias

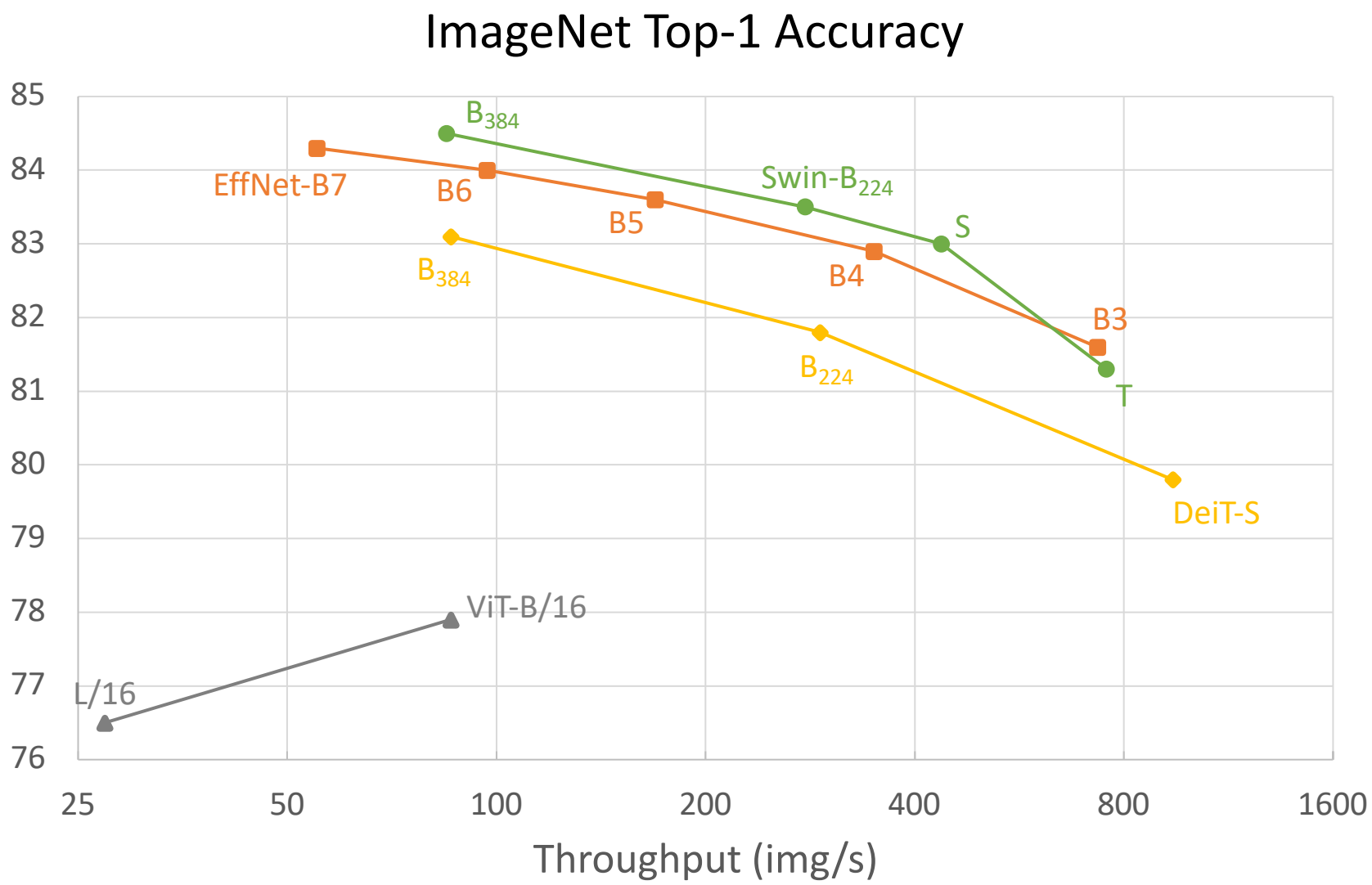
- Swin Transformer does not use positional embeddings, which would encode the absolute position of patches

- Instead, it injects a learnable **relative positional bias** matrix in the attention computation: B of size $M^2 \times M^2$

$$AV = \text{softmax}\left(\frac{\text{QK}^T}{\sqrt{D_h}} + B\right)V$$

- The relative positional bias matrix is different for each head

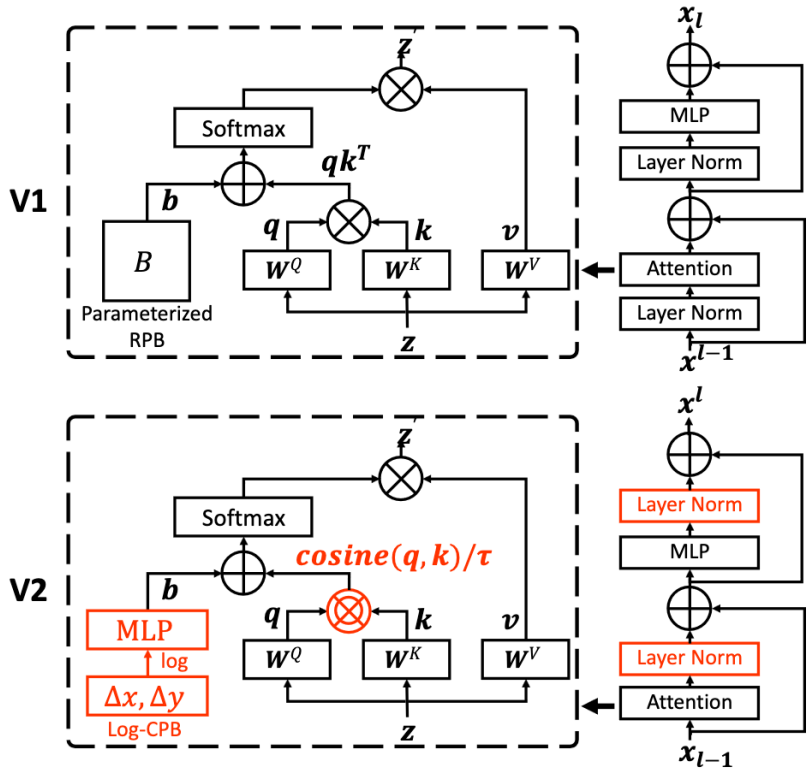
Swin Transformer vs. State-of-the-art

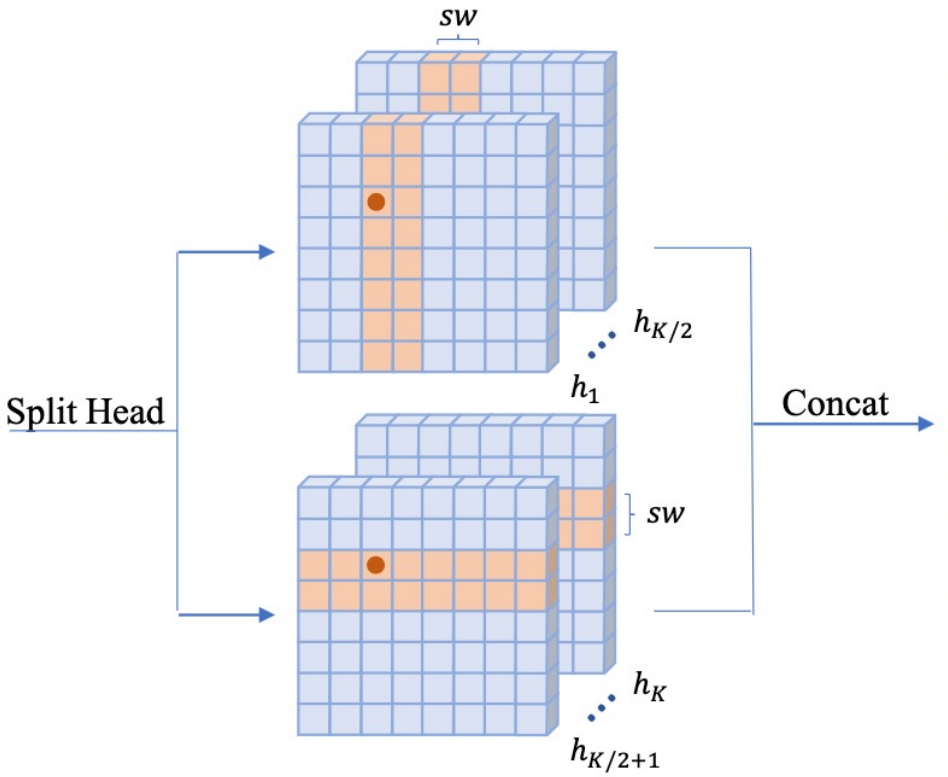


Some extensions of Swin Transformers

SwinV2 introduces changes to **scale up** Swin Transformers to larger models ($3B$ parameters): self-supervised pre-training, post-normalization…

CSwin Transformers [Dong, 2022] propose an alternative hierarchical vision transformer backbone, with improved performance ($+1-2\%$)



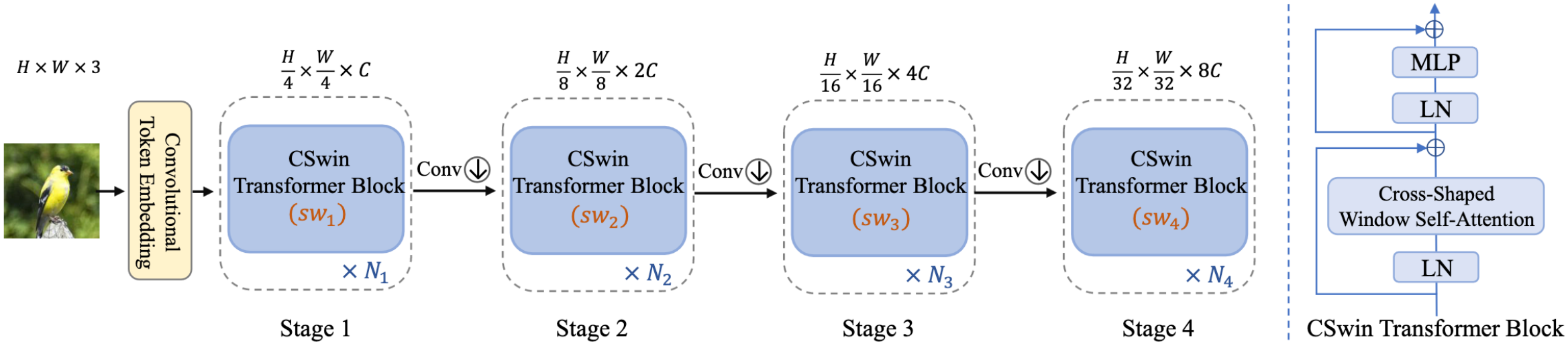


[Liu, 2022] **Swin transformer v2: Scaling up capacity and resolution**, Liu, Z. et al., *CVPR, 2022*

[Dong, 2022] **Cswin transformer: A general vision transformer backbone with cross-shaped windows**, Dong, X. et al., *CVPR, 2022*

CSwin Transformers

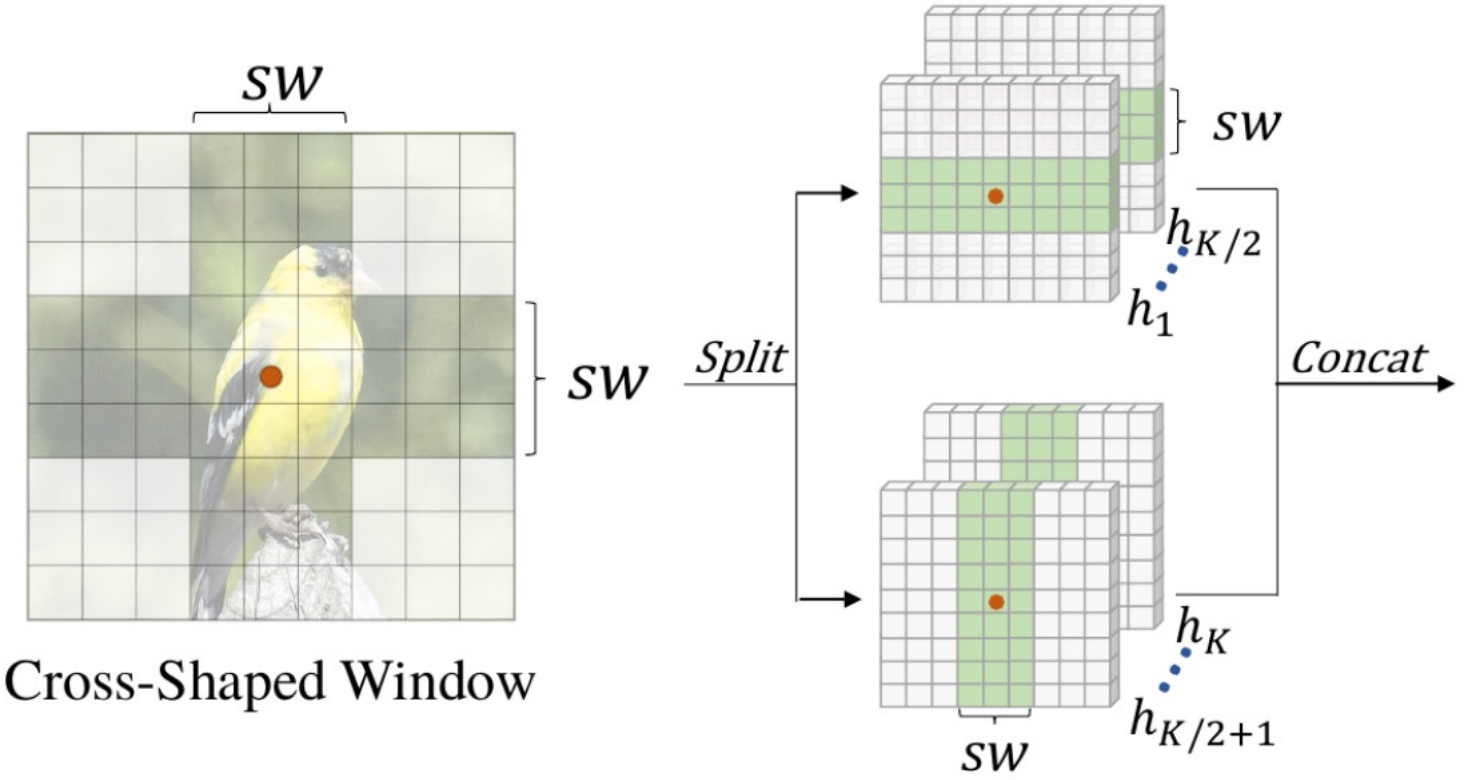
- The tokenization (7×7, stride 4) and patch merging layers (3×3, stride 2) now involve convolutions
- The two types of Swin Transformer blocks are replaced by a single CSwin Transformer block



[Dong, 2022] **Cswin transformer: A general vision transformer backbone with cross-shaped windows**, Dong, X. et al., *CVPR*, *2022*

Cross-Shaped Window Self-Attention

- The CSwin Transformer block has semi-global attention, using **cross patterns**
- In practice, the cross is **divided** into **vertical and horizontal stripes**; half of the heads in multi-head self-attention work with horizontal stripes, the other half with vertical

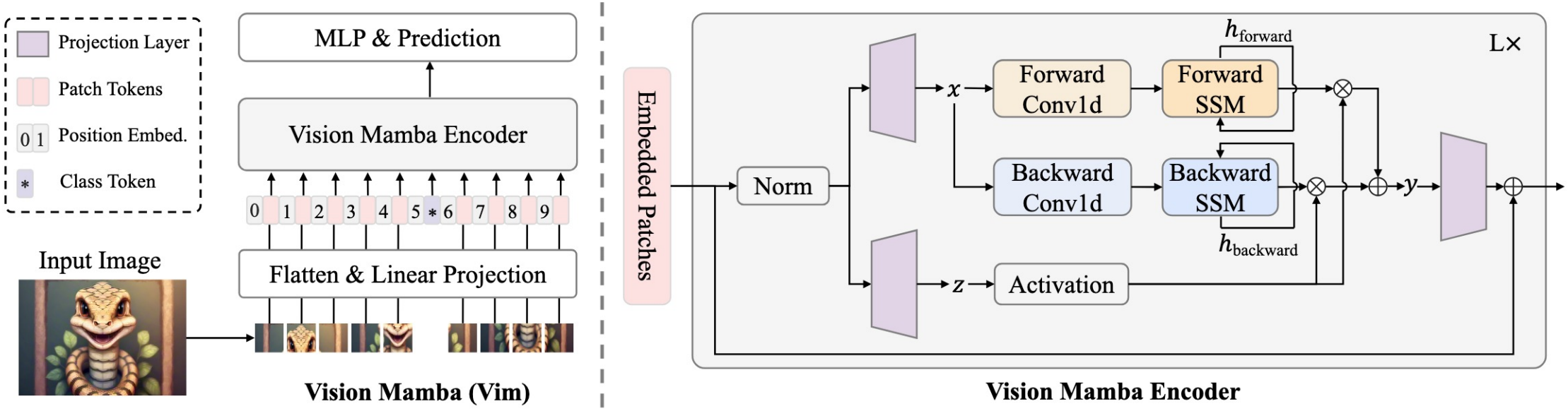


[Dong, 2022] **Cswin transformer: A general vision transformer backbone with cross-shaped windows**, Dong, X. et al., *CVPR*, *2022*

Mamba

- Mamba and Vision Mamba are competitive **state-space model alternatives** to Transformers and ViTs that aim to find a **linear time replacement for attention**

- Also look up for Mamba-2 and Mamba-3, as well as a hybrid attention/SSM model: Hymba



[Dao, 2023] **Mamba: Linear-Time Sequence Modeling with Selective State Spaces**, Dao, T. et al., *arXiv*, *2023*
[Zhu, 2024] **Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model**, Zhu, L. et al., *ICML*, *2024*
[Dong, 2025] **Hymba: A Hybrid-head Architecture for Small Language Models**, Dong, X. et al., *ICLR*, *2025*
[Anonymous, 2026] **Mamba-3: Improved Sequence Modeling using State Space Principles**, Anonymous, *probably accepted at ICLR, 2026*

Pre-Training Transformers

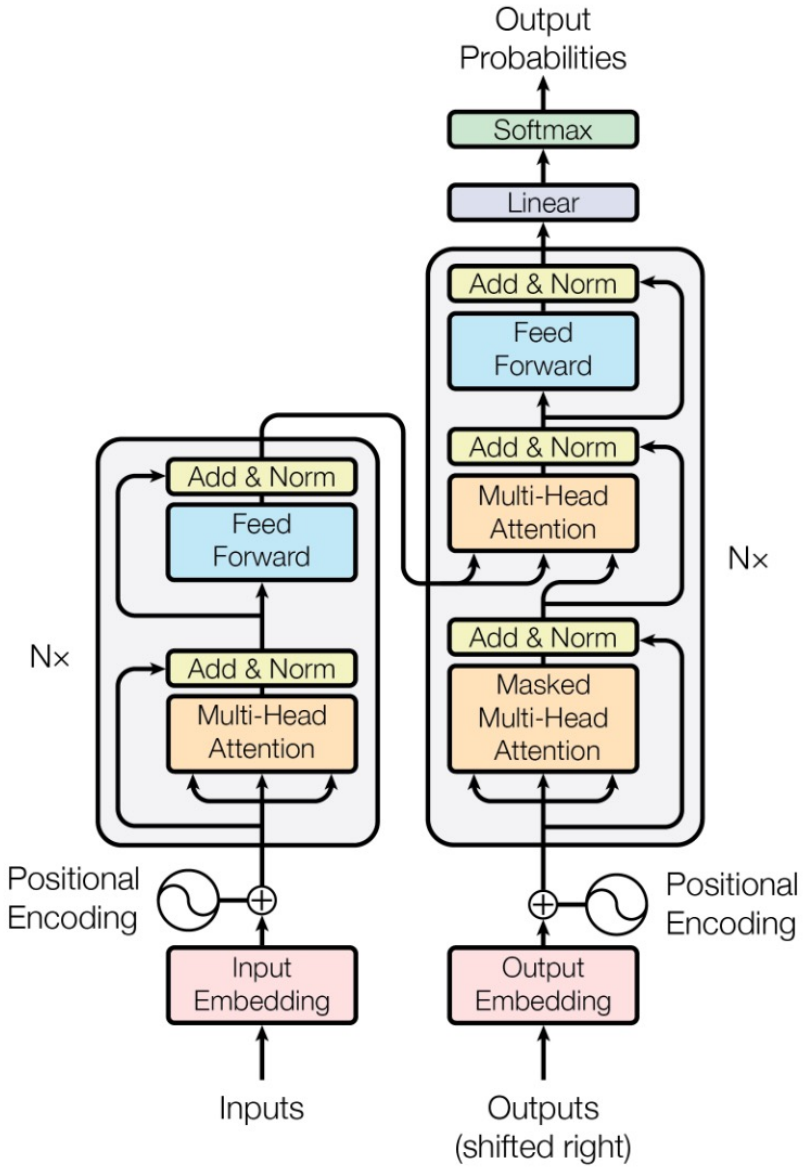
Training NLP transformers: machine translation

- Original transformer [Vaswani, 2017]
- The model is trained to translate sentences from one language to another
  (English → German, English → French)
  - The encoder can be re-used for other purposes

- A **transformer decoder** is added to the model

- Encoder: outputs a semantic embedding of the input English tokens

- Decoder: *outputs the next token*, *given the translated text up to this token*, and given the encoded embeddings of the full English sentence



[Vaswani, 2017] **Attention is all you need**, Vaswani, A. et al., *NeuRIPS*, *2017*

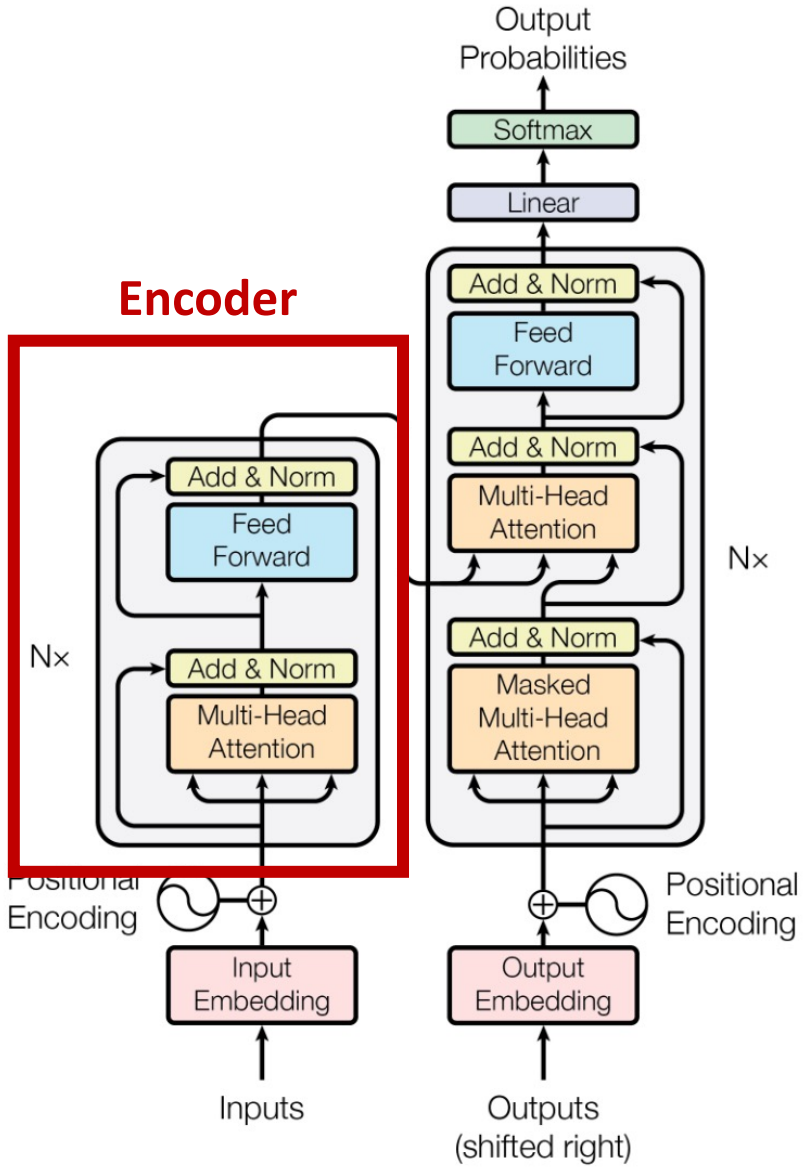
Training NLP transformers: machine translation

- Original transformer [Vaswani, 2017]
- The model is trained to translate sentences from one language to another
  (English → German, English → French)
  - The encoder can be re-used for other purposes

- A **transformer decoder** is added to the model

- Encoder: outputs a semantic embedding of the input English tokens

- Decoder: *outputs the next token*, *given the translated text up to this token*, and given the encoded embeddings of the full English sentence



[Vaswani, 2017] **Attention is all you need**, Vaswani, A. et al., *NeuRIPS*, *2017*

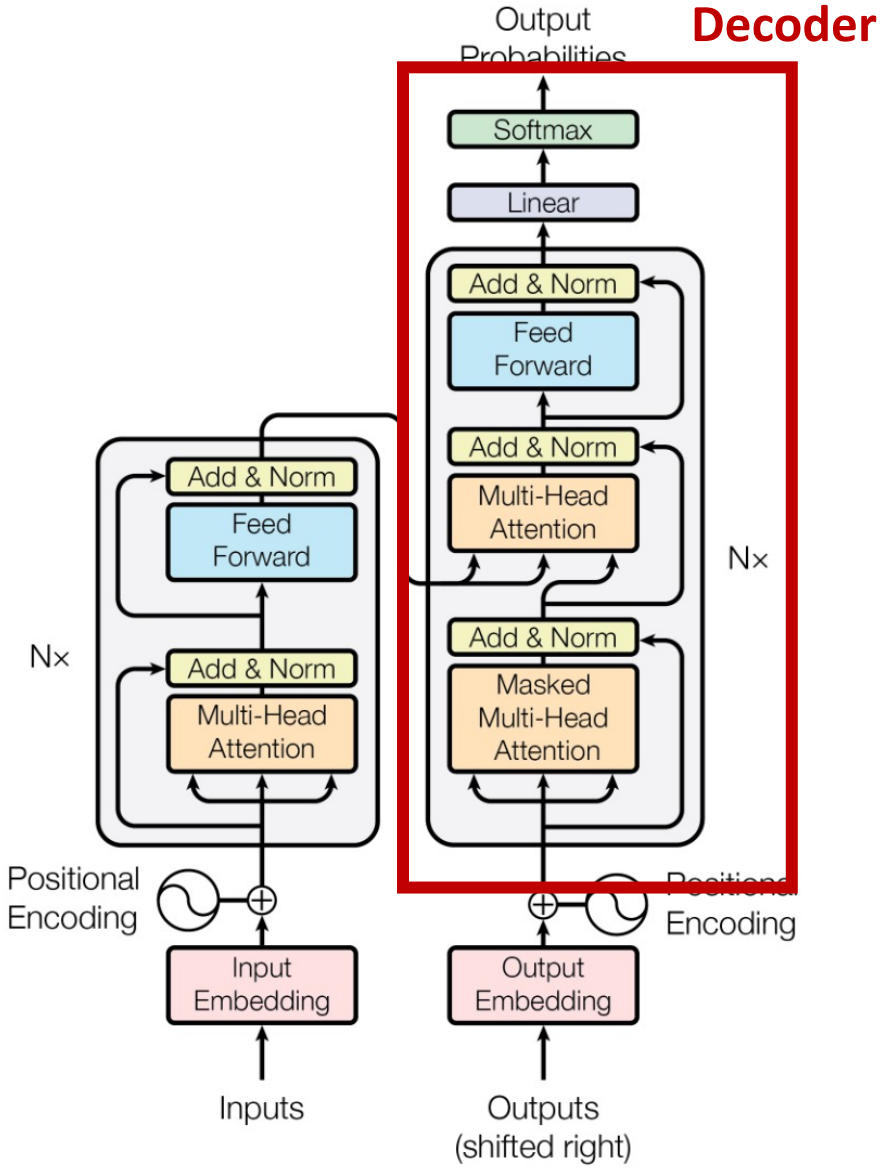
Training NLP transformers: machine translation

- Original transformer [Vaswani, 2017]
- The model is trained to translate sentences from one language to another
  (English → German, English → French)
  - The encoder can be re-used for other purposes

- A **transformer decoder** is added to the model

- Encoder: outputs a semantic embedding of the input English tokens

- Decoder: *outputs the next token*, *given the translated text up to this token*, and given the encoded embeddings of the full English sentence



[Vaswani, 2017] **Attention is all you need**, Vaswani, A. et al., *NeuRIPS*, *2017*

Training NLP transformers: machine translation

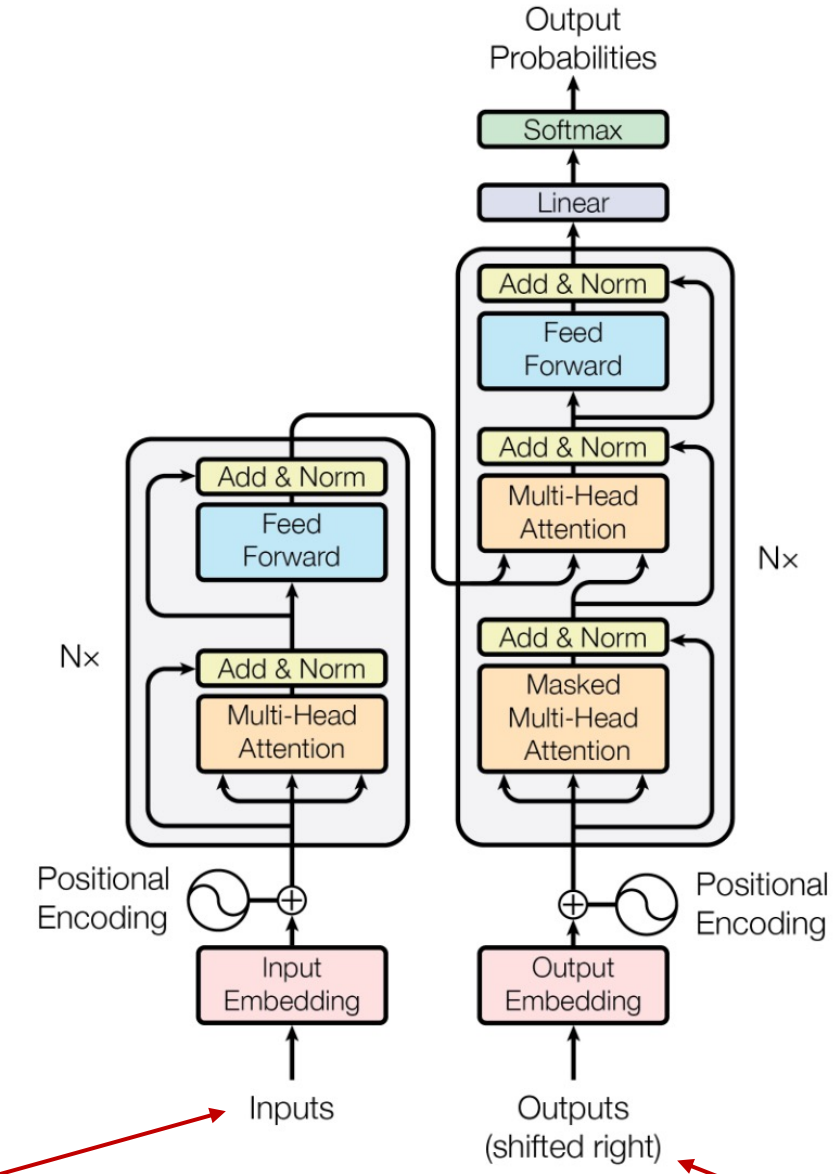
'Je' 'parle' 'anglais' [END]

- Original transformer [Vaswani, 2017]
- The model is trained to translate sentences from one language to another
  (English → German, English → French)
  - The encoder can be re-used for other purposes

- A **transformer decoder** is added to the model

- Encoder: outputs a semantic embedding of the input English tokens

- Decoder: *outputs the next token*, *given the translated text up to this token*, and given the encoded embeddings of the full English sentence



[START] 'I' 'can' 'speak' 'English' [END]

[START] 'Je' 'parle' 'anglais'

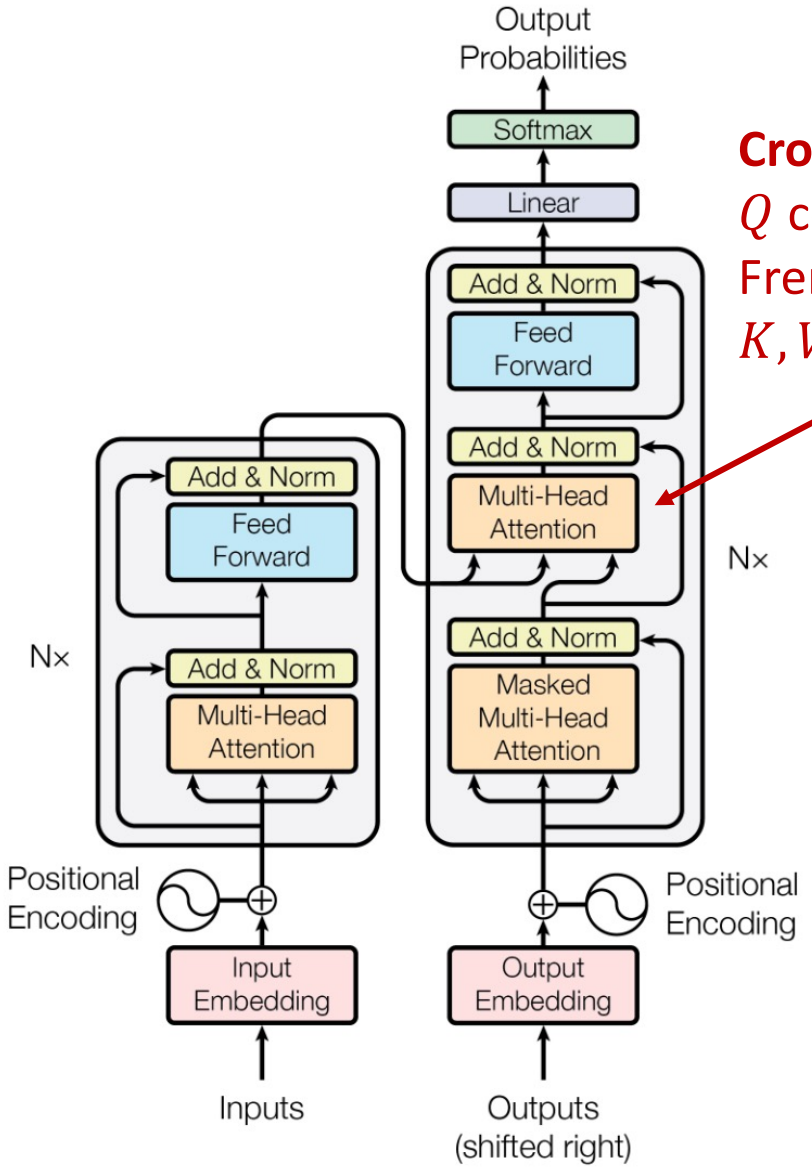
Training NLP transformers: machine translation

- Original transformer [Vaswani, 2017]
- The model is trained to translate sentences from one language to another
  (English → German, English → French)
  - The encoder can be re-used for other purposes

- A **transformer decoder** is added to the model

- Encoder: outputs a semantic embedding of the input English tokens

- Decoder: *outputs the next token*, *given the translated text up to this token*, and given the encoded embeddings of the full English sentence



Cross-attention:
Q comes from French tokens,
K, V from English

[Vaswani, 2017] **Attention is all you need**, Vaswani, A. et al., *NeuRIPS, 2017*

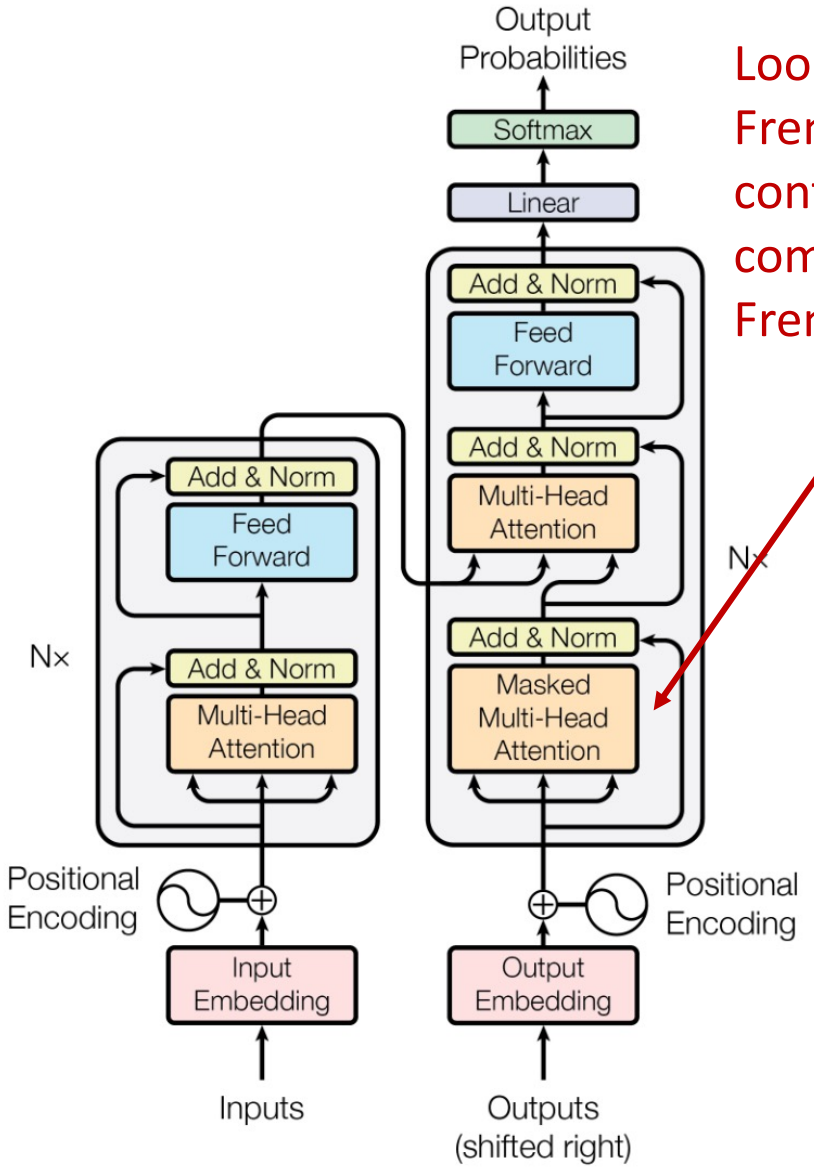
Training NLP transformers: machine translation

- Original transformer [Vaswani, 2017]
- The model is trained to translate sentences from one language to another
  (English → German, English → French)
  - The encoder can be re-used for other purposes

- A **transformer decoder** is added to the model

- Encoder: outputs a semantic embedding of the input English tokens

- Decoder: *outputs the next token*, *given the translated text up to this token*, and given the encoded embeddings of the full English sentence



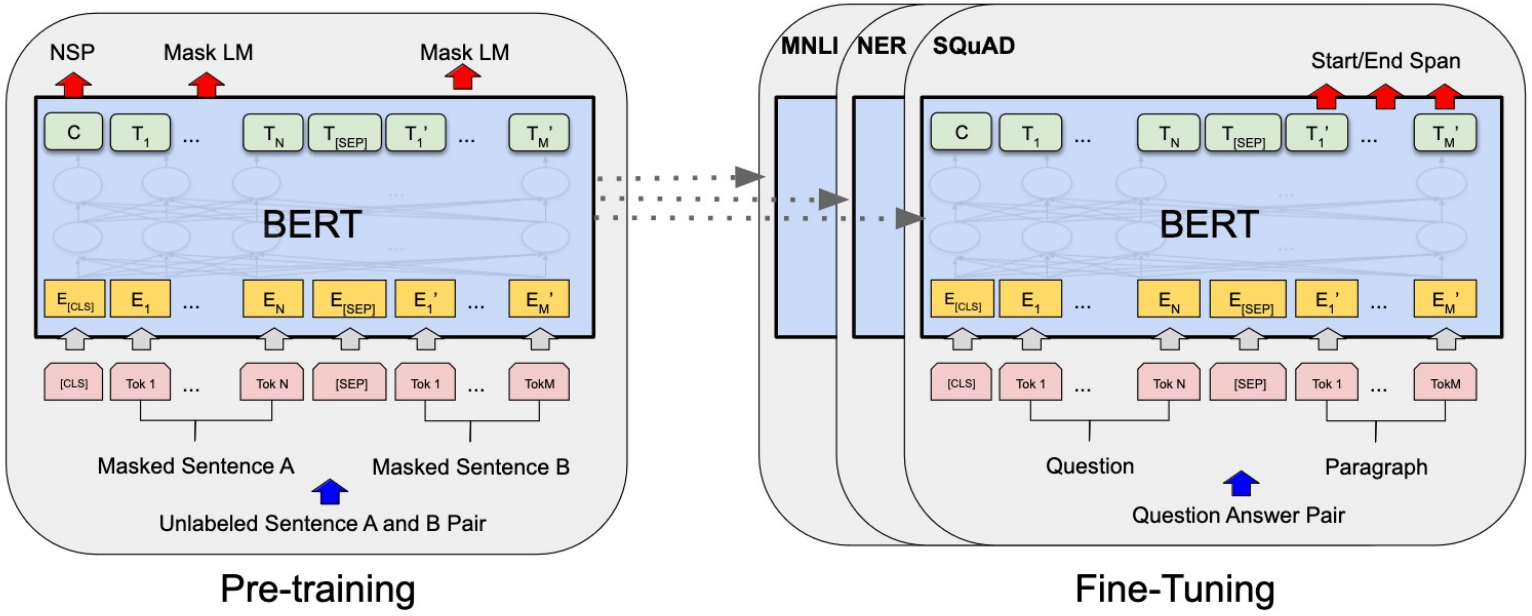
Masked Self-attention:
Look at *previous* French tokens for context (Q, K, V come from French tokens)

[Vaswani, 2017] **Attention is all you need**, Vaswani, A. et al., *NeuRIPS, 2017*

BERT pre-training of NLP transformers

- BERT is **an encoder** only, **pre-trained** by **unsupervised** / **self-supervised** learning, which can be fine-tuned for downstream tasks: Question Answering, Named Entity Recognition, sentiment analysis...

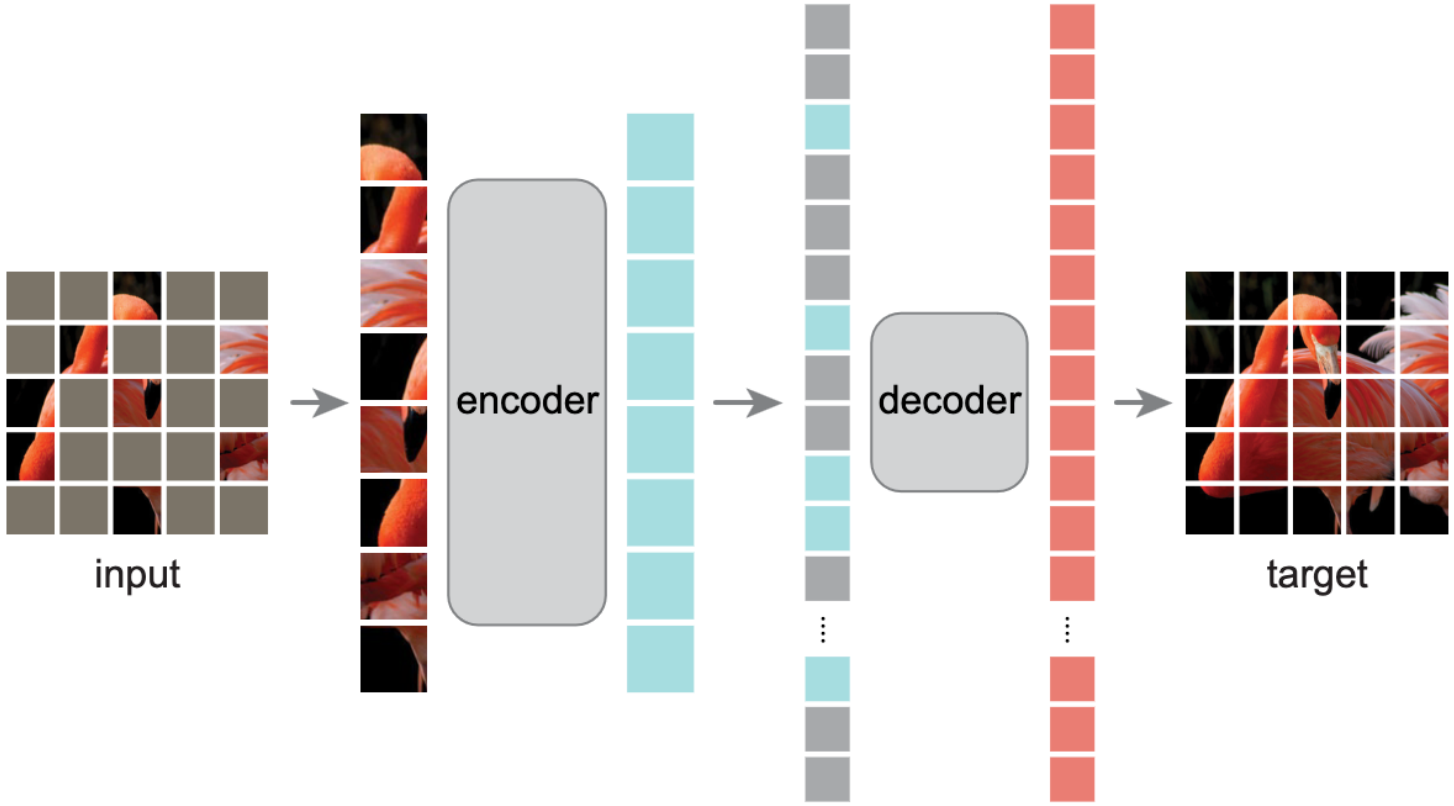
- The pre-training combines:
  - Masked Language Modeling: Mask 15% of input tokens → predict the right tokens
  - Next Sentence Prediction: given a pair of sentences, correctly predict whether they are consecutive sentences in a text or just randomly associated



[Devlin, 2018] **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding**, Devlin, J. et al., *arXiv*, *2018*

Pre-training Vision Transformers: Masked Auto-Encoding

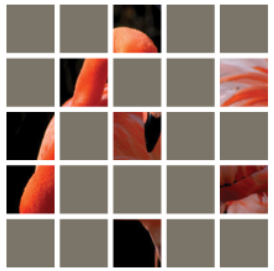
- Inspired by BERT, [He, 2022] proposes to pretrain ViTs using **Masked Image Modeling** (MIM)
- Pretext task: from partially masked image, reconstruct full image



[He, 2022] **Masked autoencoders are scalable vision learners**, He, K. et al., *CVPR*, *2022*

Pre-training Vision Transformers: Masked Auto-Encoding

- Inspired by BERT, [He, 2022] proposes to pretrain ViTs using **Masked Image Modeling** (MIM)
- Pretext task: from partially masked image, reconstruct full image



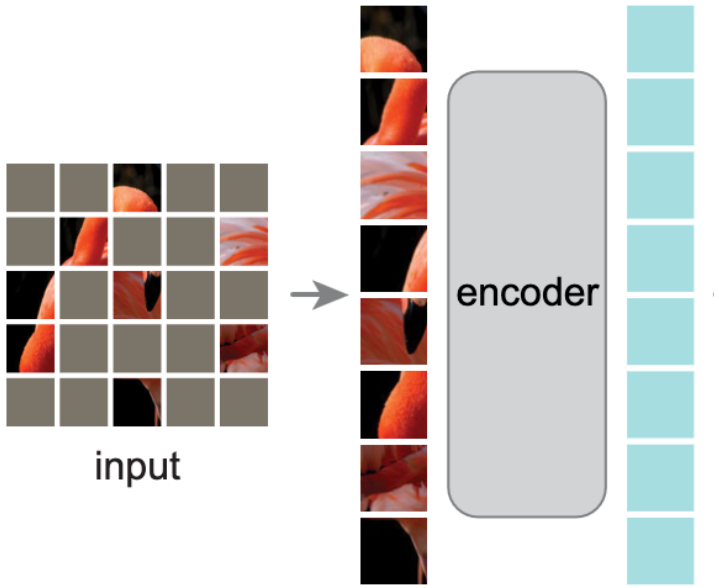
input

- 75% of the input tokens are masked
  - Masked data is replaced by a learnable [MASK] token

[He, 2022] **Masked autoencoders are scalable vision learners**, He, K. et al., *CVPR*, *2022*

Pre-training Vision Transformers: Masked Auto-Encoding

- Inspired by BERT, [He, 2022] proposes to pretrain ViTs using **Masked Image Modeling** (MIM)
- Pretext task: from partially masked image, reconstruct full image



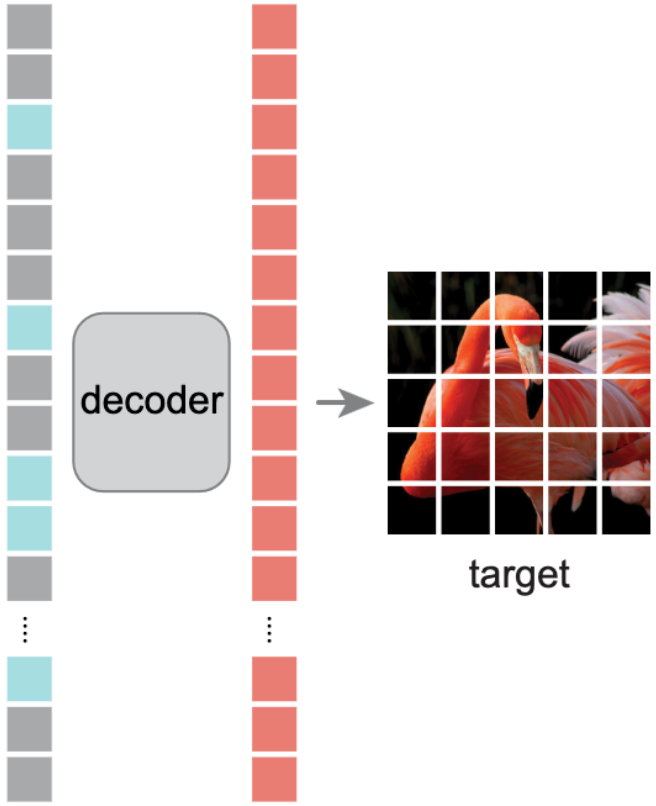
- 75% of the input tokens are masked
  - Masked data is replaced by a learnable [MASK] token
- The visible tokens are processed by the ViT encoder
  - [MASK] tokens are discarded for now
  - but we are still position-aware thanks to positional embeddings

[He, 2022] **Masked autoencoders are scalable vision learners**, He, K. et al., *CVPR*, *2022*

Pre-training Vision Transformers: Masked Auto-Encoding

- Inspired by BERT, [He, 2022] proposes to pretrain ViTs using **Masked Image Modeling** (MIM)
- Pretext task: from partially masked image, reconstruct full image

- A decoder is added for pre-training
  - It can be discarded afterwards
  - Its architecture is similar to the encoder, but with fewer layers ($L := 8$), smaller width ($D := 512$)
- The decoder sees all tokens, including the [MASK] tokens

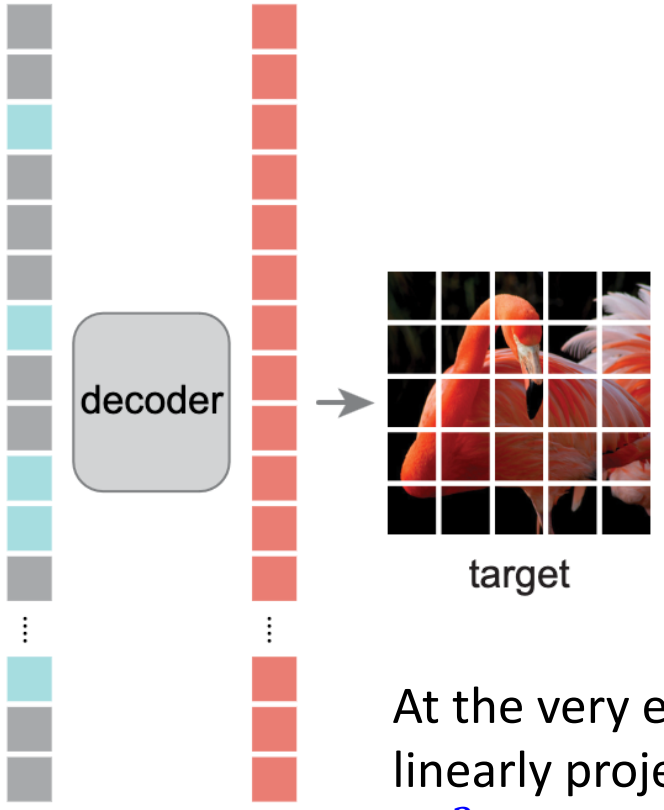


[He, 2022] **Masked autoencoders are scalable vision learners**, He, K. et al., *CVPR*, *2022*

Pre-training Vision Transformers: Masked Auto-Encoding

- Inspired by BERT, [He, 2022] proposes to pretrain ViTs using **Masked Image Modeling** (MIM)
- Pretext task: from partially masked image, reconstruct full image

- A decoder is added for pre-training
  - It can be discarded afterwards
  - Its architecture is similar to the encoder, but with fewer layers ($L := 8$), smaller width ($D := 512$)
- The decoder sees all tokens, including the [MASK] tokens

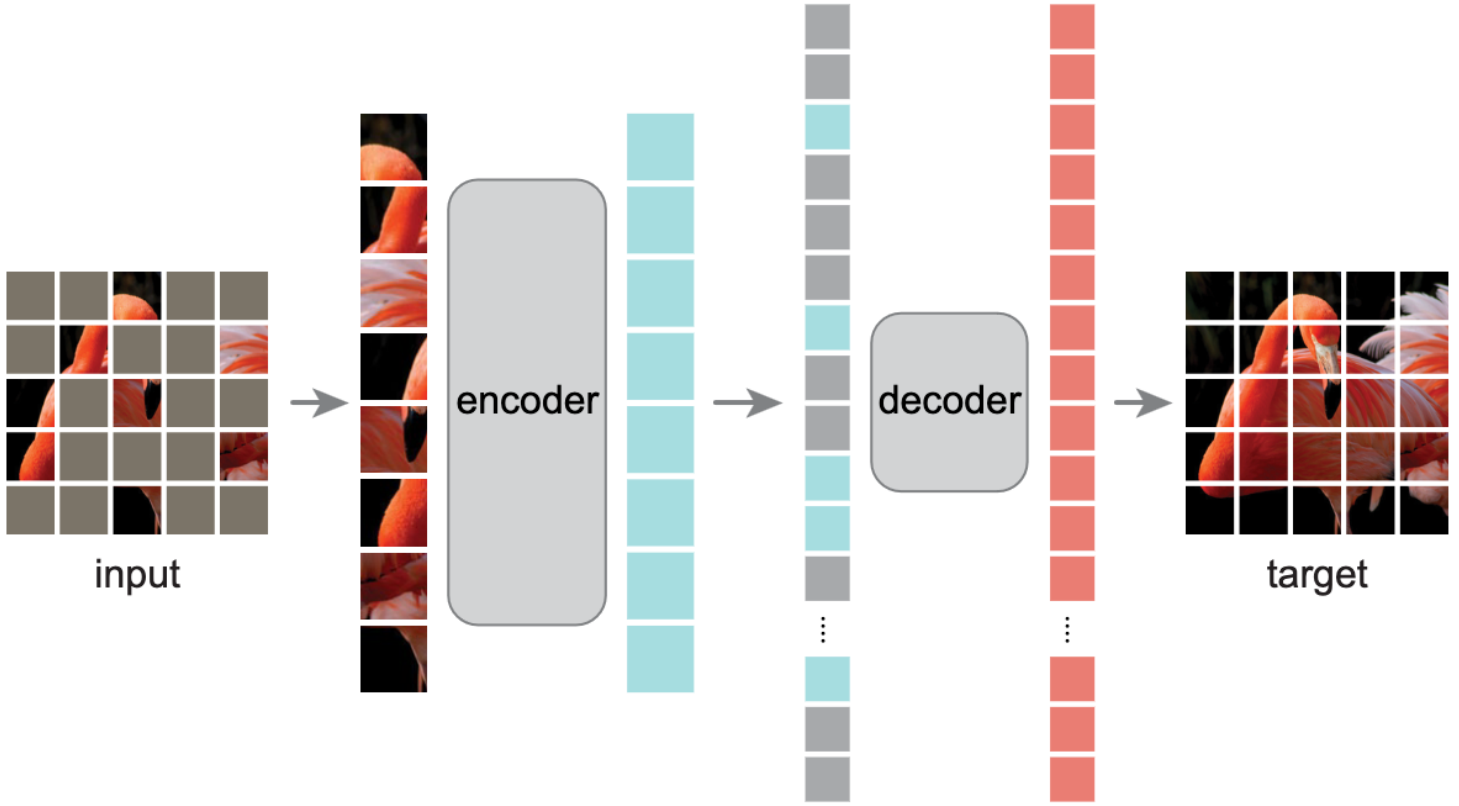


At the very end, each token is linearly projected from D to $P^2C = 16^2 \times 3$ to predict pixel values

[He, 2022] **Masked autoencoders are scalable vision learners**, He, K. et al., *CVPR*, *2022*

Pre-training Vision Transformers: Masked Auto-Encoding

- Inspired by BERT, [He, 2022] proposes to pretrain ViTs using **Masked Image Modeling** (MIM)
- Pretext task: from partially masked image, reconstruct full image

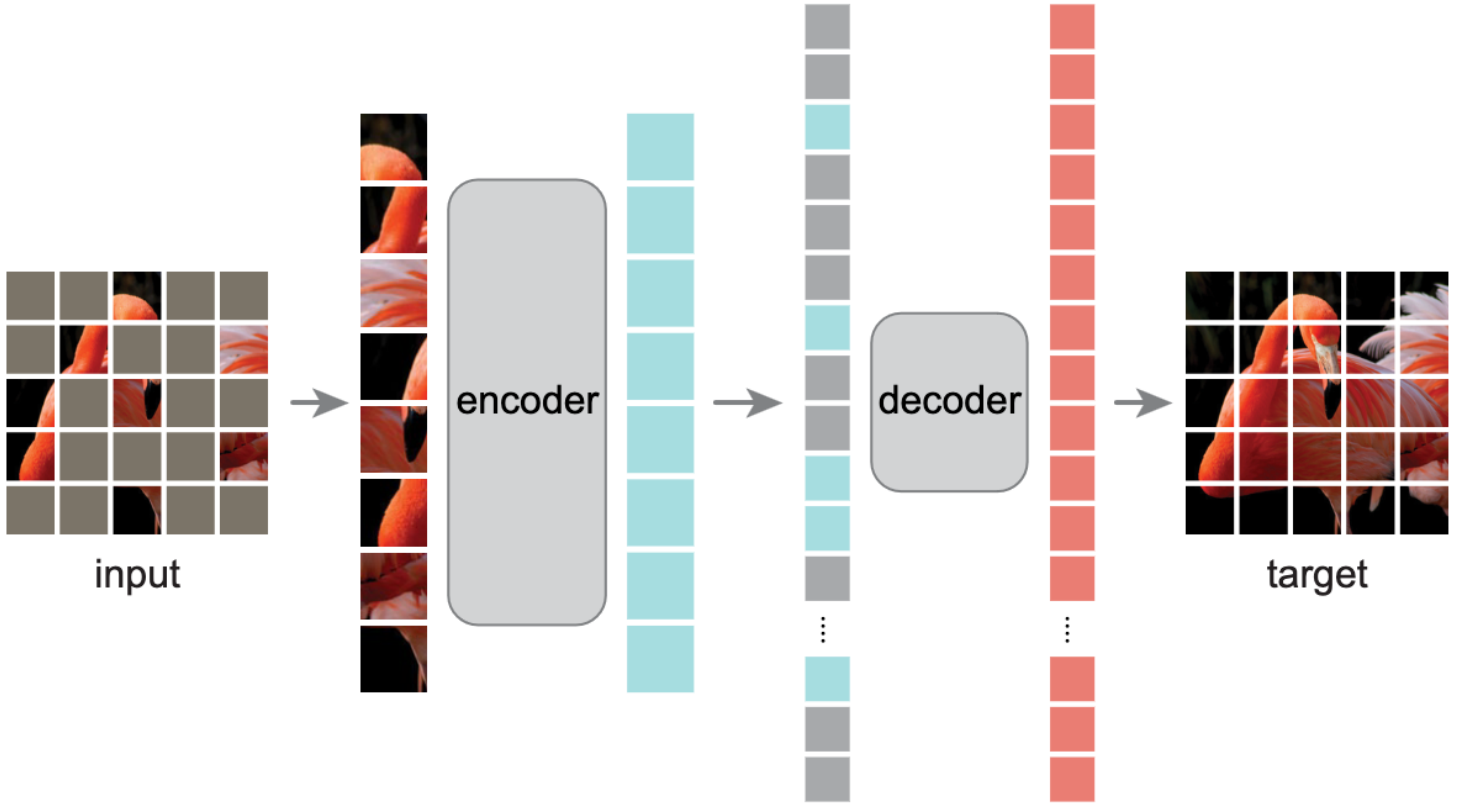


Train **end-to-end** with **MSE loss on masked pixels**

[He, 2022] **Masked autoencoders are scalable vision learners**, He, K. et al., *CVPR*, *2022*

Pre-training Vision Transformers: Masked Auto-Encoding

- Inspired by BERT, [He, 2022] proposes to pretrain ViTs using **Masked Image Modeling** (MIM)
- Pretext task: from partially masked image, reconstruct full image

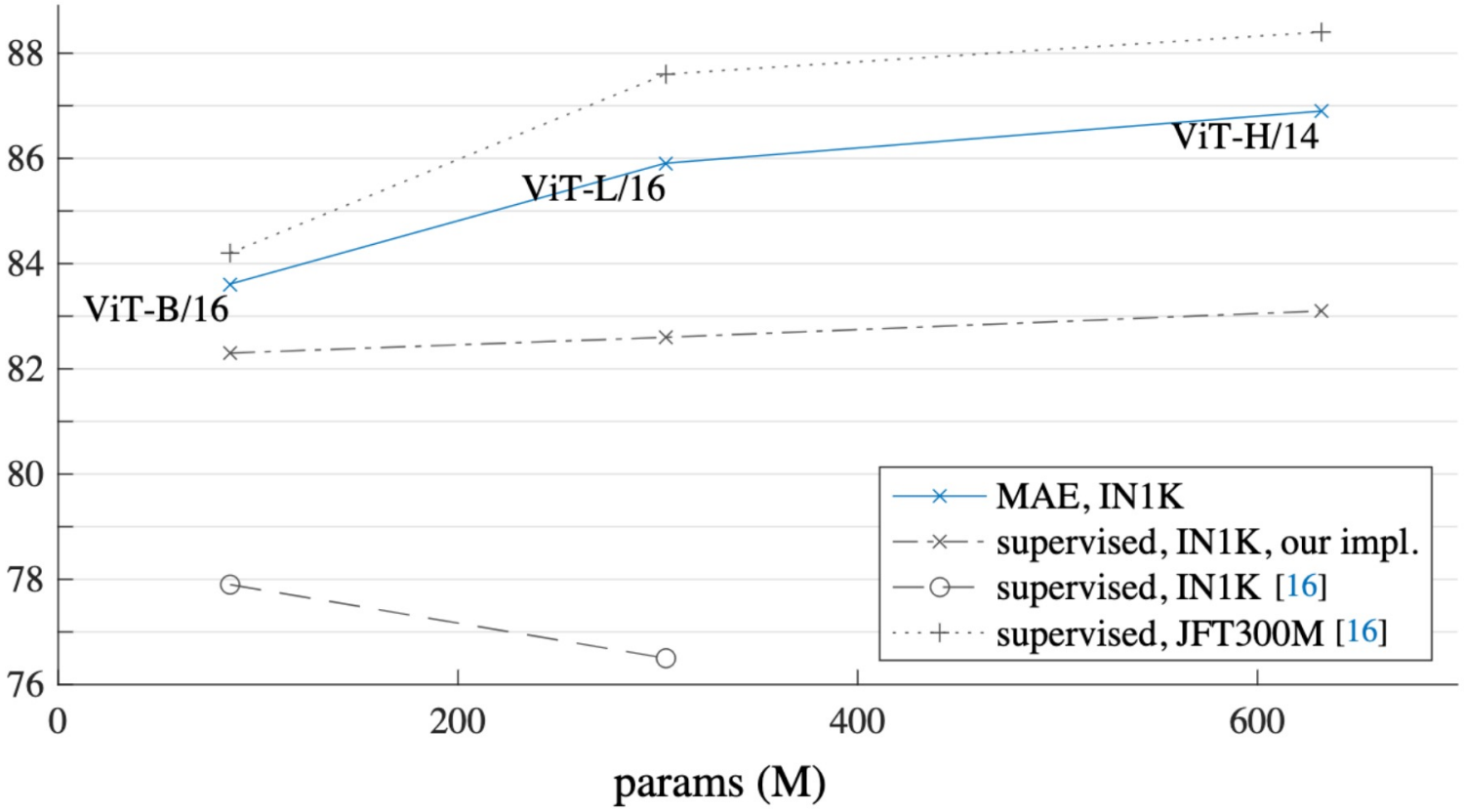


Once pre-trained, discard the decoder and **fine-tune the encoder** for the target task

[He, 2022] **Masked autoencoders are scalable vision learners**, He, K. et al., *CVPR*, *2022*

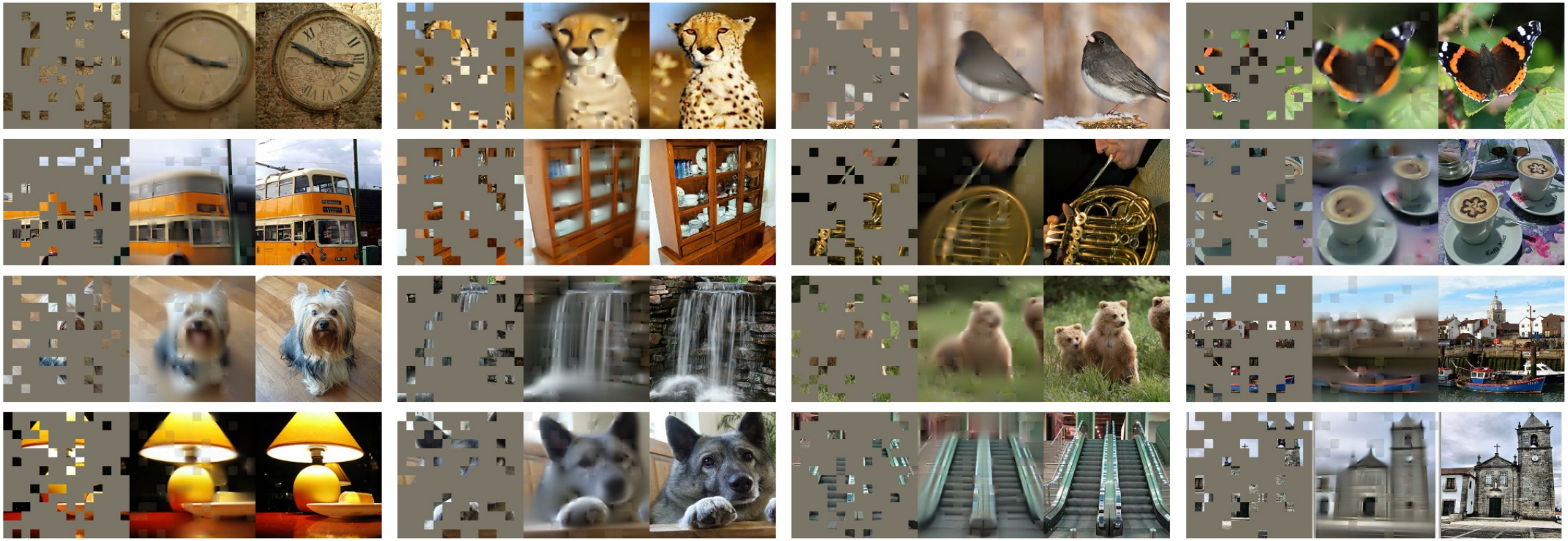
MAE pre-training vs. Training from scratch

- Between $+1.5\%$ (ViT-B/16) and $+4\%$ (ViT-H/14) over training from scratch with good implementation



[He, 2022] **Masked autoencoders are scalable vision learners**, He, K. et al., *CVPR*, *2022*

MAE: example reconstructions from the decoder

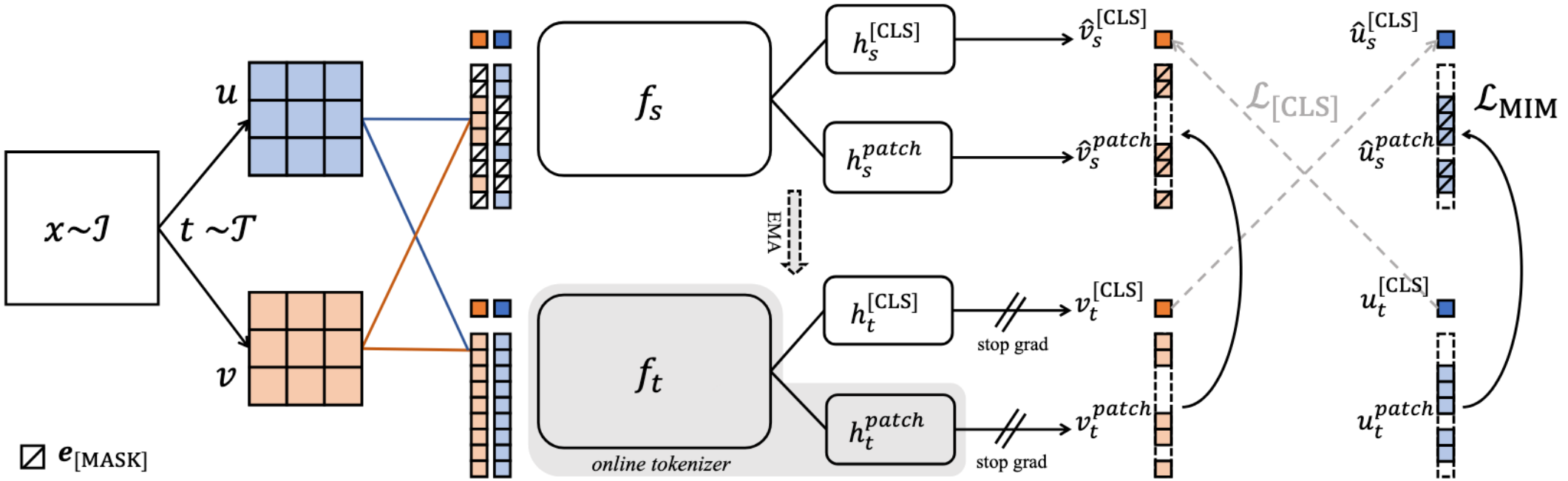


[He, 2022] **Masked autoencoders are scalable vision learners**, He, K. et al., *CVPR*, *2022*

iBOT: Global and Local Representation Learning

- Like MAE, iBOT enables both global (image-level) and local (patch-level) representation learning for global (recognition…) and dense (segmentation…) tasks

- **Local loss**: iBOT moves the **masked patch prediction task** from pixel-space (as in MAE) **to representation space** with a cross-entropy based loss inspired by DINO



[Zhou, 2022] **iBOT: Image BERT pre-training with online tokenizer**, Zhou, J. et al., *ICLR*, *2022*

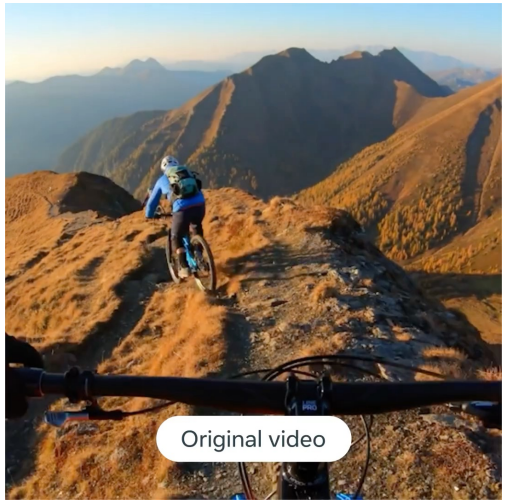
DINOv3: self-supervised training of generalist foundation models

- **DINOv3 massively scales** the DINO+iBOT recipe into a **foundation model**, training on a dataset curated from an initial 17B image data pool

- DINOv3 [Simeoni, 2025] addresses challenges in training dense (patch-level) representations arising from the competing local and global losses via an ad-hoc mechanism called **Gram anchoring**

DINOv3 (here!) provides both image-level and patch-level features usable across many tasks: segmentation, image classification, depth estimation, video tracking, instance retrieval…

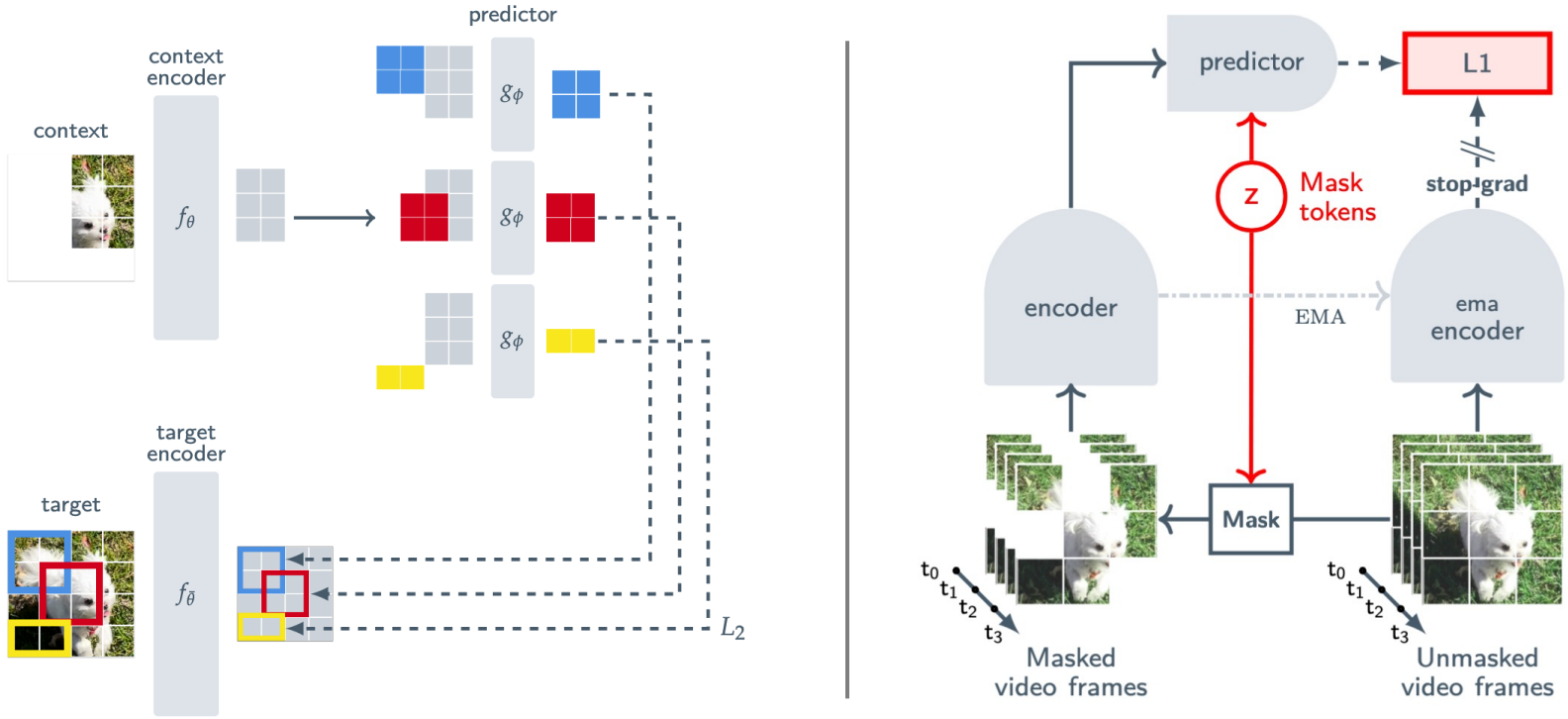




[Siméoni, 2025] **DINOv3**, Siméoni, O. et al., *arXiv*, *2025*

JEPA: Joint-Embedding Predictive Architectures

- I-JEPA (for images) and **V-JEPA 2** (for videos) are closely related self-supervised recipes within the **masked token prediction** paradigm
  - But like MAE, the encoder only processes visible tokens; a decoder/predictor is added to output masked tokens' representations and an L1 loss is minimized in representation space



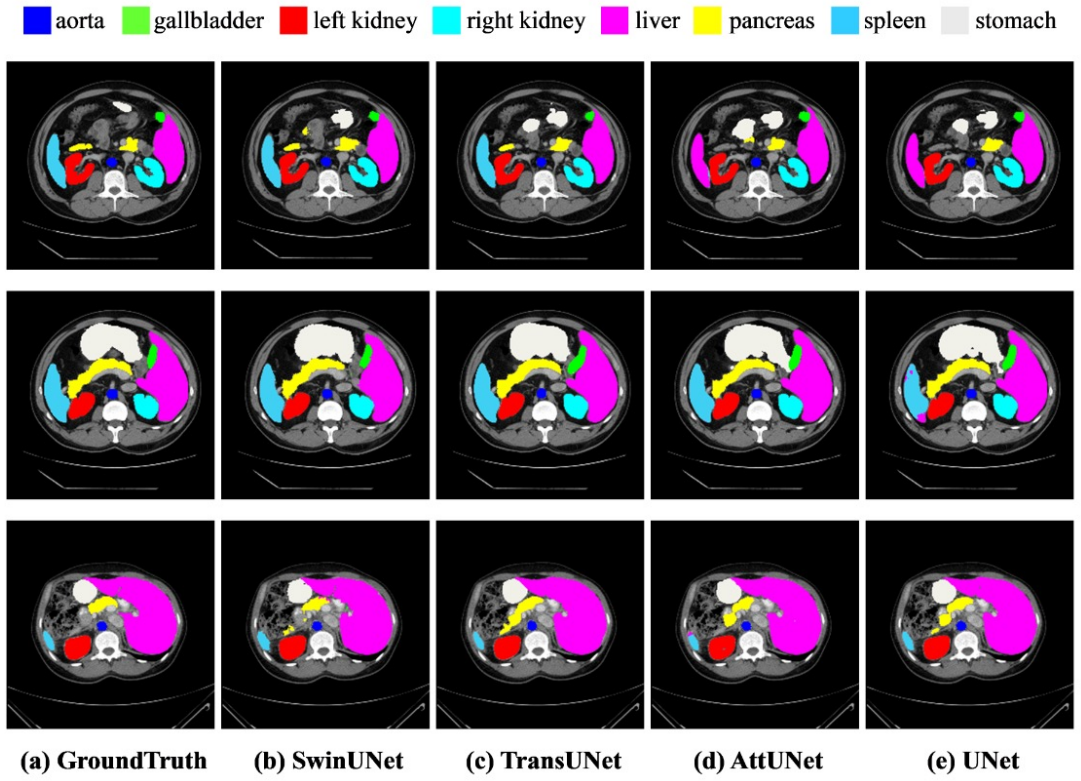
[Assran, 2023] **Self-supervised learning from images with a joint-embedding predictive architecture**, Assran, M. et al., *CVPR, 2023*
[Assran, 2025] **V-JEPA 2: Self-supervised video models enable understanding, prediction and planning**, Assran, M. et al., *arXiv, 2025*

Applications

Swin-unet for image segmentation

- Many segmentation architectures based on vision transformers have been proposed: TransUNet, DPT, SegFormer, UNETR…

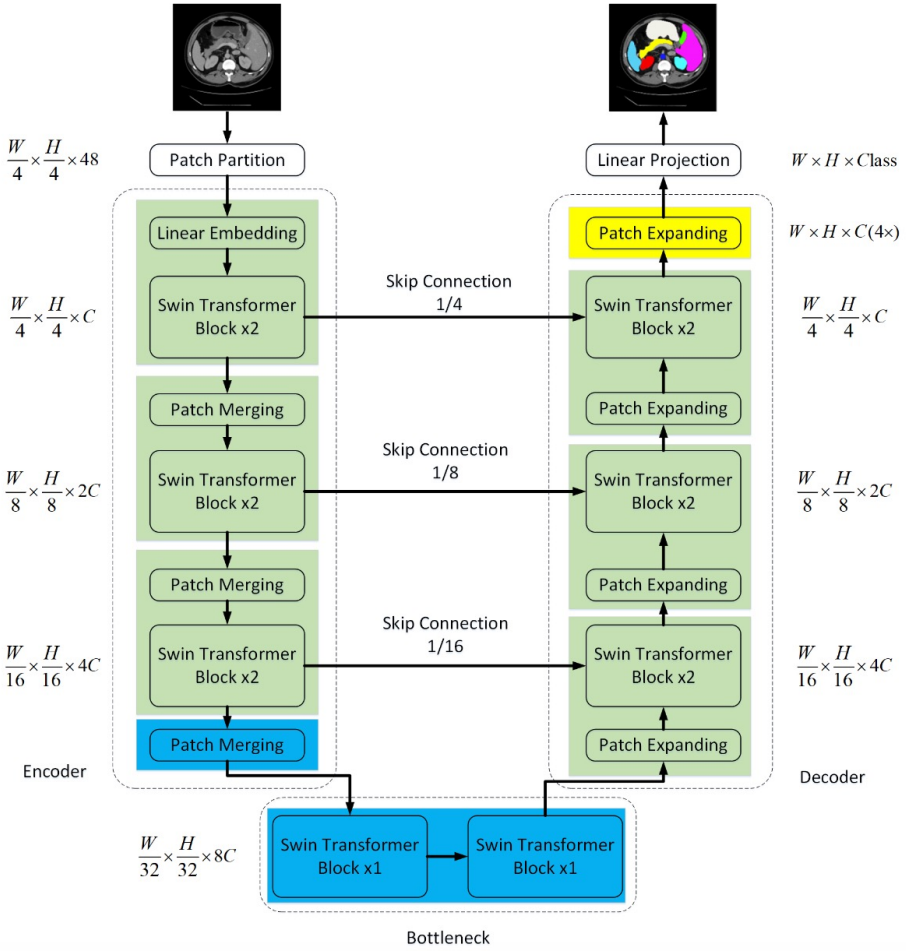
- **Swin-unet** [Cao, 2021] is a pure transformer architecture based on the acclaimed U-Net and Swin Transformer designs



[Cao, 2022] **Swin-unet: Unet-like pure transformer for medical image segmentation**, Cao, H. et al., *ECCV*, *2022*

Swin-unet for image segmentation

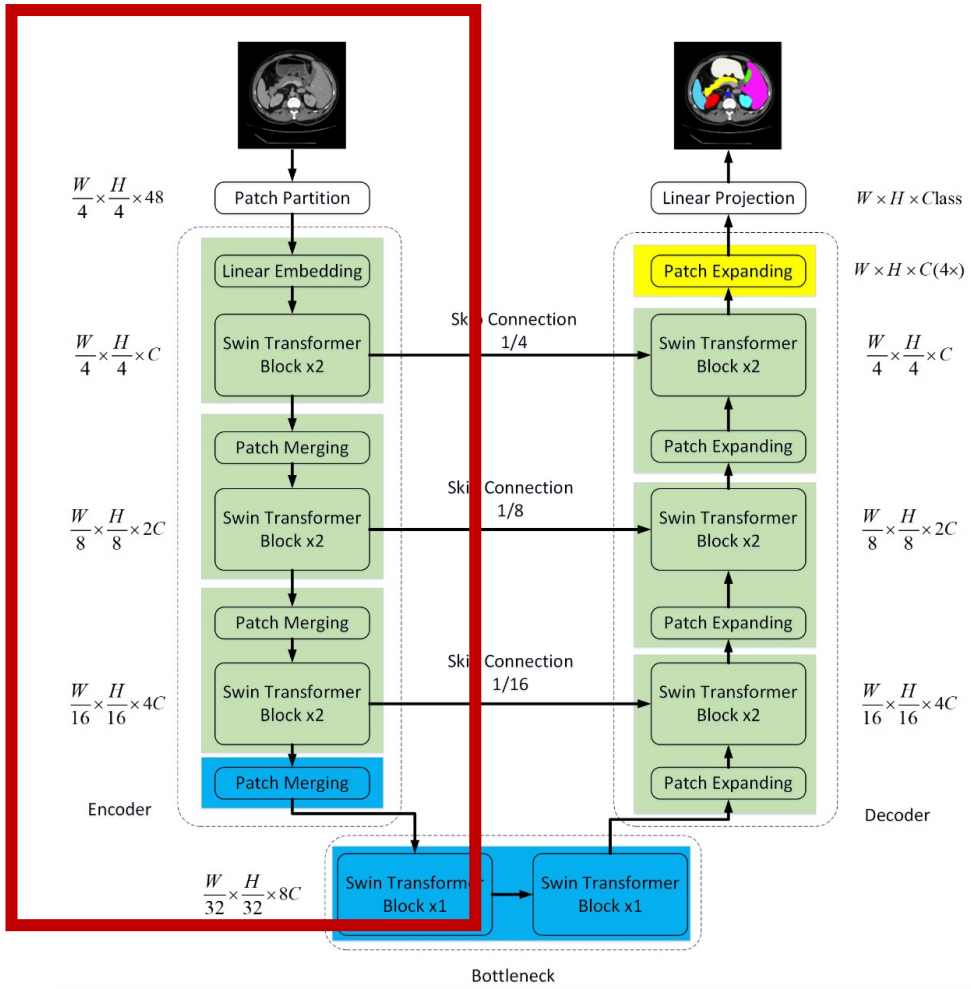
- Swin-unet uses an **encoder-decoder architecture** with skip connections to guide the segmentation both with low-level details and high-level context



[Cao, 2022] **Swin-unet: Unet-like pure transformer for medical image segmentation**, Cao, H. et al., *ECCV*, *2022*

Swin-unet for image segmentation

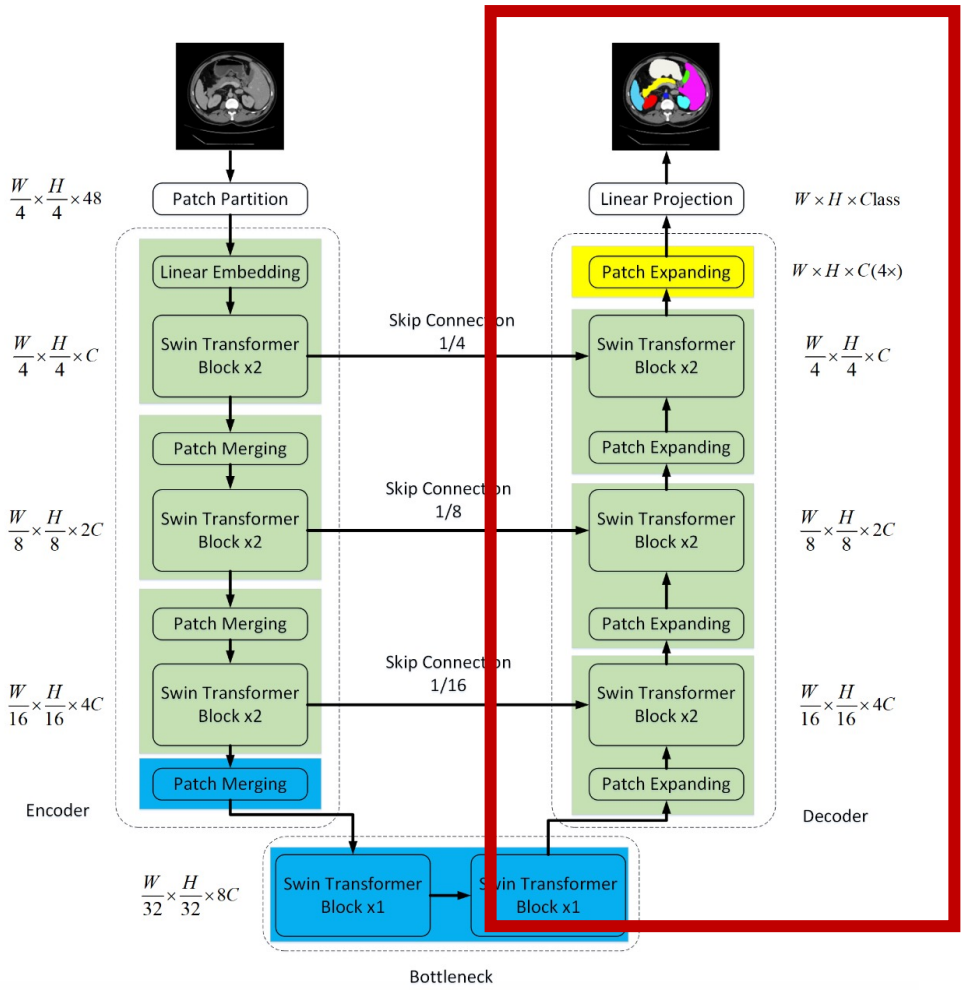
- The encoder replicates Swin Transformer's design



[Cao, 2022] **Swin-unet: Unet-like pure transformer for medical image segmentation**, Cao, H. et al., *ECCV*, *2022*

Swin-unet for image segmentation

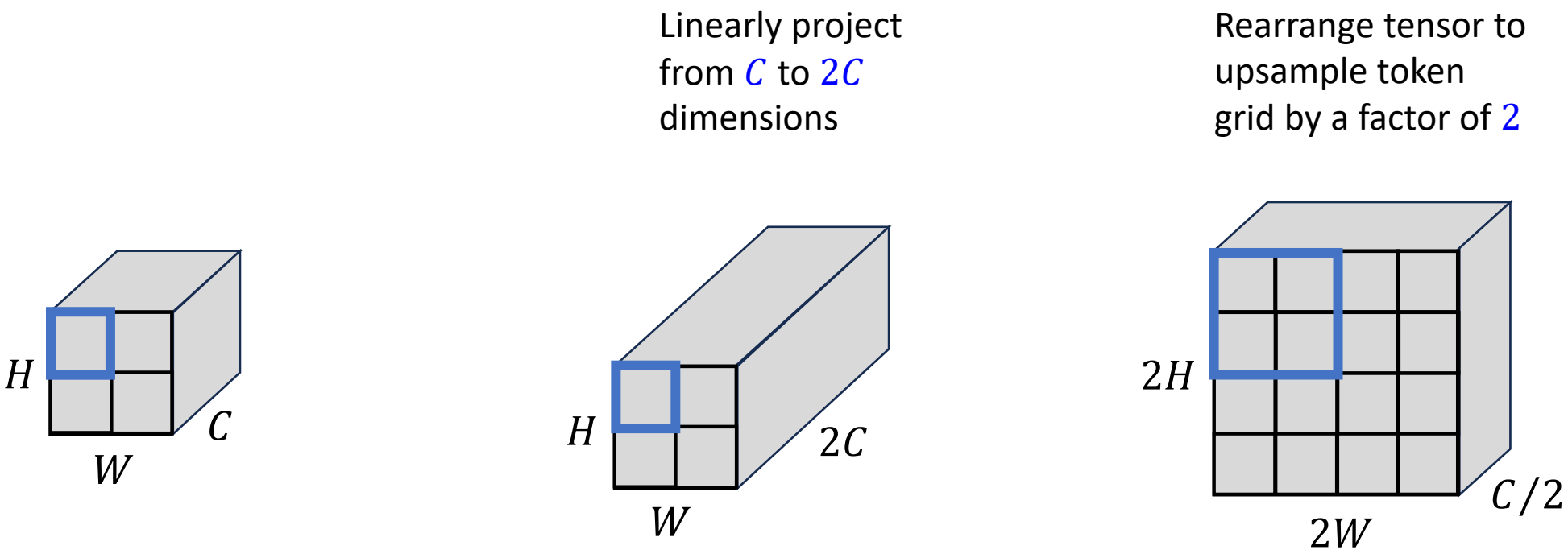
- The **decoder** mirrors Swin Transformer's design, introducing **patch expanding layers** to upsample the image grid



[Cao, 2022] **Swin-unet: Unet-like pure transformer for medical image segmentation**, Cao, H. et al., *ECCV*, *2022*

Swin-unet for image segmentation

- The **decoder** mirrors Swin Transformer's design, introducing **patch expanding layers** to upsample the image grid
  - The mechanism (below) is akin to PixelShuffle [Shi, 2016] in the super-resolution literature



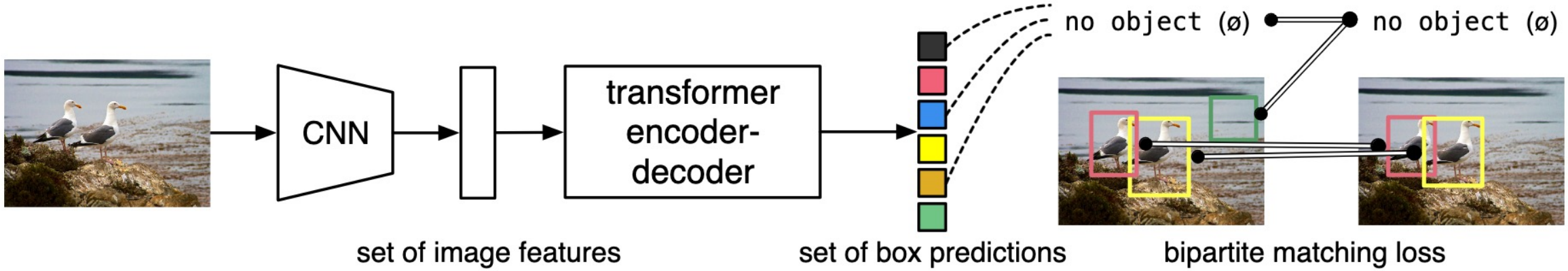
[Shi, 2016] **Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network**, Shi, W. et al., *CVPR*, *2016*

DETR for object detection

- The Detection Transformer is a mixed CNN / vision transformer model to perform object detection i.e., for finding **object bounding boxes and their classes** in images

- DETR predicts a set of N unique bounding boxes and their classes
  - If there are fewer than N objects, it assigns the empty set $\emptyset$ to some (no object)

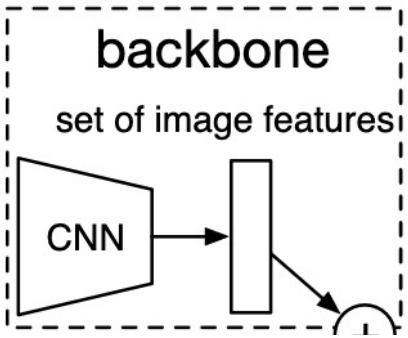
- It uses a **bipartite matching** formulation to assign predicted BBs to ground truth BBs at training time, then it can directly backpropagate using the sum of BB regression and classification losses



[Carion, 2020] **End-to-end object detection with transformers**, Carion, N. et al., *ECCV, 2020*

DETR for object detection

- The $H_0 \times W_0 \times 3$ input image is passed through a CNN backbone (e.g., ResNet-50) to extract features on a lower resolution $H \times W \times C$ output grid ($H, W := {}^{H_0}/_{32}, {}^{W_0}/_{32}; \; C := 2048$)

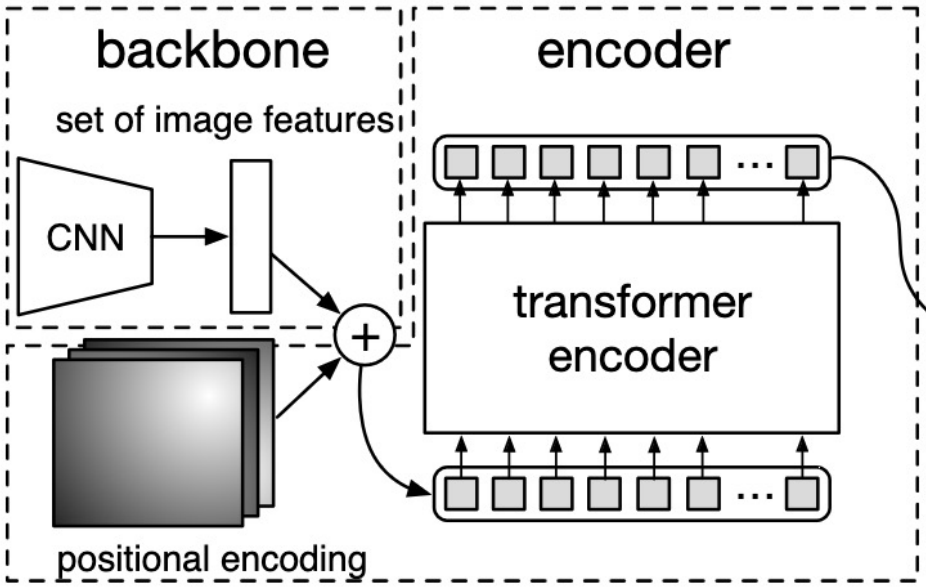


[Carion, 2020] **End-to-end object detection with transformers**, Carion, N. et al., *ECCV, 2020*

DETR for object detection

- The $H \times W \times C$ CNN output is passed through a **transformer encoder**, after projecting it to $d < C$ dimensions and adding positional encodings

- The encoder outputs summarize the visual information from the scene in $H \times W \times d$ tokens

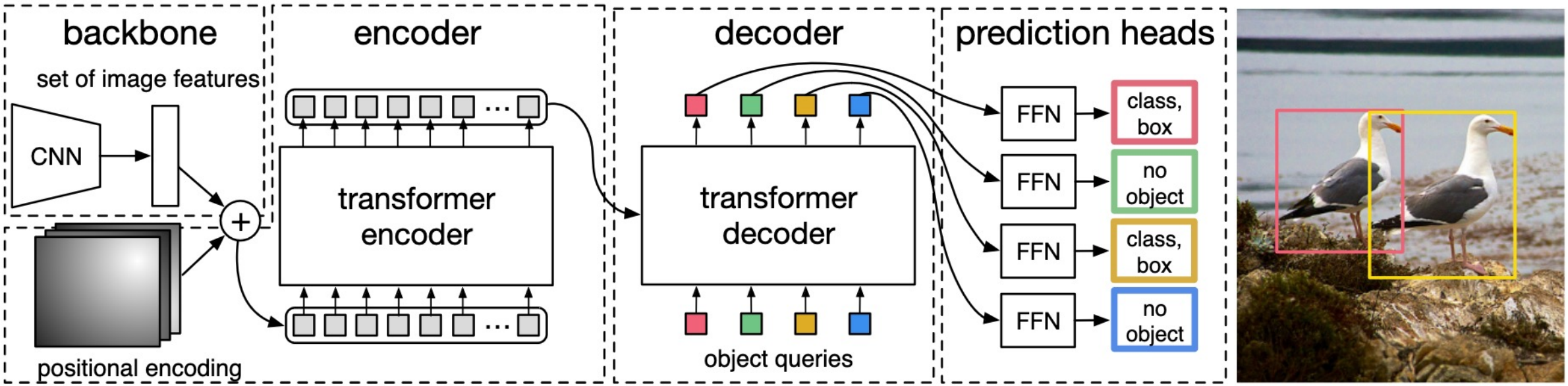


[Carion, 2020] **End-to-end object detection with transformers**, Carion, N. et al., *ECCV, 2020*

DETR for object detection

- A **transformer decoder** takes as input N distinct "object queries" and outputs N embeddings
  - Object queries are actually learnable positional embeddings
  - The decoder, like in [Vaswani, 2017], uses cross-attention to incorporate visual information from the encoder

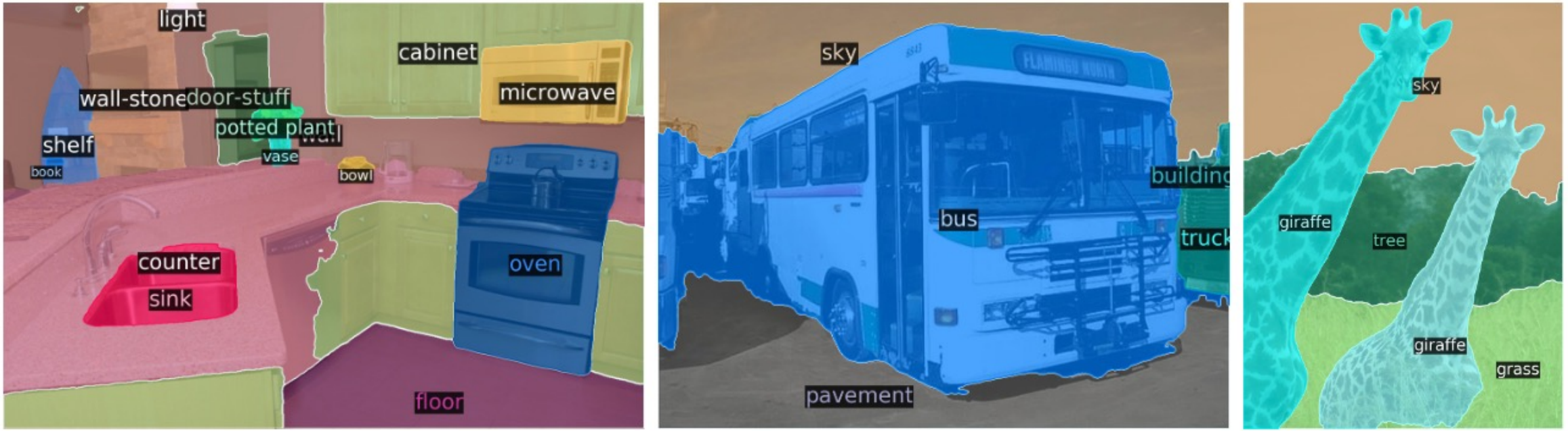
- The N embeddings are processed by two MLP heads to get bounding box coordinates and classes



[Carion, 2020] **End-to-end object detection with transformers**, Carion, N. et al., *ECCV, 2020*

DETR for object detection

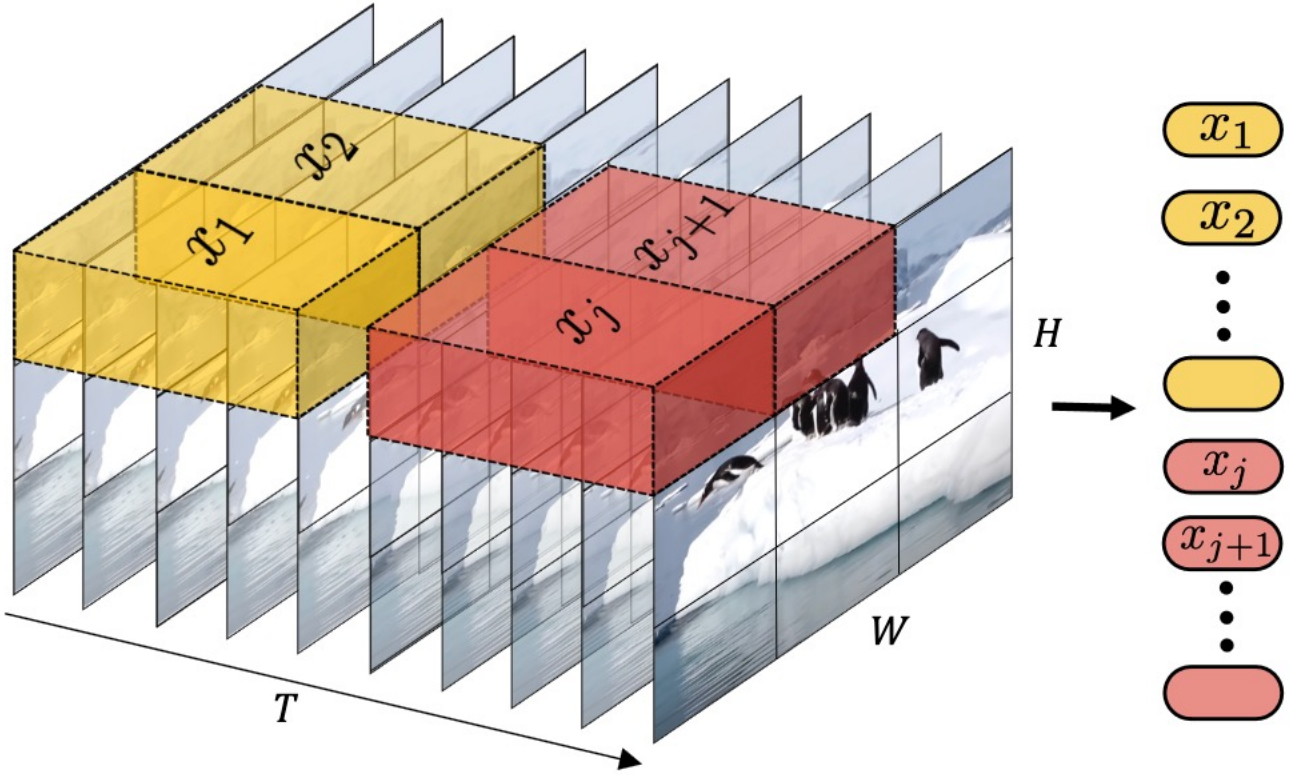
- DETR's framework can be extended to perform **panoptic segmentation**, i.e. semantic segmentation of the whole scene, but with each instance of the same class separately labeled



[Carion, 2020] **End-to-end object detection with transformers**, Carion, N. et al., *ECCV*, *2020*

Action recognition in videos

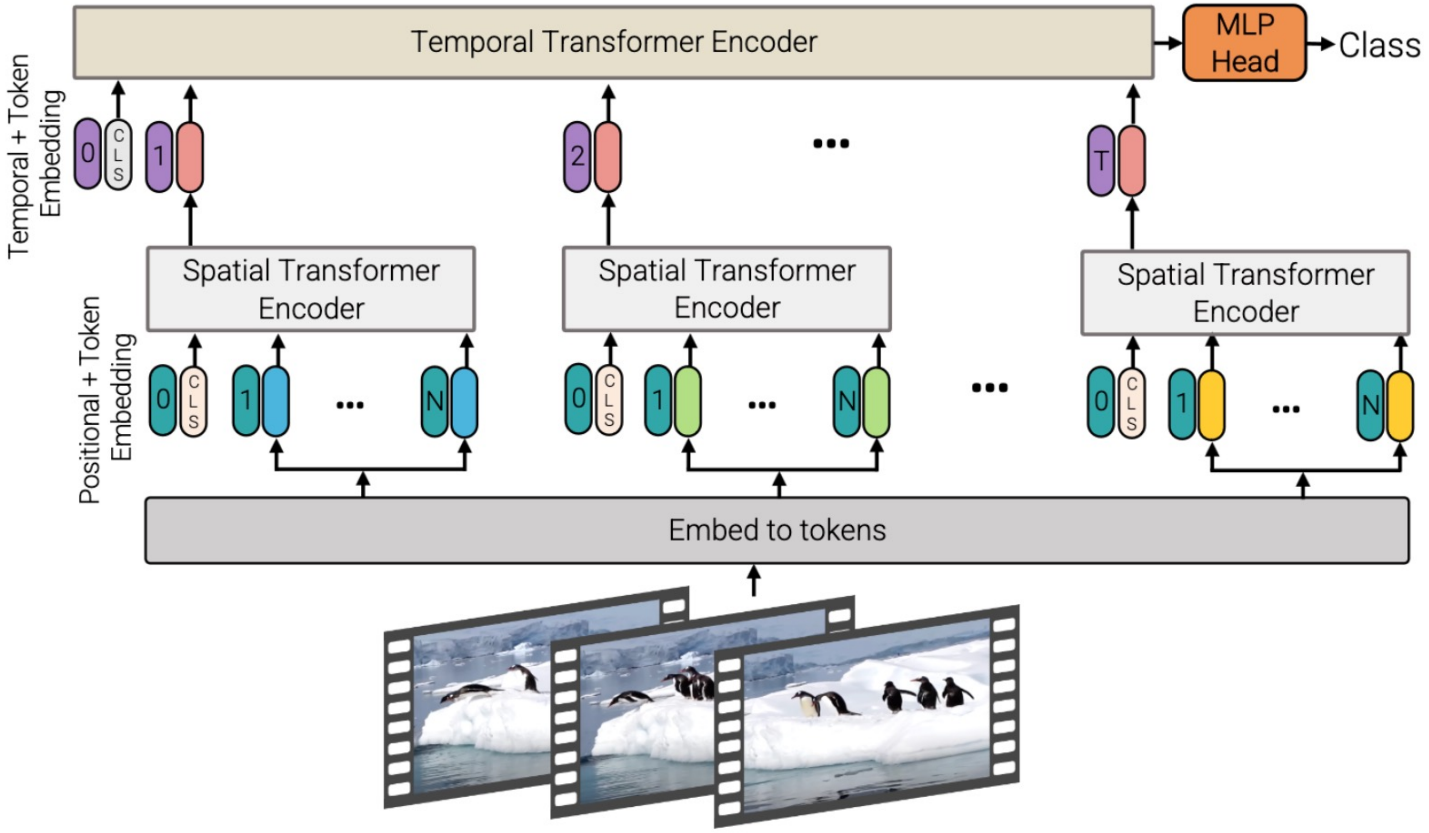
- ViViT [Arnab, 2021] extends vision transformer architectures to **$3D$ (video) inputs** for action recognition
  - It proposes “tubelet embedding”: forming tokens out of 3D spatio-temporal patches of videos



[Arnab, 2021] **Vivit: A video vision transformer**, Arnab, A. et al., *ICCV*, *2021*

Action recognition in videos

- ViViT processes several variants of vision transformers to process spatio-temporal tokens, including:
  - An unfactorized model with spatio-temporal attention
  - A factorized model, with spatial encoding first, then temporal encoding (shown below)



[Arnab, 2021] **Vivit: A video vision transformer**, Arnab, A. et al., *ICCV*, *2021*

Generative modeling with pure transformers

- TransGAN [Jiang, 2021] and ViTGAN [Lee, 2022] build on the **GAN framework** for image generation; but replace the CNN backbone with **pure-transformer backbones**



(a) Synthesized Image

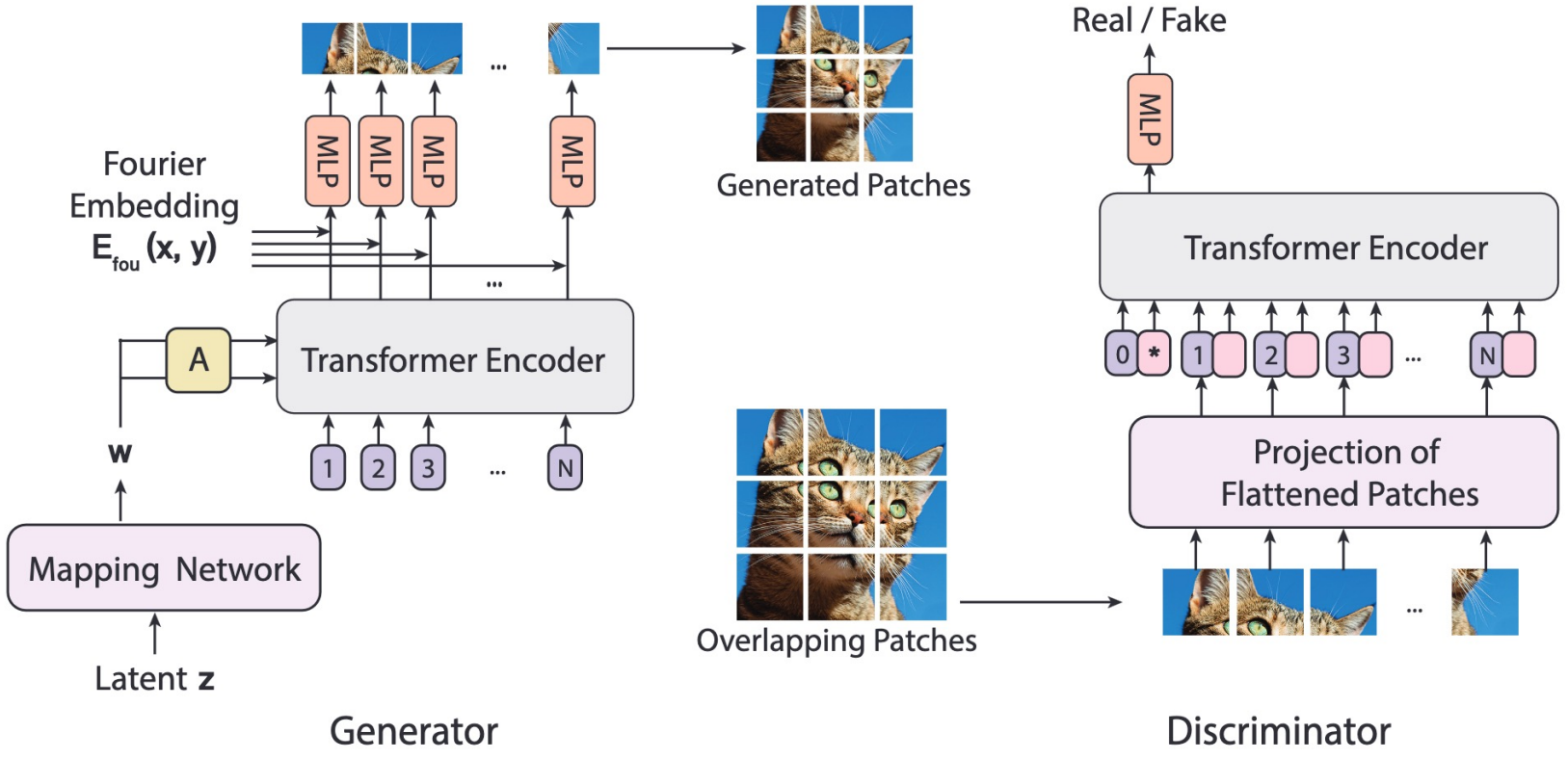
(b) Interpolation on Latent Space

Source: TransGAN paper

[Jiang, 2021] **Transgan: Two pure transformers can make one strong gan, and that can scale up**, Jiang, Y. et al., *NeuRIPS*, *2021*
[Lee, 2022] **Vitgan: Training gans with vision transformers**, Lee, K. et al., *ICLR*, *2022*

ViTGAN

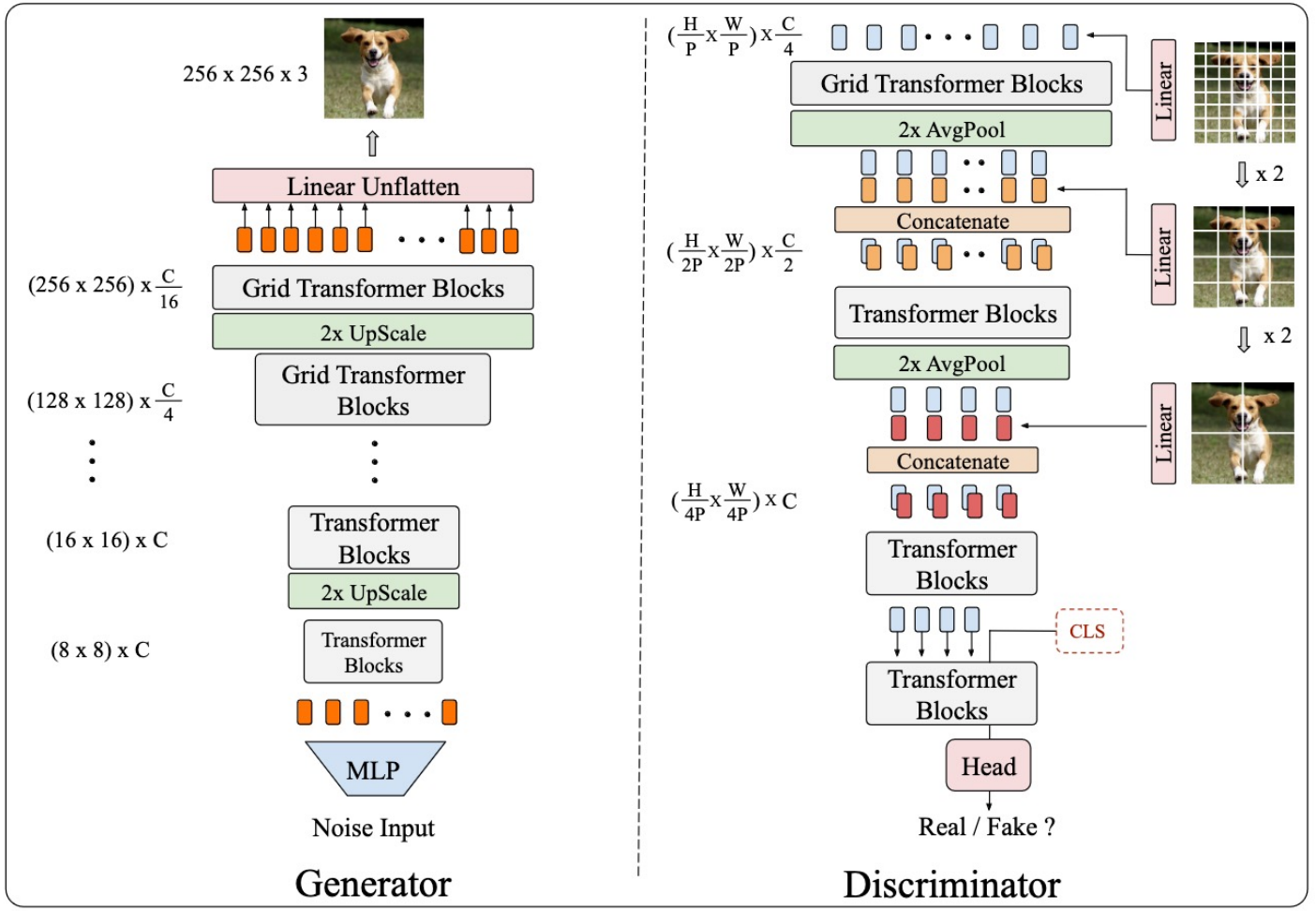
- ViTGAN is designed based on the original ViT, with a flat generator and discriminator design
  - The latent code information is incorporated via Self-Modulated LayerNormalization



[Lee, 2022] **Vitgan: Training gans with vision transformers**, Lee, K. et al., *ICLR, 2022*

TransGAN

- TransGAN uses a hierarchical design, like Swin Transformer does; hence it scales to higher resolution images



[Jiang, 2021] **Transgan: Two pure transformers can make one strong gan, and that can scale up**, Jiang, Y. et al., *NeuRIPS, 2021*

Generative modeling with pure transformers

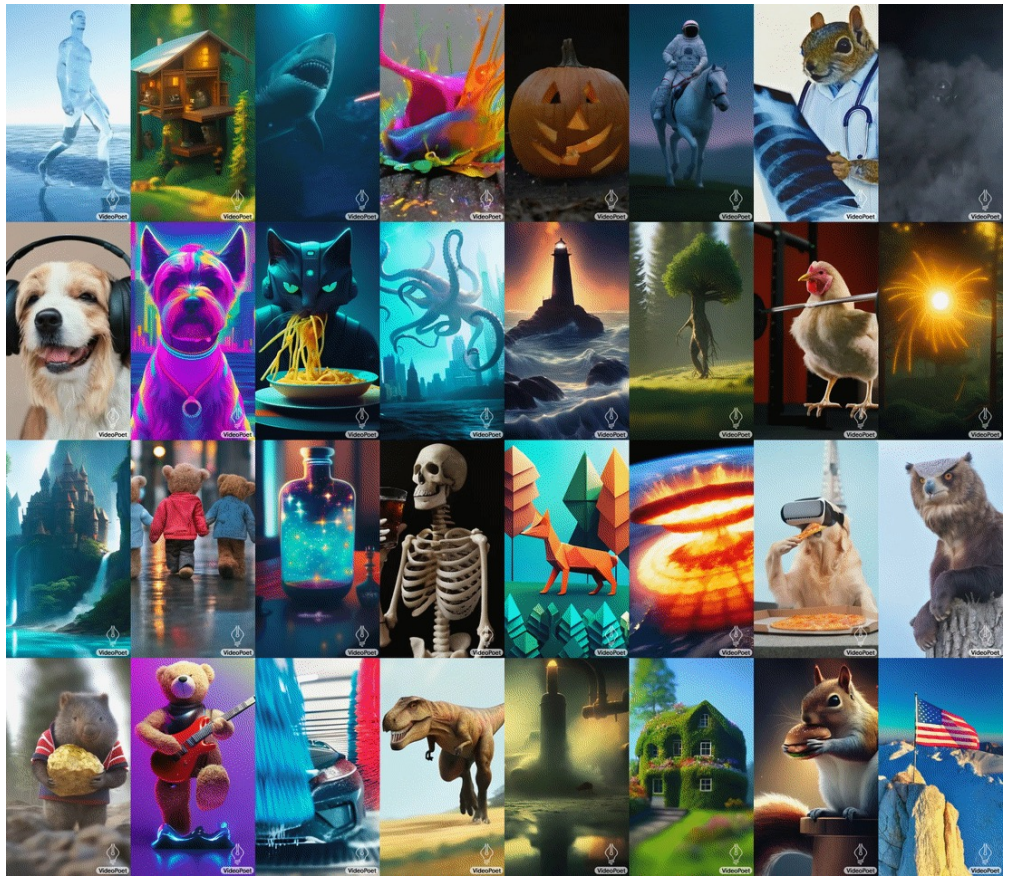
- **Diffusion Transformers** (DiTs) bring transformer backbones to **latent diffusion models**



[Peebles, 2023] **Scalable Diffusion Models with Transformers**, Peebles, W. et al., *CVPR, 2023*

Generative modeling for videos

- Both diffusion models and LMs have SOTA capabilities in video synthesis thanks to transformers (e.g., **MAGVIT**-v2, or of course **Sora 2** from OpenAI)

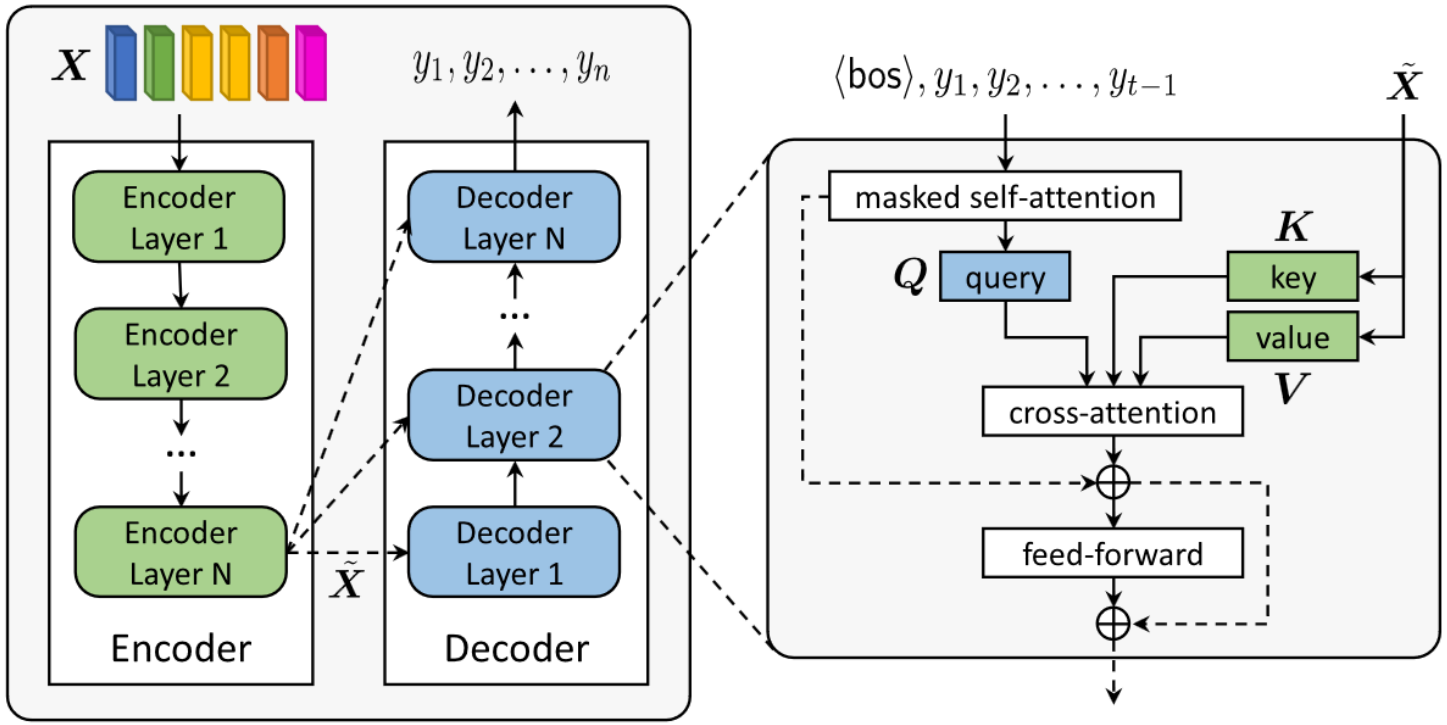


Source: Google, Carnegie Mellon University research

[Yu, 2024] **Language Model Beats Diffusion - Tokenizer is key to visual generation**, Yu, L. et al., *ICLR*, *2024*

Image-to-text modeling

- Several transformer models exist that take an **image as input** (in an encoder) and produce **text as output** (via a decoder) e.g., for **image captioning**

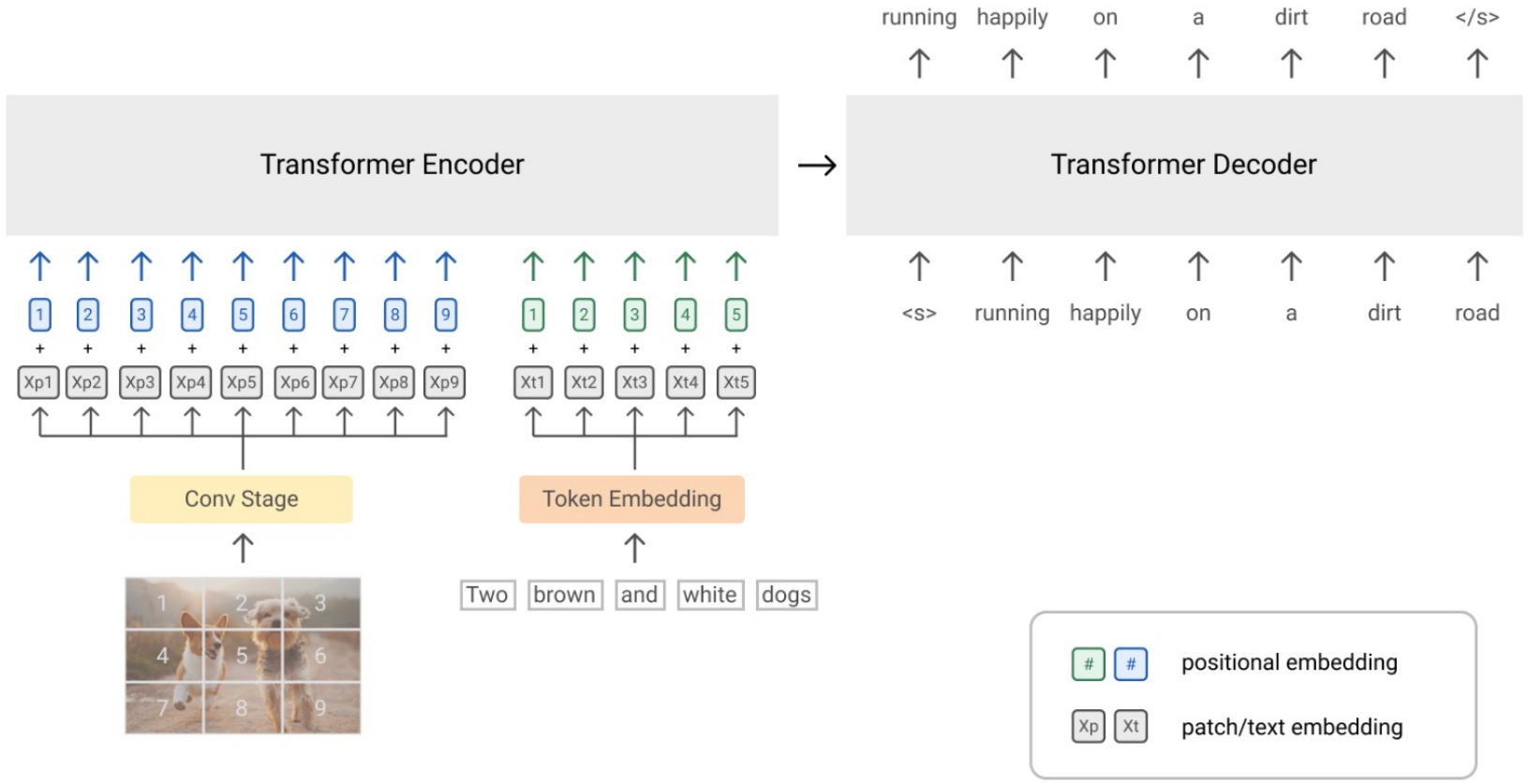


[Stefanini, 2022] **From show to tell: A survey on deep learning-based image captioning**, Stefanini, M. et al., *IEEE PAMI*, *2022*

SimVLM

- SimVLM [Wang, 2022] pretrains an image-text transformer model on a task of Prefix Language Modeling i.e., text generation given an image and a prefix

- The model has bidirectional contextualized representation of the prefix, as in MLM, but is trained for text-generation as in LM



[Wang, 2022] **SimVLM: Simple Visual Language Model Pretraining with Weak Supervision**, Wang, Z. et al., *ICLR*, *2022*

SimVLM

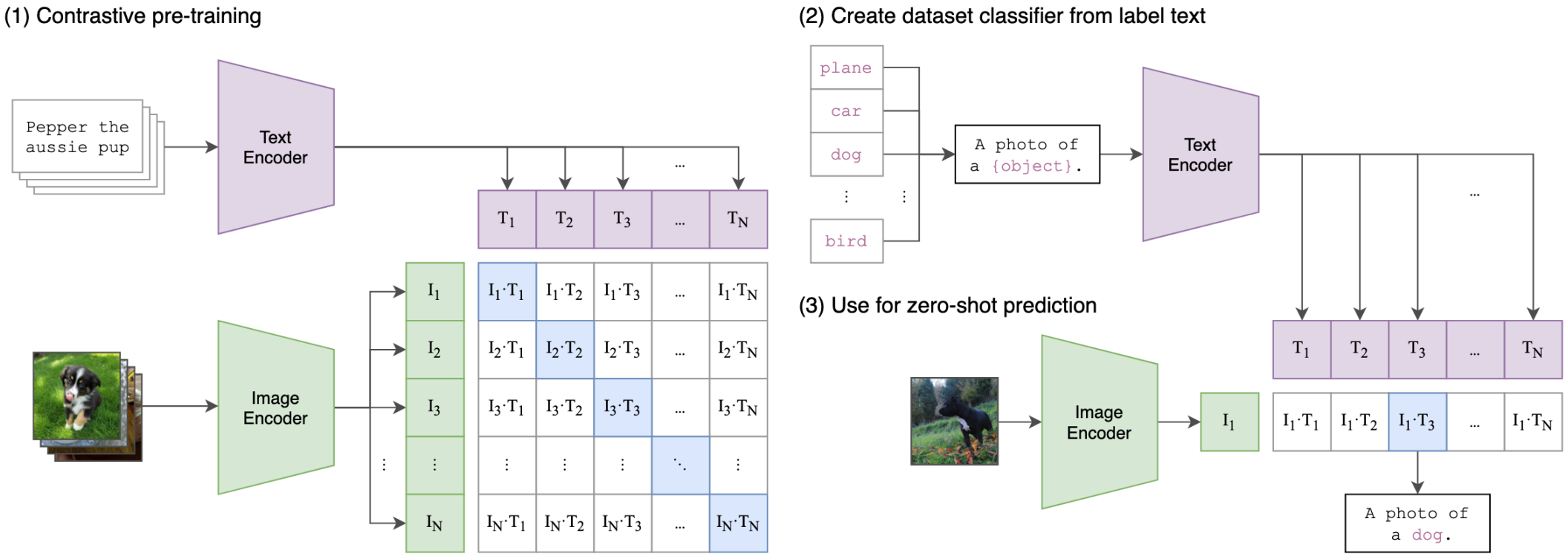
- It can be used for image captioning, visual question answering…



[Wang, 2022] **SimVLM: Simple Visual Language Model Pretraining with Weak Supervision**, Wang, Z. et al., *ICLR*, *2022*

CLIP: visual representations from NLP supervision

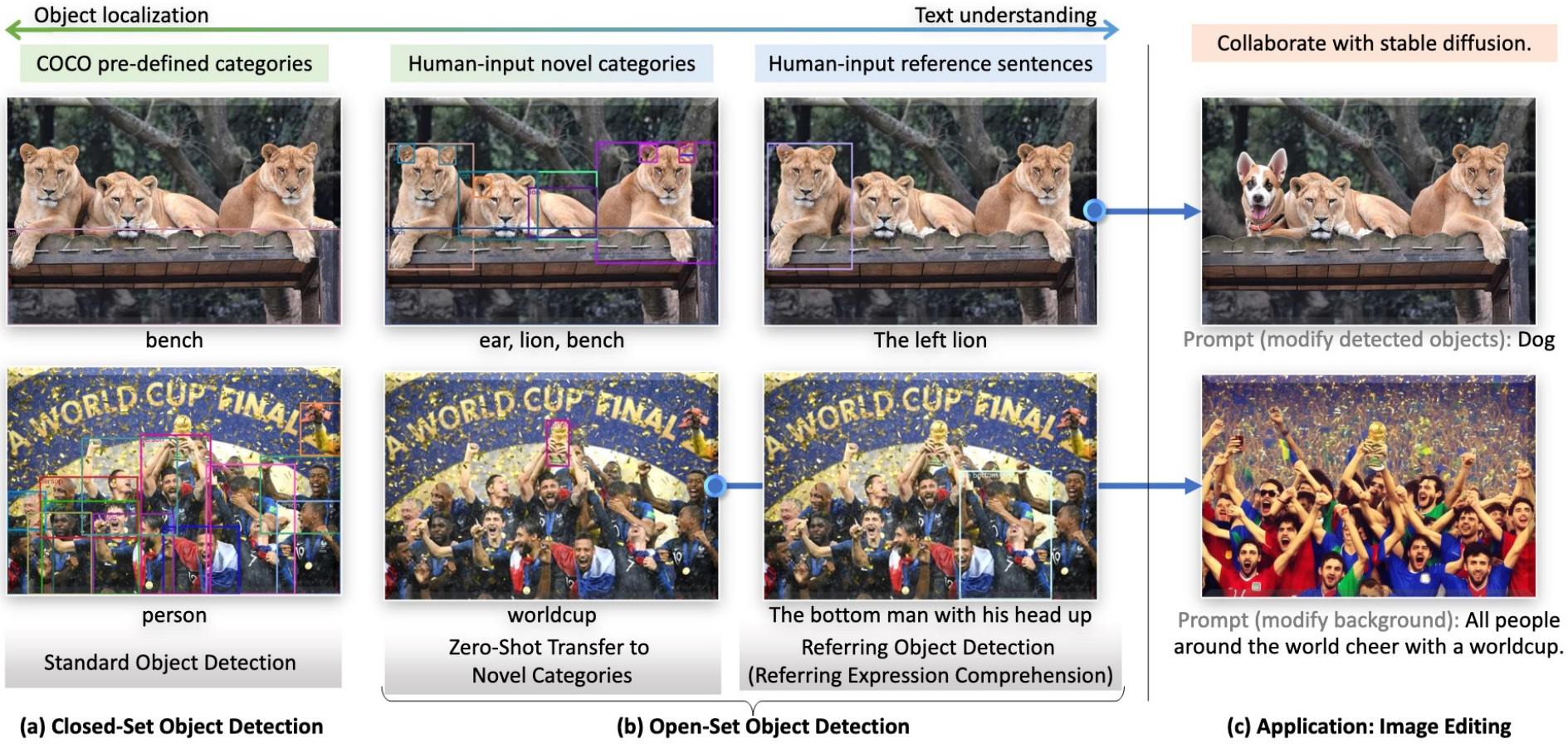
- CLIP [Radford, 2021] proposes **contrastive pretraining** of an **image encoder and a text encoder** from large (image, text) datasets
  - This enables **zero-shot learning** on many **vision tasks** via natural language references to visual concepts, for which it considerably raised the SOTA



[Radford, 2021] **Learning Transferable Visual Models From Natural Language Supervision**, Radford, A. et al., *ICML, 2021*

GLIP / Grounding DINO: Open-set object detection & phrase grounding

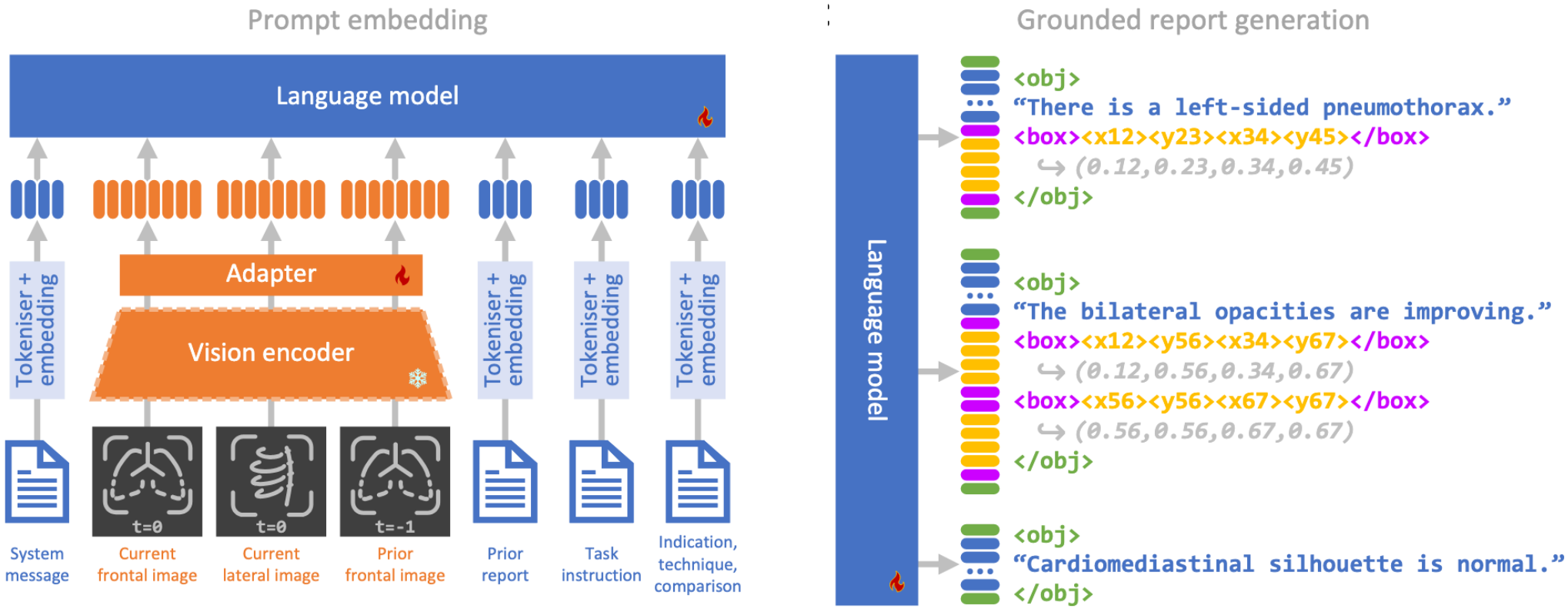
- Phrase **grounding** = detecting objects that match the noun phrases found in a text description
  - GLIP [Li, 2022], Grounding DINO [Liu, 2025], Grounded SAM… enable **open-set object detection** by aligning (open-set) noun phrases with object-level image concepts (bounding boxes)



[Li, 2022] **Grounded Language-Image Pre-training**, Li, L. H. et al., *CVPR, 2022*
[Liu, 2025] **Grounding dino: Marrying dino with grounded pre-training for open-set object detection**, Liu, S. et al., *ECCV, 2025*

Vision Language (Generative) Models & Radiological report generation

- A **vision encoder** encodes the image(s) as tokens, a **text tokenizer** encodes prior information and instructions as tokens; then a **language model** generates the radiological report



[Bannur, 2024] **MAIRA-2: Grounded Radiology Report Generation**, Bannur, S. et al., *arXiv:2406.04449 (2024)*

Summary and perspectives

- Vision Transformers **challenge the** CNN **state-of-the-art** in many vision domains/problems in the past 4 years

- They have **fewer inductive biases**, thus generally require more training data to shine;
- But vision transformers are **highly scalable** models with high top-end performance
  - This makes them suitable backbones for **foundation models**

- Much progress has been made w.r.t.:
  - How to best train ViTs?
  - What are the best design choices for attention & vision transformer architectures?

- Like CNNs, ViTs can be pre-trained without labeled data → you already know about **self-supervised learning** from previous lectures

- Transformers and **tokenization enable convergence of recipes** to process historically separate data types (audio, text, images, tabular data), which induces quick progress in **multimodal learning**